

PROJECT ESDM
ENTERPRISE ARCHITECTURE AND SYSTEM INTEGRATION DESIGN

NG YU HIN A23CS0418
CHAU YING JIA A23CS0213
CHERYL CHEONG KAH VOON A23CS0060
LAU YEE WEN A23CS0099
POH LOK YEE A23CS0262

UNIVERSITI TEKNOLOGI MALAYSIA

TABLE OF CONTENTS

	TITLE	PAGE
	TABLE OF CONTENTS	iii
	LIST OF TABLES	v
	LIST OF FIGURES	vii
CHAPTER 1	INTRODUCTION	1
1.1	Overview	1
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	Project Scope	3
1.5	Project Importance	4
CHAPTER 2	LITERATURE REVIEW	7
2.1	Introduction	7
2.1.1	Definition and Concepts of EA	7
2.1.2	Review of Previous Studies related to EA (University Enterprise Architecture)	8
2.2	Related Sub-System	9
2.3	Technology Used	11
2.4	Summary	12
CHAPTER 3	RESEARCH METHODOLOGY	15
3.1	Introduction	15
3.2	Chosen Methodology	16
3.3	Phases of Methodology	16
3.4	Project Planning	19
3.4.1	Gantt Chart	19
3.4.2	Major Project Milestones	20
3.5	Summary	21

CHAPTER 4	ANALYSIS AND DESIGN	23
4.1	Introduction	23
4.2	Company Organization Structure	24
4.3	Current System Analysis	24
4.4	Comparison between Existing and Proposed System	25
4.5	System Requirements	28
4.5.1	Functional Requirements	28
4.5.2	Non-Functional Requirements	29
4.6	System Design	31
4.6.1	Enterprise Architecture	31
4.6.2	System Architecture	32
4.6.3	Project Design Tools	33
4.6.3.1	Use Case Diagram	33
4.6.3.2	Use Case Description	34
4.6.3.3	Activity Diagram	55
4.6.3.4	Sequence Diagram	62
4.6.3.5	Database Design	70
4.6.3.6	Interface Design	71
4.7	Summary	105
CHAPTER 5	CONCLUSION	107
5.1	Introduction	107
5.2	System Contribution / Achievements	107
5.3	System Constraints	108
5.4	Future Works	109
5.5	Summary	110
	REFERENCES	111
	LIST OF PUBLICATIONS	114

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 4.1	Comparison Between UTHM Current Operational Ways and Our Proposed Integrated System	27
Table 4.2	Functional Requirements of Hostel Management System	29
Table 4.3	Non-Functional Requirements of Hostel Management System	30
Table 4.4	Description of Use Case View Room Availability	36
Table 4.5	Description of Use Case Select Room Preferences	37
Table 4.6	Description of Use Case Submit Room Application	37
Table 4.7	Description of Use Case Upload Supporting Documents	38
Table 4.8	Description of Use Case View Applications	38
Table 4.9	Description of Use Case View Applications	39
Table 4.10	Description of Use Case View Room Availability	40
Table 4.11	Description of Use Case Allocate Room	41
Table 4.12	Description of Use Case Update Room Assignment	42
Table 4.13	Description of Use Case Perform Digital Check-In/Check-Out	44
Table 4.14	Description of Use Case View Check-In/Check-Out Status	44
Table 4.15	Description of Use Case Submit Room Condition Checklist	44
Table 4.16	Description of Use Case Report Room Damage	45
Table 4.17	Description of Use Case Review Damage Report	45
Table 4.18	Description of Use Case Issue Fine	45
Table 4.19	Description of Use Case Send Maintenance Request	46
Table 4.20	Description of Use Case Generate Check-In/Check-Out Report	46
Table 4.21	Description of Use Case Set Check-In/ Check-Out Period	46
Table 4.22	Description of Use Case Review Early Check-In / Check-Out Request	47
Table 4.23	Description of Use Case Submit Maintenance Ticket	49
Table 4.24	Description of Use Case View Ticket Status	49

Table 4.25 Description of Use Case View Incoming Request	50
Table 4.26 Description of Use Case Assign Technician	50
Table 4.27 Description of Use Case Update Repair Status	51
Table 4.28 Description of Use Case View Outstanding Balance	52
Table 4.29 Description of Use Case Make Payment	53
Table 4.30 Description of Use Case View Payment History	53
Table 4.31 Description of Use Case Generate Report	54
Table 4.32 Description of Use Case Send Payment Reminder	54

LIST OF FIGURES

TABLE NO.	TITLE	PAGE
Figure 3.1	Waterfall Model	16
Figure 3.2	Project Planning Gantt Chart	19
Figure 4.1	UTHM Organisation Chart	24
Figure 4.2	Enterprise Architecture Diagram of UTHM Student Housing & Residential Services Management	31
Figure 4.3	System Architecture Diagram	32
Figure 4.4	Use Case Diagram for Hostel Management System	34
Figure 4.5	Use Case Diagram for Hostel Application Module	36
Figure 4.6	Use Case Diagram for Room Allocation Module	40
Figure 4.7	Use Case Diagram for Check In/ Check Out Module	43
Figure 4.8	Use Case Diagram for Maintenance Request Module	48
Figure 4.9	Use Case Diagram for Hostel Fee Monitoring Module	52
Figure 4.10	Activity Diagram for Hostel Application Module	56
Figure 4.11	Activity Diagram for Room Allocation Module	57
Figure 4.12	Activity Diagram for Hostel Check-In/Check-Out Module	58
Figure 4.13	Activity Diagram for Maintenance Request Module	60
Figure 4.14	Activity Diagram for Hostel Fee Monitoring Module	61
Figure 4.15	Sequence Diagram for Hostel Application Module	63
Figure 4.16	Sequence Diagram for Room Allocation Module	64
Figure 4.17	Sequence Diagram for Student Hostel Check In/ Check Out Module	65
Figure 4.18	Sequence Diagram for Admin Hostel Check In/ Check Out Module	66
Figure 4.19	Sequence Diagram for Maintenance Request Module	67
Figure 4.20	Sequence Diagram for Hostel Fee Monitoring Module	69
Figure 4.21	Database Design for Hostel Management System	70
Figure 4.22	Hostel Management System Student Portal Dashboard Page	71

Figure 4.23 Room Availability Page	73
Figure 4.24 Room Application Form	74
Figure 4.25 Submission Success Pop-Up Message	74
Figure 4.26 Upload Supporting Documents	74
Figure 4.27 Locked Main Page of Digital Hostel Check-In or Check-Out	75
Figure 4.28 Request Early Check-in or Check-out Pop-up Form	76
Figure 4.29 Note Update at Locked Check-in Page	76
Figure 4.30 Main Page of Digital Hostel Check-In or Check-Out When Access Opened	77
Figure 4.31 Identity Verification Successful Pop-up	77
Figure 4.32 Room Condition Checklist Page	78
Figure 4.33 Check-in or Check-Out Completed Page	78
Figure 4.34 Room Condition Checklist with Damage Report for Minor or Major Damage	79
Figure 4.35 Maintenance Request Form	80
Figure 4.36 Dropdown List of Issue Category Selection	80
Figure 4.37 Maintenance Request Dashboard	81
Figure 4.38 Ticket Details	81
Figure 4.39 Financial Status Page	82
Figure 4.40 Financial Status Page	82
Figure 4.41 Successful Payment Pop-up Message	83
Figure 4.42 Financial Status Page	83
Figure 4.43 The Official Receipt Generated After a Successful Payment	84
Figure 4.44 Notification Bar	84
Figure 4.45 Hostel Management System Admin Portal Dashboard Page	85
Figure 4.46 View Applications	87
Figure 4.47 Accept or Reject Applications	87
Figure 4.48 View Room Availability	88
Figure 4.49 Allocate Room Page	88
Figure 4.50 Allocate Room Form Pop-up Window	89

Figure 4.51 Room Allocate Successfully Pop-Up Window	89
Figure 4.52 Updated Room Allocation Page	90
Figure 4.53 Update Room Assignment Page	90
Figure 4.54 Update Room Assignment Form Pop-up Window	91
Figure 4.55 Assignment Update Successfully Pop-Up Window	91
Figure 4.56 Updated Room Assignment Page	92
Figure 4.57 Check-in and Check-out Page	92
Figure 4.58 Set Check-in and Check-out Period Pop-up Window	93
Figure 4.59 Early Check-in and Check-out Page	93
Figure 4.60 Student's Early Request Details Pop-up Window	94
Figure 4.61 Confirmed Request and Rejected Request Pop-up Window	94
Figure 4.62 Damage Report Page	95
Figure 4.63 Damage Report Details Page	95
Figure 4.64 Issue Fine Pop-up Window	96
Figure 4.65 Action Box after Issue Fine Complete	96
Figure 4.66 Send Maintenance Request Pop-up Window	97
Figure 4.67 Damage Report Details Page with All Action Completed	97
Figure 4.68 File Generate Successfully Pop-up	98
Figure 4.69 Maintenance Dashboard for Admin	99
Figure 4.70 Assign Technician Page	99
Figure 4.71 Pop-up Message: Successfully Assign the Technician	100
Figure 4.72 Status Update from Pending to In Progress	100
Figure 4.73 Update Repair Status Page	101
Figure 4.74 Pop-Up Message: Ticket Completed	101
Figure 4.75 Status Changed to Completed	102
Figure 4.76 Latest Dashboard Status	102
Figure 4.77 The List of All Students	103
Figure 4.78 The Summarized List of Paid Students	104
Figure 4.79 The Summarized List of Unpaid Students	104

CHAPTER 1

INTRODUCTION

1.1 Overview

Digital transformation plays an increasingly crucial role in modern higher education institutions as universities rely on multiple interconnected information systems to manage academic, administrative, and campus operations. Universities must manage huge volumes of data related to students, staff, facilities, and campus services, making system integration, data consistency, and efficient digital workflows are critical for effective operations.

This project focuses on analysis, design, and development of a proposed information system designed using enterprise architecture principles for a university environment. This study adopts Universiti Tun Hussien Onn Malaysia (UTHM) as a reference case to understand the structure, scale, and characteristics of a public technical university. UTHM provides a realistic example of an institution that operates several digital platforms, such as student and staff information systems, identity management system and mobile applications, which are commonly found in higher education institutions.

The proposed architecture addresses common challenges in university operations, such as fragmented systems, manual processes, and limited integration across services, while ensuring logical alignment with data sources and authentication mechanisms. By applying a structured software development methodology and utilizing SAP Build Apps, this project shows how a university-focused enterprise architecture framework can be designed and modelled as a practical exercise in system analysis and design. This approach helps illustrate how digital systems in universities can be organized to work more efficiently and in a more integrated way.

1.2 Problem Statement

Many higher education institutions operate multiple information systems to manage their academic activities, administrative work and campus services. Over time, these systems are often developed separately to solve some functional needs, which results in a collection of systems that do not communicate well with one another. This situation makes it difficult to achieve seamless information flow across the organization.

In many university settings, operational processes are still affected by system fragmentation and manual processes. Data related to students, facilities, and services may be stored in different systems or managed through separate platforms such as forms and spreadsheets. This leads to duplicate data entry, inconsistent records, and increased workload for administrative staff. The lack of integration also limits real-time visibility, making it difficult for admins to monitor operations and respond efficiently when problems arise.

These issues reduce the effectiveness of digital transformation initiatives and affect the overall quality of university services. Without a clear enterprise-level system structure, decision-making becomes slower, information access is reduced, and achieving operational efficiency becomes difficult. Therefore, it is necessary to identify and address these existing inefficiencies at the enterprise level to support a more integrated, efficient, and sustainable university digital ecosystem.

1.3 Project Objectives

The objectives of the project are:

The overall project aims to enable students to design an enterprise-level system by applying enterprise architecture principles, software development methodology, and appropriate technologies such as SAP and web framework. Based on the identified gaps in UTHM's digital ecosystem, the Hostel Management System seeks to achieve the following specific, measurable objectives

Design an integrated system that centralizes key residential student processes into a unified digital platform.

Define logical integration points with UTHM's existing master data sources, specifically demonstrating how the system would retrieve student status from the Student Information System (SMP) and link fee monitoring to the Finance system.

Develop a functional prototype using SAP Build Apps that demonstrates the core workflow logic to solve real operational problems within the UTHM environment.

1.4 Project Scope

The scope of this project is divided into two distinct levels: the design of the Enterprise Architecture (EA) and the specific development of the Hostel Management Subsystem. In terms of system analysis and design, the project includes the formulation of a comprehensive Enterprise Architecture (EA) for Universiti Tun Hussein Onn Malaysia (UTHM). This encompasses the modeling of the Business, Data, Application, and Technology layers to illustrate how various departmental systems, such as the Student Information System (SMP) and Staff Information System (SMS) to interoperate within a unified ecosystem. The architectural design serves to identify shared data entities and process flows to ensure organizational alignment across the institution.

For the system implementation phase, the technical scope is focused specifically on the Hostel Management Subsystem. Using SAP Build Apps as the development platform, a functional prototype will be created to handle core residential operations, including student accommodation booking, room allocation, and facility maintenance reporting. This module will also demonstrate logical integration capabilities by simulating data retrieval from SMP and finance system to verify student eligibility and fee status before processing applications.

Several aspects are explicitly excluded from this project to maintain manageability. The project does not involve the full-scale development of the university's entire ERP suite; systems other than the Hostel Management module are only conceptualized in the architecture. Furthermore, the prototype will not support real-time monetary transactions via banking gateways or integrate with physical hardware such as IoT sensors or biometric access controls.

1.5 Project Importance

This project overcomes the operational inefficiencies and data silos present in UTHM's current fragmented digital environment by developing a comprehensive Enterprise Architecture (EA) framework. Instead of using isolated applications, this project shows how to integrate different operational domains such as academic, residential, and administrative services into one single ecosystem that is interrelated with each other. This could be the best approach to solve the systemic fragmentation that currently hinders university overall operations.

Our project provides data consistency and integrity across the whole university by defining logical integration points with the central master data sources, such as SMP and Finance system. This architectural alignment prevents data silos from happening and ensures information such as student status and financial records remains in sync in real time across all departments. As a result, it can reduce administrative errors as redundant manual entry is now being minimal or eliminated.

The development of the Hostel Management System is crucial proof of concept within this enterprise framework. It shows how different operational subsystems can be upgraded and integrated into the broader university architecture. By centralizing residential processes within this connected ecosystem, we get to enhance students' accommodation yet provide a scalable model to integrate future digital services seamlessly to reach UTHM's strategic vision of a fully integrated Smart Campus.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing literature related to enterprise architecture and digital systems in higher education institutions. Universities operate in a complex environment that involves academic administration, campus service facility management and student support systems. As digital transformation becomes more important, the previous research has widely explored the need for structured approaches to managing and integrating these systems.

The literature review in this chapter is grouped into three main areas. The first focuses on enterprise architecture and its application in higher education institutions such as universities. The second review related to the subsystems commonly implemented in the market. The third review discusses the technologies used to develop and integrate enterprise information systems.

By reviewing these areas, this chapter provides a theoretical foundation for the proposed system design discussed in later chapters. The insights gained from the literature help justify the choice of enterprise architecture as an approach for this project and support the selection of a specific sub-system for further analysis and implementation.

2.1.1 Definition and Concepts of EA

Enterprise Architecture (EA) is a management tool that provides a useful plan for the structure and functioning of an organization. It acts as a linkage between business strategy and information technology, such that information technology is then optimized to support organization business strategies to achieve its goals. Enterprise

Architecture connects between organization's business processes, data flows, and technical systems and enables them to standardize operations and navigate complex digital transformations effectively. As described by Dumitriu and Popescu, EA serves as a complete expression of the modern organization, a holistic plan which acts as a collaboration force between objective, vision, strategy and governance principles (Dumitriu & Popescu, 2020).

Enterprise Architecture is separate into four fundamental architectural layers to achieve a holistic alignment and centralizes governance. These layers could be categorized into four fundamental architectures. First, the Business Architecture, which outlines the business strategies and key processes. Data Architecture that describes the organization's logical and physical data assets. The Application Architecture which describes the software solutions that support the execution of key business processes. Lastly, the Technology Architecture which defines the hardware and network infrastructure that need to support these applications. Through this layered framework, the organization could establish a centralized architectural strategy that represents the structure of the business, its IT landscapes, and their interrelations (Brée & Karger, 2022).

2.1.2 Review of Previous Studies related to EA (University Enterprise Architecture)

The implementation of Enterprise Architecture (EA) in higher education is increasingly recognized as a key approach to align complex business processes with information technology infrastructure. University as large academic organizations might still use the "legacy systems". Many operational areas still function in silos or rely on fragmented systems, leading to data duplication, inconsistent workflows, and manual interventions (Supriadi et al., n.d.). A structured EA framework, such as one based on the Zachman Framework, provides an essential blueprint for integrating these heterogeneous applications into an integrated system.

Since connecting these separate systems is essential, choosing the right framework to guide the process is key to success. The integration of all these needs may now be based on choosing an appropriate framework, which becomes crucial for the success factor. Most university prefer The Open Group Architecture Framework (TOGAF) due to its detailed Architecture Development Method (ADM) (Meutia et al., 2022). This framework works effectively for universities as it provides a flexible yet a thorough methodology in mapping out the current and future states of architecture. Both the TOGAF and ADM offers a systematic step-by-step approach that helps universities which lack of deep EA expertise to successfully plan and integrate complex academic information systems (Ulmi et al.,2020). It is important for handling the completely different data requirements of departments like research and grant management, libraries, and finance all in one system.

Furthermore, the practical application of these frameworks also concerns the optimization of particular IT services which form the backbone of any university nowadays. Batmetan illustrates how the creation of a strategic EA plan helps universities to align their information systems with business strategies, such as enhancing the quality of services between students and staffs (Batmetan, 2022). By employing tools like the Value Chain and SWOT analysis within the TOGAF model, universities can formulate a comprehensive strategic plan that encompasses data, application, and technology architectures to develop a more responsive and efficient digital environment.

2.2 Related Sub-System

A Hostel Management System is a critical subsystem within a university's digital ecosystem, designed to streamline the complex operations of student accommodation. In a large institution like Universiti Tun Hussein Onn Malaysia (UTHM), the management of residential colleges involves handling thousands of student applications, room allocations, fee tracking, and facility maintenance. An effective Hostel Management System functions not merely as a standalone record-keeping tool, but as an integrated platform that connects residential operations with the university's broader academic and financial data. The primary objective of this

system is to replace manual dependencies with automated workflows, ensuring that students have seamless access to accommodation services while providing administrators with real-time visibility into occupancy and facility conditions.

Several public universities in Malaysia have successfully transitioned from manual processing to integrated residential systems, serving as benchmarks for this project. Universiti Teknologi MARA (UiTM) utilizes the "e-Kolej" system, which was implemented to replace manual registration processes. A study by Redzuan (2016) found that the system improved individual staff performance and user satisfaction by streamlining operational tasks. This validates the project's objective to replace manual room applications with a unified digital platform. Similarly, research on Universiti Utara Malaysia (UUM) residential halls proves that efficient facility maintenance is the most important factor for ensuring student satisfaction (Abdul Kahar et al., 2017). Furthermore, recent reviews of UUM's facilities management suggest a growing necessity for digitalization to overcome issues of fragmented information and delayed responses (Hashim, 2025). These examples confirm that a modern Hostel Management System must not only handle room bookings but also integrate a responsive maintenance tracking module.

Despite UTHM's digital advancements, residential management remains fragmented, often relying on manual methods or disconnected tools like Google Forms and Excel. This isolation creates data silos where hostel records do not synchronize with the central Student Information System (SMP), resulting in data duplication. Consequently, wardens cannot verify student status or fee payments in real-time, leading to administrative bottlenecks and potential errors during room allocation.

The proposed Hostel Management System aims to resolve these inefficiencies by developing a centralized, web-based solution compatible with UTHM's enterprise architecture. To achieve this, the subsystem encompasses five key modules: Room Application, Room Allocation, Check-In/Check-Out verification, Maintenance Request tracking, and Hostel Fee Monitoring. Unlike the existing manual methods, the proposed system is designed to simulate logical integration points with UTHM's master data. It will utilize the Student Information System (SMP) to validate student eligibility and the Finance System to check payment status, thereby ensuring that room

allocation is both automated and compliant with university policies. This integrated approach not only enhances the user experience for students but also provides a scalable model for modernizing UTHM's residential operations.

2.3 Technology Used

This section describes and justifies the key technologies choices that can be selected for the development and conceptual integration of the proposed Hostel Management System, ensuring alignment with the Enterprise Architecture (EA) goals of the project.

SAP Business Technology Platform (BTP)

SAP BTP provides the foundation for UTHM's intelligent enterprise by functioning as a unified Integration Platform as a Service (iPaaS). Through the SAP Integration Suite, the Hostel Management System can seamlessly connect with legacy Student Information (SMP) and Finance systems, ensuring real-time data synchronization. To handle high-volume operations, SAP HANA Cloud manages data at scale, providing the performance necessary to track room occupancy and fee payments for thousands of students. Unlike standalone tools, BTP enables developers to seamlessly integrate and extend the Hostel Management System by connecting it to legacy SMP and Finance systems through the SAP Integration Suite. This ensures that administrative data flows consistently across the university's existing infrastructure.

SAP Build Apps (Low-Code/No-Code Platform)

The technology that can be chosen for prototyping the front-end user interfaces (UI) and system workflow logic is SAP Build Apps. SAP Build Apps is a low-code/no-code development platform that allows users to create enterprise-grade applications quickly. It supports the design of modern user interfaces and the modeling of complex business logic, such as approval workflows and notifications. The platform is justified because it allows for the rapid development of a functional prototype that clearly demonstrates the system's design and functional requirements within the project

timeframe. Furthermore, using an SAP-aligned technology provides relevant exposure to industry-leading enterprise software environments.

Microsoft Power Apps

Microsoft Power Apps is a versatile low-code platform that is a primary alternative to SAP Build Apps, particularly for organizations heavily invested in the Microsoft 365 and Azure ecosystems. It excels at rapid prototyping through its "Canvas Apps," which provide complete control over the user interface, and "Model-Driven Apps," which automatically generate UIs based on an underlying data structure. The platform's greatest strength lies in its vast library of pre-built connectors, allowing the Hostel Management System to easily integrate with common data sources like SharePoint, SQL Server, and Microsoft Teams. Microsoft Power Apps is justified as an alternative because of its "Common Data Service" (Microsoft Dataverse), which acts as a single source of truth for the enterprise. This makes it an ideal choice for the UTHM environment, where integrating fragmented data from disparate departments is a key project objective.

2.4 Summary

Enterprise Architecture provides a structured approach for aligning organizational strategies with information technology through clearly defined architectural layers, including business, data, application, and technology architectures. In higher education institutions, this approach is particularly important due to the complexity of academic, administrative, and campus service systems that must operate in an integrated and efficient manner. Previous studies have shown that the adoption of established EA frameworks, such as TOGAF, helps universities address challenges related to legacy systems, fragmented data, and siloed operations while supporting long-term digital transformation initiatives.

The discussion on university enterprise architecture highlights the importance of system integration and centralized governance in improving service delivery and

operational efficiency. Effective EA implementation enables institutions to standardize processes, enhance data consistency, and support decision-making across different departments. These principles are directly relevant to residential management systems, which require close coordination with academic and financial systems to ensure accurate and timely information flow.

The analysis of the Hostel Management System as a related subsystem demonstrates the need for a centralized and integrated solution to replace manual and disconnected tools. By logically integrating with existing master systems such as the Student Information System and Finance System, the proposed system supports automated workflows, reduces administrative bottlenecks, and improves the overall user experience for both students and administrators. The selection of SAP Build Apps and structured database modelling techniques further supports the development of a scalable and well-aligned system that conforms to enterprise architecture principles.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Introduction

This chapter outlines the methodology adopted for this project that serves as a strategic roadmap guiding the entire development lifecycle from planning to implementation. A well-structured methodology is essential to ensure that the project objectives are met and the final deliverable meets the requirements of the UTHM case study. This chapter discusses the rationale behind the selected development approach, followed by a detailed explanation of the phases involved in the operational framework and presents the project planning schedule used to monitor progress. This structure is ideal for this project as it ensures that the Enterprise Architecture design and system integration logic are fully finalized and documented before the actual development of the prototype using SAP Build Apps.

For the development of the Hostel Management System at UTHM, Waterfall model has been selected as our primary methodology. We choose Waterfall because there are already fixed requirements and defined academic milestones of our case study. Comparing to an Agile methodology which focuses on iterative development for evolving new requirements, the Waterfall approach follows a linear and sequential flow that is more suitable for our case. By strictly following this sequence, my team can minimize errors or uncertainties throughout the implementation stage, ensuring a robust and logical system design.

3.2 Chosen Methodology

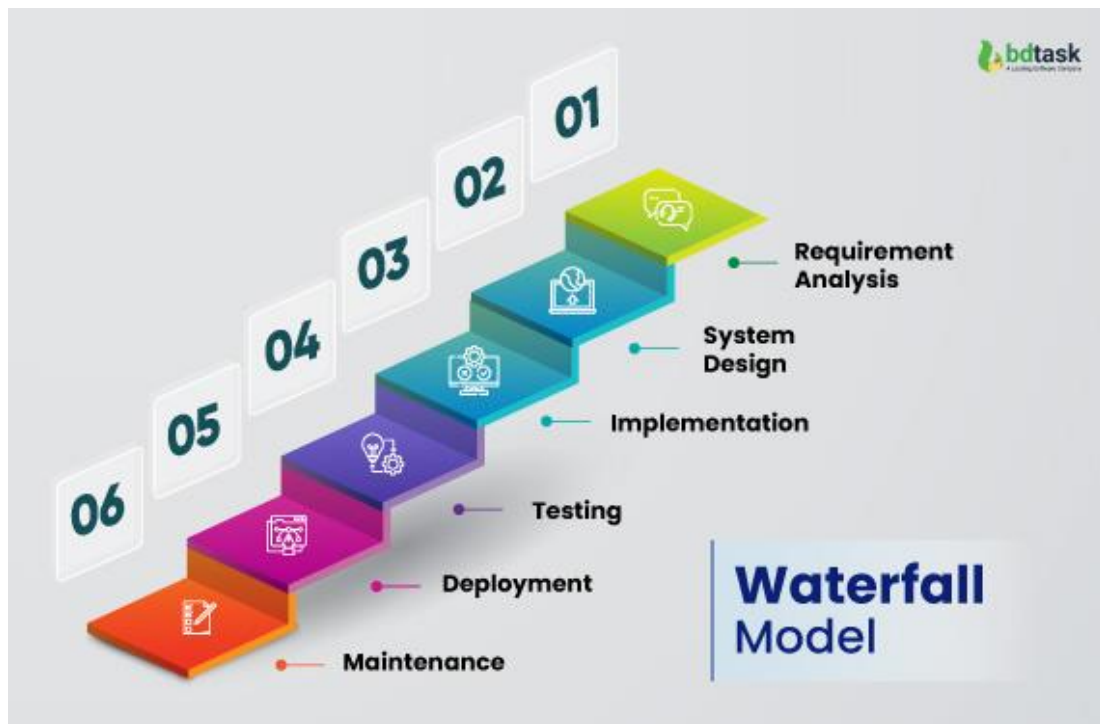


Figure 3.1 Waterfall Model

The methodology selected for the development of the Hostel Management System is the Waterfall System Development Life Cycle. This approach follows a linear and sequential path, where each phase must be completed and documented before the next phase begins. The Waterfall approach allows for a dedicated design phase to map these integration points before moving into the technical build. Each phase has specific entry and exits criteria, which helps the group stay on track to meet the final submission deadline.

3.3 Phases of Methodology

The Waterfall System Development Life Cycle are divides into several phase. Each phase produces a clear output serve as input for next phase. These approaches ensured that the requirement are well-defined and the project's goal are clear.

1. Phase 1: Requirement Analysis

This phase will examine the current digital environment of UTHM to understand how existing systems support daily academic and administrative operations. We will review systems such as student information system, financial system, staff management system and hostel related process to determine data management methods and workflows across departments. This analysis will help reveal several issues, including data duplication, limited integration, reliance on manual coordination and information update delays caused by independent systems. Based on these findings, we will define high-level system requirement and system boundaries at the enterprise level to ensure that the proposed solution meets the university's overall needs rather than focusing on individual or isolated system functions.

2. Phase 2: System Design

In this phase, the overall system architecture will be designed based on the requirements identified in the previous phase. Enterprise architecture principles will be applied to define the business layer, application layer, data layer and technology layer of the proposed university information system. The relationship between core systems such as Student Information System (SMP), Staff Management System (SMS) and other supporting systems will be clearly mapped to ensure better data flow and system integration. System models such as use case diagrams, activity diagram and sequence diagram will be developed to illustrate system interactions and workflows. This phase also includes the design of subsystem architectures to ensure consistency with the enterprise level architecture before proceeding to the system development phase.

3. Phase 3: Implementation

In this phase, the system design will be transformed into a working prototype based on the approved architecture and design specifications. Figma will be used to develop system interface and application logic, focusing on demonstrating how different systems can interact within an integrated enterprise environment. Core functionalities will be

implemented according to the define requirement and selected subsystems will be developed to demonstrate practical system integration. The implementation process will follow the design document closely to ensure consistency with the planned architecture and reduce development errors.

4. Phase 4: Testing

In this phase, testing activities will be conducted to verify that the developed system functions according to the specified requirement and design. Functional testing aims to ensure that key features operate as intended while integration testing focuses on validating data flow between different systems and subsystems. Testing also helps identify system errors, usability issues and workflow inconsistencies. Any problems discovered during testing will be documented and resolve to improve system reliability and overall performance.

5. Phase 5: Deployment

In this phase, the completed system is ready for deployment in the university environment. The system will be configured based on the defined architecture and made available to intended users. This includes setting user access permissions, ensuring system components are correctly connected and validating that data flows are functioning as expected. Although the system is a prototype developed by SAP Build Apps, the deployment phase demonstrates how the proposed enterprise architecture and its subsystems can be integrated into an operational environment. This phase marks the transition of the system from system development to actual system usage.

6. Phase 6: Maintenance

During this phase, we will conduct ongoing maintenance activities to ensure the system continues to operate efficiently after deployment. This includes monitoring system performance, fixing identified issues and making minor improvements based on user feedback or operational needs. Maintenance also includes updating system components to accommodate

changes in university process or policy. While full maintenance is not within the scope of this project, this phase highlights the importance of long-term system support for maintaining an integrated enterprise architecture.

3.4 Project Planning

This section presents the project planning schedule used to guide the development of the proposed Enterprise Architecture (EA) for the university environment. The project focuses on analysing and designing an integrated EA framework, while a Hostel Management System subsystem is developed as a proof-of-concept prototype to demonstrate the applicability of the proposed architecture.

The project was carried out from 3 December 2025 to 14 January 2026 and follows the Waterfall System Development Life Cycle, which is suitable due to the clearly defined scope and emphasis on architectural design and documentation rather than full system implementation.

3.4.1 Gantt Chart

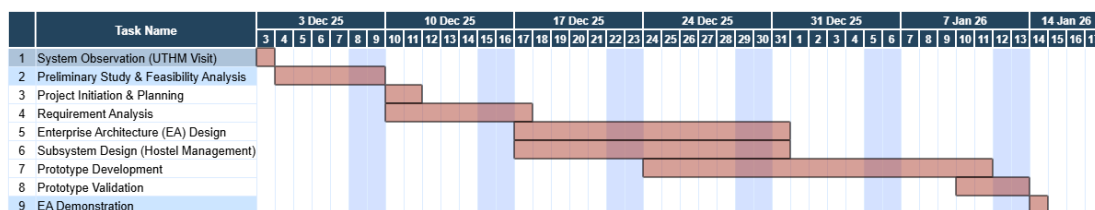


Figure 3.2 Project Planning Gantt Chart

The project timeline is illustrated using a Gantt chart, as shown in Figure 3.2, which outlines the sequence and duration of key project activities. The schedule prioritises enterprise-level analysis and architectural modelling before progressing to subsystem-level prototyping.

The project began with System Observation, including an on-site visit to UTHM on 3 December 2025, to understand the existing system landscape and

operational challenges. This was followed by a Preliminary Study and Feasibility Analysis to evaluate the suitability of applying Enterprise Architecture principles within the university context.

Subsequently, Project Initiation and Planning were conducted to define the project scope, objectives, and deliverables. Requirement Analysis focused on identifying enterprise-wide issues such as system fragmentation, data redundancy, and lack of integration between subsystems.

The Enterprise Architecture Design phase forms the core of the project, during which the Business, Data, Application, and Technology architecture layers were developed to illustrate an integrated university system environment. Following this, Subsystem Design was performed to select and design a suitable subsystem as a representative proof of concept aligned with the proposed EA structure.

The Prototype Development phase involved the creation of a conceptual user interface prototype to demonstrate key workflows and system interactions of the selected subsystem. This prototype does not represent a fully functional system but serves to validate the feasibility and consistency of the proposed enterprise architecture. The project concluded with Prototype Validation and EA Demonstration, ensuring that the prototype aligns with the architectural models and project objectives.

3.4.2 Major Project Milestones

To effectively monitor progress and ensure timely completion, several major project milestones were identified throughout the development lifecycle. These milestones serve as control points aligned with the Waterfall methodology and the Gantt chart timeline.

1. Completion of System Observation and Feasibility Study (9 December 2025)
This milestone confirms a comprehensive understanding of the existing

university system environment and validates the adoption of an EA-based approach.

2. Finalization of Requirements (17 December 2025)

At this stage, enterprise-level requirements related to integration, data consistency, and process alignment were established.

3. Completion of Enterprise Architecture Design and Hostel Management System Design (31 December 2025)

This milestone represents the Business, Data, Application, and Technology architecture models were finalised and the completion of detailed system designs, including use case and sequence diagrams for the modules.

4. Prototype Development Completion (11 January 2026)

The functional prototype was successfully developed using SAP Build Apps, demonstrating the core workflows and system logic.

5. Prototype Validation and Submission Readiness (14 January 2026)

The prototype was reviewed to ensure consistency with the enterprise architecture and to demonstrate how the proposed EA can support subsystem implementation.

These milestones correspond directly to the phases illustrated in the Gantt chart and provide clear checkpoints for tracking project progress throughout the development lifecycle.

3.5 Summary

This chapter has detailed the research methodology and operational framework adopted for the development of the Hostel Management System at UTHM. The Waterfall System Development Life Cycle (SDLC) was selected as the primary methodology due to its structured, linear approach, which aligns well with the fixed

requirements and defined academic milestones of this case study. By strictly following the six distinct phases, from Requirement Analysis and System Design to Implementation, Testing, Deployment, and Maintenance, the project ensures that all architectural integration points are logically mapped before development begins.

Furthermore, the project planning section demonstrated how the timeline is managed using a Gantt Chart and specific Major Project Milestones. This rigid planning structure ensures that the Enterprise Architecture principles are correctly applied and the final prototype is delivered on time. Ultimately, the methodology outlined in this chapter serves as the strategic foundation for the Analysis and Design activities that will be discussed in the subsequent chapters.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter focuses on the detailed analysis and architectural design of the proposed **Hostel Management System** for Universiti Tun Hussein Onn Malaysia (UTHM). While the previous chapter defined the methodology, this chapter translates those theoretical plans into concrete system specifications and visual models.

The analysis phase begins by examining UTHM's current organizational structure and the existing manual processes used in residential colleges. This section identifies specific operational bottlenecks, such as data silos and manual redundancy, which serve as the basis for the system's functional requirements. Following this, the design phase presents the **Enterprise Architecture (EA)** layers— Stakeholders, Business, Data, Application, and Technology to demonstrate how the new subsystem aligns with UTHM's broader digital ecosystem.

Finally, this chapter documents the technical blueprints of the system using Unified Modelling Language (UML) diagrams. This includes Use Case diagrams to visualize user interactions, Sequence diagrams to map logical data flows, and Entity-Relationship (ER) diagrams to define the database structure. These diagrams provide the comprehensive foundation required to generate the high-fidelity prototype to demonstrate the proposed solution's interface and user experience.

4.2 Company Organization Structure

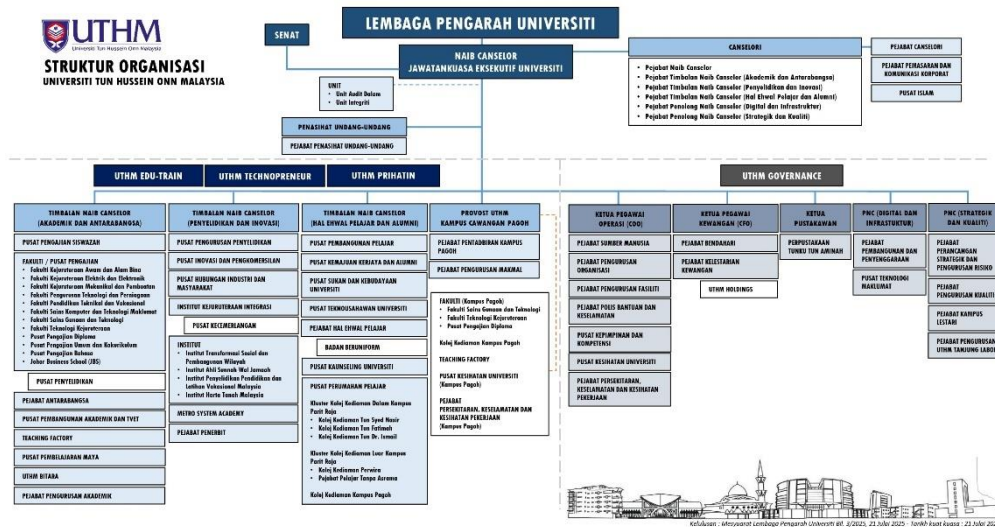


Figure 4.1 UTHM Organisation Chart

The organisational chart of UTHM as shown in Figure 4.1 above illustrates a hierarchical governance and operational structure that supports strategic leadership, academic management, and administrative functions across the university. At the highest level, the university is governed by a University Board of Directors and Senate, followed by the Vice-Chancellor, who oversees multiple senior leadership offices, including Deputy Vice-Chancellors responsible for key domains such as academic affairs, research and innovation, student affairs, and alumni relations. Under these leadership roles, various administrative divisions and units such as academic faculties, administrative departments, and support offices operate to deliver educational, research, and campus services. The structure is designed to enable clear reporting lines, coordinated decision-making, and integrated governance across university operations.

4.3 Current System Analysis

UTHM currently operates multiple system to support their academic, administrative and student service operations. These systems are developed at different

time to meet specific requirement, resulting in a digital environment where each system operates independently rather than as a fully integrated enterprise platform.

The Student Information System (SMP) serves as the core system for managing student biodata and academic record, while other systems support staff management, finance and campus services. Each system plays a specific role in supporting daily university operations and was designed to address functional needs.

Beyond the core system, UTHM operates several supporting systems such as hostel management, finance and payment systems, library service and IT support services. These systems are often developed separate solutions or only partially connected to core system such as Student Information System (SMP) and Staff Management System (SMS). As a result, information related to student accommodation, financial status or facility usage are stored across multiple systems, leading to data duplication and inconsistencies.

One of the key challenges facing the current system environment is system fragmentation. Data exchange between system is limited and update in one system are not always reflected in other systems in real-time. For example, changes in student status recorded in SMP may not automatically update to the hostel management system. In addition, several operational processes, including maintenance request, residential application and club management still rely on manual coordination, such as form submission, email communication and spreadsheet tracking. This increases administrative workload and slow down the service delivery, while also affecting the user experience for student.

Overall, UTHM's current system architecture highlights several challenges, including limited integration, data inconsistencies and inefficient workflows. These issues indicate a need for more structured enterprise architecture approach to improve system coordination and support more efficient university operations.

4.4 Comparison between Existing and Proposed System

This section evaluates the transition from UTHM's current operational ways to the proposed digital environment. At present, UTHM's digital ecosystem faces

challenges like system fragmentation and unintegrated, handles manually processes that lead to terrible data silos. The proposed Hostel Management System and the other subsystems are designed to address these specific gaps by implementing a centralized Enterprise Architecture (EA) framework that replaces isolated legacy systems with a unified digital platform. By shifting from standalone management to an integrated model, the proposed system ensures that residential operations are logically aligned with the university's master data sources, specifically the Student Information System (SMP) and the central Finance system. The following table provides a detailed comparison of the functional and structural improvements offered by the proposed system over the existing environment.

Operational Area	UTHM Existing Environment	Proposed Integrated System
Room Application	Often handled manually or through disconnected tools like Google Forms and Excel.	Centralized digital application module where students can apply through the application and view real-time status.
Room Allocation	Manual process where admins cannot verify student status in real-time, leading to administrative bottlenecks.	Automated allocation logic that retrieves student eligibility (check in/check out) directly from SMP master data.
Check-In / Check-Out	Relies on paper-based forms and manual checklists that remain unintegrated with the central system.	Digital verification system using QR codes for room access and digitized room condition checklists.
Maintenance Requests	Maintenance is often handled via google forms or paper-based systems,	Integrated a page in application that allows students to submit requests and enables admins to

	lacking a tracking mechanism.	assign technicians and track status.
Fee Monitoring	Hostel payment data is silo and does not synchronize with the central Finance system, causing data duplication.	Direct logical integration with the Finance system to monitor charges, payment statuses, and automated reminders.
Data Architecture	Characterized by "Data Silos" where student details are duplicated across fragmented systems.	Unified data model that eliminates duplication by leveraging UTHM's central SMP and SMS master tables.

Table 4.1 Comparison Between UTHM Current Operational Ways and Our Proposed Integrated System

4.5 System Requirements

The system requirements serve as the blueprint for the development of the Hostel Management System. These are categorized into **Functional Requirements**, which define what the system must do, and **Non-Functional Requirements**, which define how the system should perform.

4.5.1 Functional Requirements

The following table outlines the core functions of the system, grouped by the five key modules defined in the project scope.

Module	User	Description
Room Application	Student	• The system shall allow students to submit a hostel application by selecting specific preferences such as room type. • The system shall display the real-time status of the application (e.g., Pending, Approved, Rejected).
Room Allocation	Admin	• The system shall allow administrators to view a list of all available rooms and their current occupancy levels. • The system shall provide a function to auto-assign or manually allocate a student to a specific room based on eligibility.
Check-In / Out	Student	• The system shall generate a unique digital check-in/check-out form that includes a room condition checklist.

Maintenance	Student	The system shall allow students to report facility damages by selecting a category (Electrical, Plumbing, Furniture) and adding a description.
Maintenance	Admin	The system shall allow admin staffs to assign specific technicians to a maintenance ticket and update its status (e.g., In Progress, Completed).
Fee Monitoring	Student	- The system shall display the current semester's hostel fees and the payment status (Paid/Unpaid) by simulating data from the Finance System. - The system shall prevent room allocation approval if the "Fee Status" is flagged as "Unpaid".

Table 4.2 Functional Requirements of Hostel Management System

4.5.2 Non-Functional Requirements

The following table represents the non-functional requirement of the system. These requirements focus on how the system should perform rather than what the system does, covering aspects such as system performance, security, usability and availability.

Aspects	Description
Usability	The system should provide a user-friendly and intuitive interface, enabling student and administrative to complete task with minimal training.
Performance	The system should respond to user request within an acceptable time frame under normal usage conditions.

Security	The systems should restrict access based on the user role (Student, Admin) to prevent unauthorized operations or data access.
Availability	The system should remain accessible during standard university working hours and minimize downtime.
Reliability	The system should ensure that submitted applications, maintenance requests and allocation records are accurately stored and retrieved.
Compatibility	The system should be compatible with modern web browsers and support integration with existing university system

Table 4.3 Non-Functional Requirements of Hostel Management System

4.6 System Design

4.6.1 Enterprise Architecture

The following Figure 4.2 presents the Enterprise Architecture (EA) for the UTHM Student Housing & Residential Services Management. It provides a structured multi-layered blueprint aligning students' residential processes with UTHM's technical infrastructure.

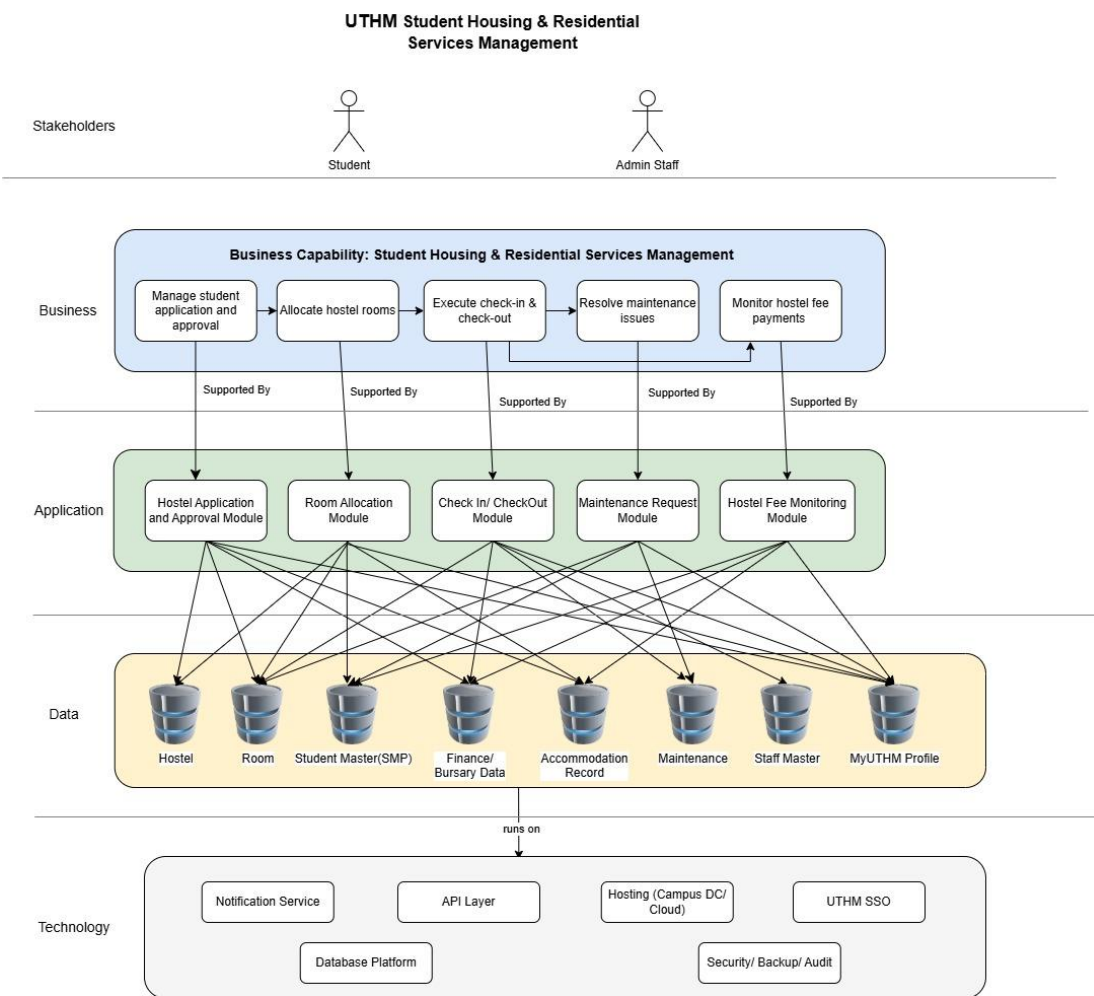


Figure 4.2 Enterprise Architecture Diagram of UTHM Student Housing & Residential Services Management

As illustrated in the diagram above, the Enterprise Architecture (EA) is organized into four distinct layers: Business, Application, Data, and Technology. The Business Layer defines core capabilities such as managing room applications,

executing check-in/check-out and monitoring fee payments. They are essential for university residential services. These business goals are supported by the Application Layer, which consists of specific software modules that students and admin staff interact with to complete their tasks. The Data Layer ensures logical integration by connecting these application modules to centralized data sources, such as the Student Master (SMP) and Finance records. With the appropriate centralized data layer, we could prevent data silos and ensure information consistency. Finally, the Technology Layer provides the underlying infrastructure, utilizing an API layer for cross-system communication and UTHM SSO to provide secure, unified authentication for all users.

4.6.2 System Architecture

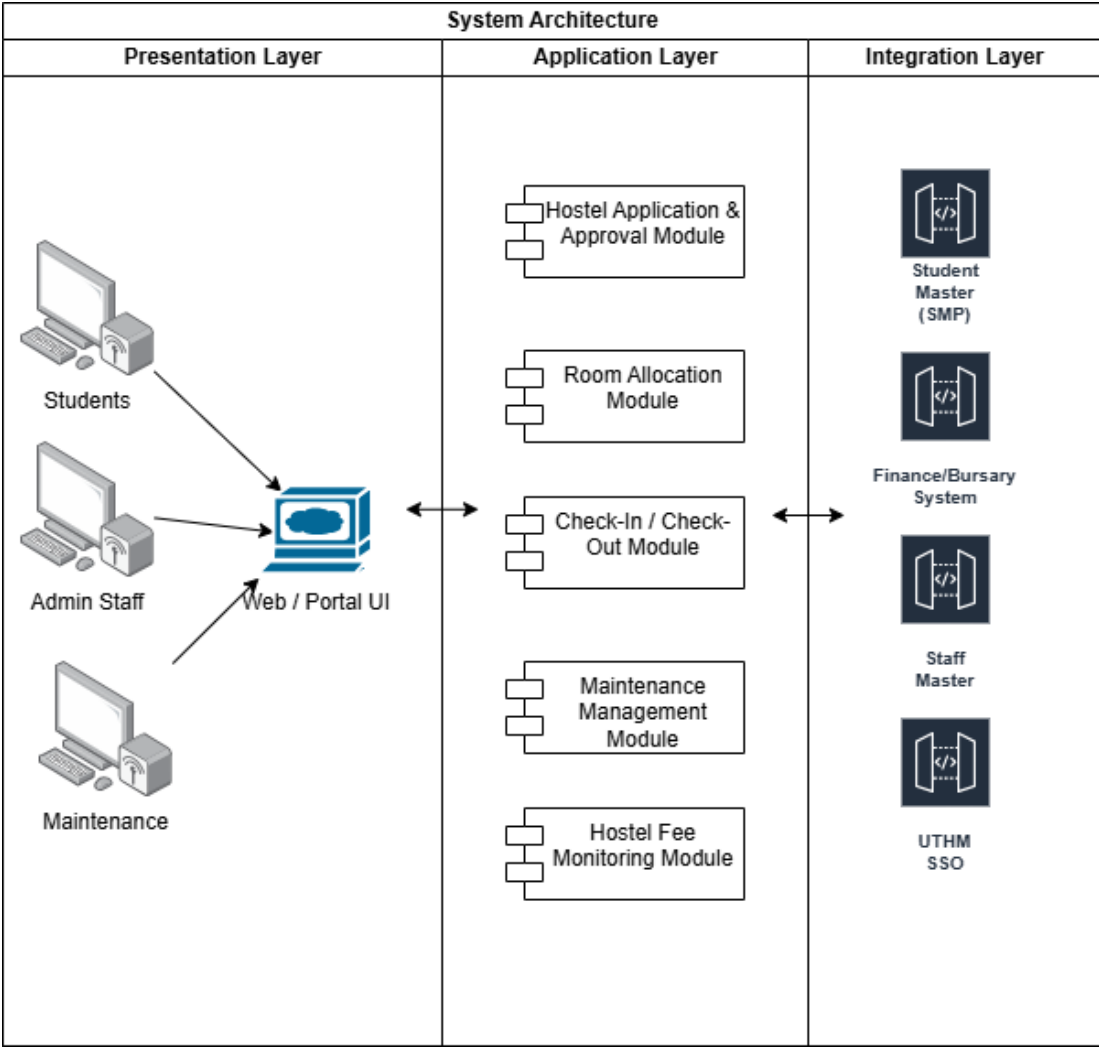


Figure 4.3 System Architecture Diagram

As Figure 4.3 shows, The System Architecture is a three-tiered, modular framework that guarantees smooth communication between centralized university data, business logic, and users. The main entry point at the Presentation Layer is a single Web/Portal UI that offers customized interfaces for maintenance staff, administrative staff, and students. The Application Layer, which serves as the system's functional center, receives user requests from this layer. Here, business rules and workflows necessary for residential management are processed by specialized modules, such as Hostel Application, Room Allocation, Check-In/Check-Out, Maintenance, and Fee Monitoring.

4.6.3 Project Design Tools

4.6.3.1 Use Case Diagram

Figure 4.4 shows the Use Case Diagram of the Hostel Management System. It provides a high-level view of the system's functionalities and illustrates how the main actors Students, Hostel/Admin Staff, and Technician/Maintenance Personnel interact with the system. The system consists of five key submodules. Students can apply for a room, while admin staff are responsible for review application and allocating rooms based on availability and eligibility. The check-in and check-out processes are managed by both students and admin staff, covering early requests, room condition checklists, and damage reports. Student and admin staff also monitor hostel fee payments and records, ensuring accurate tracking. Finally, students can submit and track maintenance requests, which are managed and resolved by the technicians or maintenance personnel. This diagram provides a high-level overview of the system's functionalities and clearly shows how each actor interacts with the various processes.

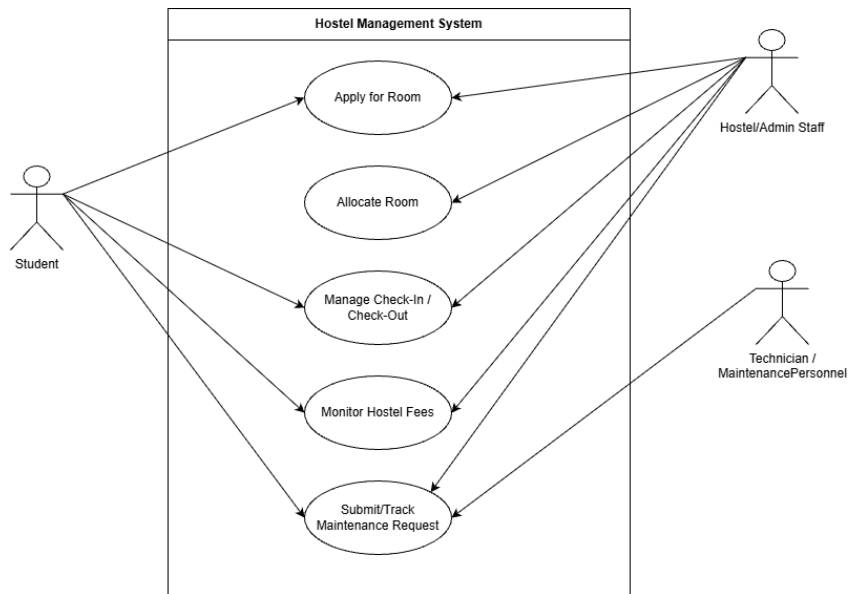


Figure 4.4 Use Case Diagram for Hostel Management System

4.6.3.2 Use Case Description

The Use Case Description explains each selected use case in detail by describing the sequence of actions, system responses, and expected outcomes. It clarifies functional requirements by defining actors, descriptions, preconditions, normal flows, and alternative scenarios for individual hostel management processes. In this section, the use case descriptions cover both student-driven and administrative operations across all five submodules, providing a clear understanding of how each process is performed, the expected outcomes, and how the system handles conditional paths.

1. Hostel Application Module

The Hostel Application Module is a core component of the Hostel Management System that supports the management of student accommodation requests. This module focuses on students apply for accommodation before room assignment and occupancy. It provides students with the ability to browse available hostel options, indicate room preferences, submit applications, and track application progress online. Supporting documents can also be uploaded to ensure eligibility verification.

From an administrative perspective, the module supports the evaluation and processing of hostel applications by enabling staff to review submissions, verify student information, and make approval or rejection decisions based on availability and policy. By digitalising the application workflow, the module improves transparency, ensures consistent decision-making, and reduces dependency on manual forms and fragmented data sources.

Figure 4.5 presents the use case diagram for the Hostel Application Module, highlighting the interactions between Student and Admin Staff actors and the system. The diagram captures the main functional capabilities provided by the module. Detailed descriptions of each use case are provided in Tables 4.4 to 4.9, which outline the system behaviour and user interactions in a structured manner.

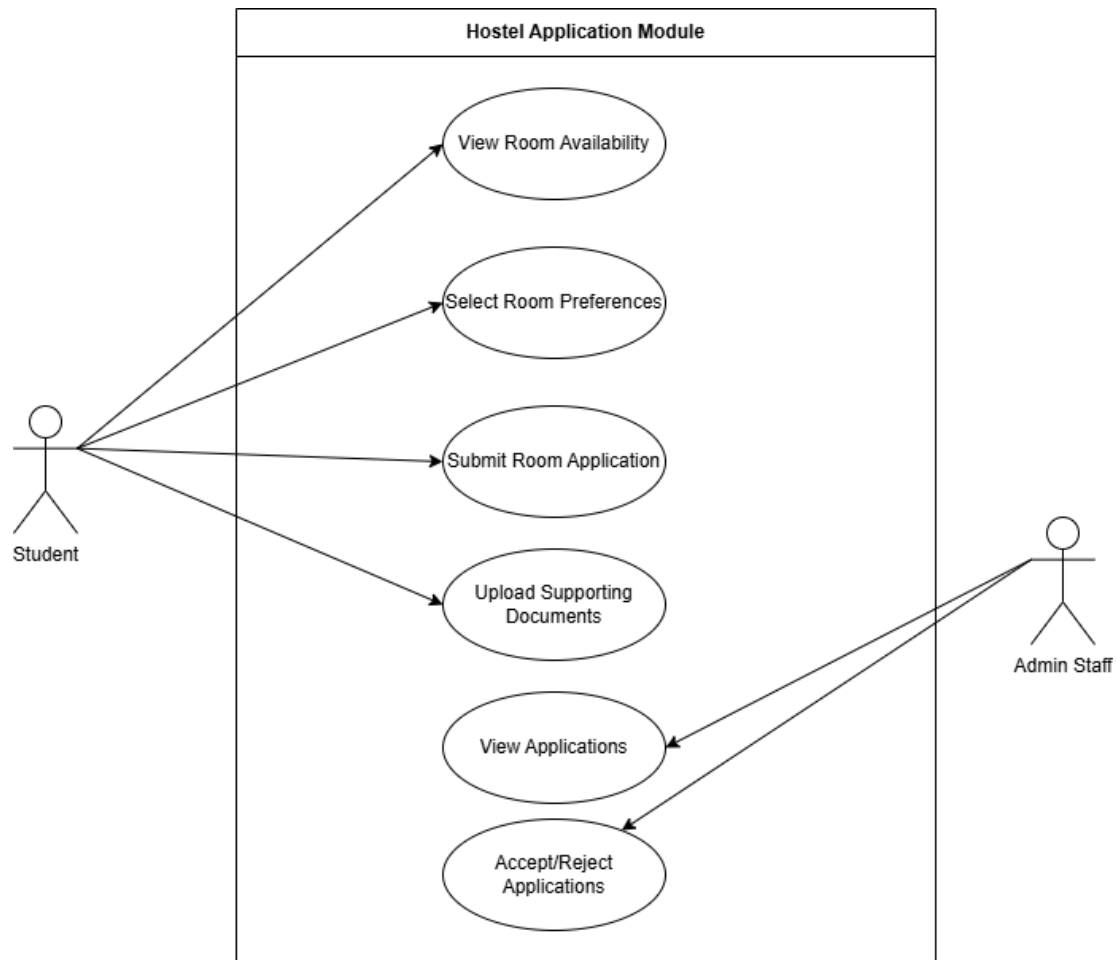


Figure 4.5 Use Case Diagram for Hostel Application Module

Use Case	View Room Availability
Actor	Student
Pre-condition	1. Student must be logged into the system via UTHM SSO 2. Hostel application period is open
Post-condition	Available hostel rooms and basic details are displayed to the student
Basic Path	1. Student opens the Hostel Application Module. 2. Student selects the option to view room availability. 3. System retrieves available room data from the hostel database. 4. System displays room availability information to the student.
Alternative Path	If no rooms are available, the system displays a “No rooms available” message.

Table 4.4 Description of Use Case View Room Availability

Use Case	Select Room Preferences
Actor	Student
Pre-condition	Student must be logged into the system
Post-condition	Room availability and occupancy information is display to the student
Basic Path	1. Student selects preferred hostel and room type. 2. Student confirms room preferences. 3. System stores the selected preferences for the current application.
Alternative Path	If selected room type becomes unavailable, the system prompts the student to reselect preferences.

Table 4.5 Description of Use Case Select Room Preferences

Use Case	Submit Room Application
Actor	Student
Pre-condition	1. Student is registered in SMP 2. Room preferences have been selected
Post-condition	1. Hostel application is successfully submitted and recorded 2. Application status is set to <i>Pending Approval</i>.
Basic Path	1. Student reviews application details. 2. Student submits the room application. 3. System validates student eligibility and room availability. 4. System records the hostel application. 5. System displays application confirmation.
Alternative Path	If eligibility validation fails, the system displays an error and prevents submission. If room availability changes, the system prompts the student to update preferences.

Table 4.6 Description of Use Case Submit Room Application

Use Case	Upload Supporting Documents
Actor	Student
Pre-condition	Student is logged in and document submission is required

Post-condition	Supporting documents are uploaded and linked to the application
Basic Path	1. Student selects the upload document option. 2. Student uploads required supporting documents. 3. System validates document format and size. 4. System saves the documents and confirms successful upload.
Alternative Path	If document format is invalid, the system rejects the upload and displays an error. If upload fails, the system prompts the student to retry

Table 4.7 Description of Use Case Upload Supporting Documents

Use Case	View Applications
Actor	Admin Staff
Pre-condition	Admin Staff is logged in
Post-condition	The list of hostel applications is displayed
Basic Path	Admin selects “Review Applications” from the system menu. Admin selects “View Applications” icon. The system will retrieve the applications records data. The system displays a list of applications with their details.
Alternative Path	If no applications is available, the system will display a message: “No applications recorded”.

Table 4.8 Description of Use Case View Applications

Use Case	View Applications
Actor	Admin Staff
Pre-condition	Admin Staff is logged in and is viewing the hostel application details
Post-condition	The hostel applications are either accepted or rejected with status updated in system.

Basic Path	Admin selects “View Applications” Admin accepts or rejects The system will update the application records data and status. The system displays new status of the application.
Alternative Path	-

Table 4.9 Description of Use Case View Applications

2. Room Allocation Module

The Room Allocation Module is designed to streamline and automate the process of assigning hostel rooms to eligible students. This module allows administrative staff to view real-time room availability, manage occupancy status, and allocate rooms to students based on predefined criteria such as eligibility and availability. At the same time, it ensures that only students with valid records are considered for allocation by verifying student information through the Student Information System (SMP). By automating room assignment and updating occupancy records, this module reduces manual coordination, minimizes allocation errors, and improves overall operational efficiency.

Figure 4.6 below illustrates the Use Case Diagram for the Room Allocation Module. The diagram identifies the primary actors involved, namely the Admin Staff and the System, and shows their interactions within the module. It outlines the key functions related to room availability viewing, room assignment, and record updates. The use cases define the operational scope of the module, ensuring that room allocation activities are carried out systematically and accurately. Tables 4.10 to Table 4.12 summarize the detailed use case descriptions for the Room Allocation Module.

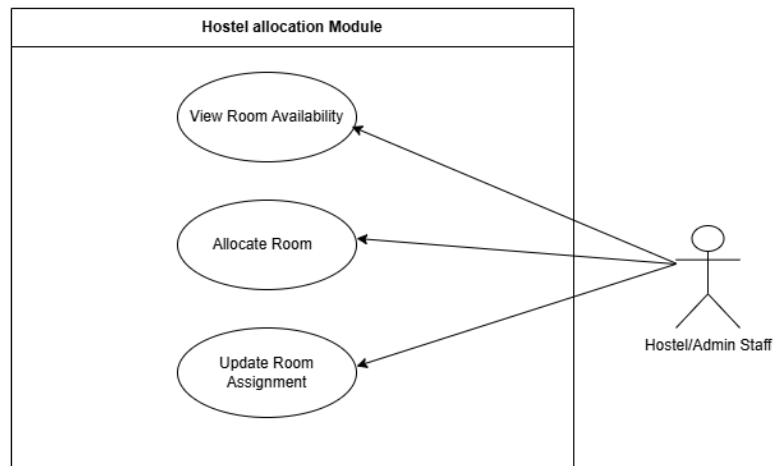


Figure 4.6 Use Case Diagram for Room Allocation Module

Use Case	View Room Availability
Actor	Admin Staff
Description	Allow admin to view current availability and occupancy status of hostel rooms
Pre-condition	Admin has login to the system
Post-condition	Room availability and occupancy information is display to the admin
Basic Path	1. Admin selects “Room Allocation” from the system menu. 2. Admin selects “View Room Availability”. 3. The system will retrieve the latest room availability data. 4. The system displays a list of rooms with their occupancy and availability status.
Alternative Path	1. If no room data is available, the system will display a message: “No room information available at the moment”.

Table 4.10 Description of Use Case View Room Availability

Use Case	Allocate Room
Actor	Admin Staff
Description	Allow admin to allocate an available room to student based on eligible and room availability
Pre-condition	1. Admin has login to the system 2. The student must have submitted a valid hostel application.

	3. Latest room availability data must exist in the system.
Post-condition	The selected room is assigned to the student and the room occupancy record is updated.
Basic Path	1. Admin selects “Allocate Room” from the Room Allocation module. 2. The system displays available room and their current occupancy status. 3. Admin select a suitable room for the student 4. Admin confirm the allocation. 5. The system assigns the room to the student. 6. The system updates the room occupancy data. 7. The system displays a confirmation message: “Room allocated successfully.”
Alternative Path	i. If no suitable room is available, the system displays an error message: “No available room for allocation.”

Table 4.11 Description of Use Case Allocate Room

Use Case	Update Room Assignment
Actor	Admin Staff
Description	Allow admin to update or change an existing room assignment for a student.
Pre-condition	1. Admin has login to the system 2. The student must already have an assigned room.
Post-condition	The student’s room assignment is updated and room occupancy information is adjusted.
Basic Path	1. Admin selects “Update Room Assignment” from the Room Allocation module. 2. The system displays the current room assignment details to admin. 3. Admin selects a new room or modifies the existing assignment. 4. Admin confirm the changes. 5. The system updates the room assignment record. 6. The system updates the room occupancy information.

	7. The system displays a confirmation message: “Room assignment updated successfully.”
Alternative Path	3. If the selected room is already full, the system displays an error message: “Selected room is not available.”

Table 4.12 Description of Use Case Update Room Assignment

3. Check-In/Check-Out Module

The Check-In / Check-Out module is part of the Hostel Management System and is responsible for managing student hostel occupancy activities. The module allows students to perform and view digital check-in and check-out, submit room condition information, and report room damages. It also enables administrative staff to review damage reports, request maintenance, issue fines, and generate reports. This module ensures accurate record-keeping, improves operational efficiency, and reduces manual processing.

Figure 4.7 illustrates the use case diagram for the Check-In / Check-Out module. The diagram identifies the main actors involved, namely Student and Admin Staff, and outlines the interactions between the actors and the system. The use cases define the functional scope of the module from both student and administrative perspectives. Tables 4.13 to 4.22 summarize the detailed use case descriptions for the Check-In / Check-Out module.

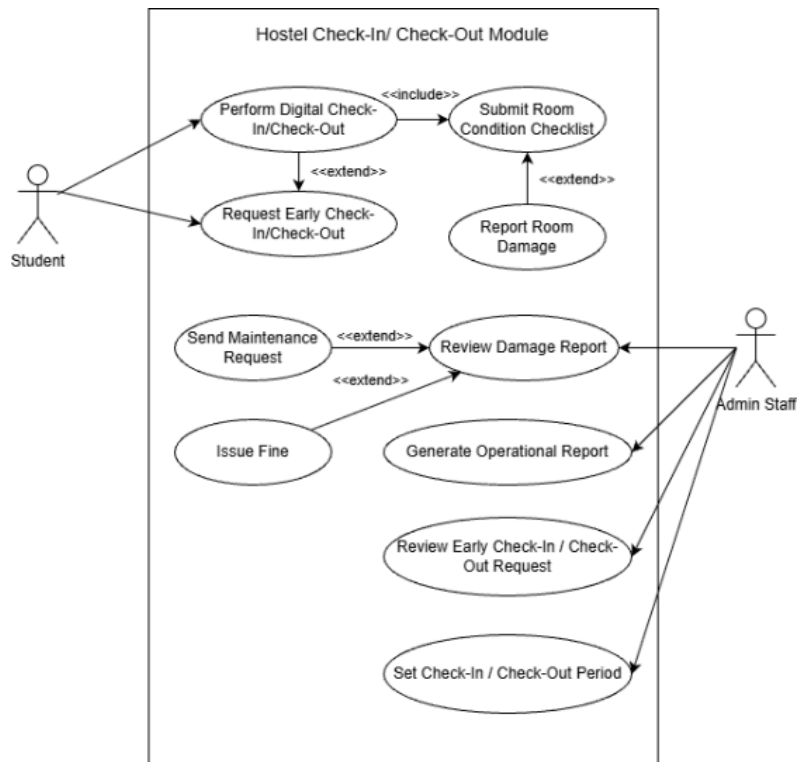


Figure 4.7 Use Case Diagram for Check In/ Check Out Module

Use Case	Perform Digital Check-In/Check-Out
Actor	Student
Description	Allows the student to perform digital check-in or check-out for the assigned hostel room when the system is open.
Pre-condition	- Student is registered and assigned a hostel room. - Check-in/check-out period is open OR early request has been approved.
Post-condition	Room condition is recorded and status updated.
Basic Path	1. Students open Digital Check-In/Check-Out UI. 2. System verifies that check-in/check-out access is allowed. 3. Student performs QR code verification. 4. System validates room assignment. 5. Student submits room condition checklist. 6. System saves room condition and updates status.
Alternative Path	If access is not allowed, system displays a locked message. If QR verification fails, prevent access to checklist.

	If room condition checklist submission fails, system displays error and prompts retry.
--	--

Table 4.13 Description of Use Case Perform Digital Check-In/Check-Out

Use Case	Request Early Check-In/ Check-Out
Actor	Student
Description	Allows the student to request early check-in or check-out before the official period.
Pre-condition	Student is registered and assigned a hostel room.
Post-condition	Early request is submitted for admin review.
Basic Path	1. Student opens Early Check-In / Check-Out request page. 2. System detect early check-in or early check-out. 3. Student enters date, reason and uploads supporting evidence. 4. Student submits request. 5. System records request with “Pending” status.
Alternative Path	If submission fails, system displays error and prompts retry.

Table 4.14 Description of Use Case View Check-In/Check-Out Status

Use Case	Submit Room Condition Checklist
Actor	Student
Description	Allows the student to submit the room condition checklist during check-in or check-out.
Pre-condition	QR code verified successfully.
Post-condition	Room condition recorded in system.
Basic Path	1. Students fills in checklist fields. 2. Student submits checklist. 3. System saved record.
Alternative Path	If submission fails, system display error and prompt retry.

Table 4.15 Description of Use Case Submit Room Condition Checklist

Use Case	Report Room Damage
Actor	Student
Description	Allows the student to report any room damage observed during check-out.

Pre-condition	Student performing check-out and observes room damage.
Post-condition	Damage report submitted to admin staff.
Basic Path	1. Students identifies and submit damaged items. 2. System saved record for admin staff review.
Alternative Path	If student choose not to report, skip this step.

Table 4.16 Description of Use Case Report Room Damage

Use Case	Review Damage Report
Actor	Admin Staff
Description	Allows the admin staff to review and assess reported room damages.
Pre-condition	Student submitted damage report.
Post-condition	Admin has reviewed and assessed reported damages.
Basic Path	1. Admin opens damage report list. 2. Admin selects a report to review and evaluate. 3. Admin decides on actions (fine or maintenance).
Alternative Path	-

Table 4.17 Description of Use Case Review Damage Report

Use Case	Issue Fine
Actors	Admin Staff
Description	Allows the admin staff to issue a fine to a student for confirmed room damage.
Pre-condition	Damage report reviewed and confirmed that fine is needed.
Post-condition	Student is fined for the damage.
Basic Path	1. Admin selects “Issue Fine” option. 2. Admin enters fine details (amount and reason). 3. System records fine and notifies student.
Alternative Path	If fine disputed, admin may cancel or adjust fine.

Table 4.18 Description of Use Case Issue Fine

Use Case	Send Maintenance Request
Actors	Admin Staff
Description	Allows the admin staff to send a maintenance request for reported room damage.

Pre-condition	Damage report reviewed and repair is required.
Post-condition	Maintenance request is sent to the maintenance module.
Basic Path	1. Admin selects “Send Maintenance Request” option. 2. Admin fills in details (damage type, urgency). 3. System forwards request to maintenance module.
Alternative Path	If maintenance request failed, system display error and admin retry.

Table 4.19 Description of Use Case Send Maintenance Request

Use Case	Generate Operational Report
Actor	Admin Staff
Description	Allows admin staff to generate operational reports related to check-in, check-out, and damage records.
Pre-condition	Relevant data exists in the system.
Post-condition	Selected report is generated and displayed.
Basic Path	1. Admin selects “Generate” option. 2. System retrieves relevant data from database. 3. System display generated reports.
Alternative Path	If data unavailable, system displays “No data available.”

Table 4.20 Description of Use Case Generate Check-In/Check-Out Report

Use Case	Set Check-In/ Check-Out Period
Actor	Admin Staff
Description	Allows admin staff to define official check-in and check-out periods for a semester.
Pre-condition	Admin staff is authenticated.
Post-condition	System schedule is updated and controls student access automatically.
Basic Path	1. Admin opens check-in/check-out period settings. 2. Admin enters start and end dates for check-in and check-out. 3. Admin saves settings. 4. System updates access rules based on defined dates.
Alternative Path	If invalid dates are entered, system displays error message.

Table 4.21 Description of Use Case Set Check-In/ Check-Out Period

Use Case	Review Early Check-In / Check-Out Request
Actor	Admin Staff
Description	Allows admin staff to review and decide on early check-in or check-out requests.
Pre-condition	Student has submitted an early request.
Post-condition	Request is approved or rejected and student is notified.
Basic Path	1. Admin opens early request list. 2. Admin selects a request to review. 3. Admin reviews reason and supporting evidence. 4. Admin approves or rejects request. 5. System updates request status and notifies student.
Alternative Path	If decision fails to save, system displays error and prompts retry.

Table 4.22 Description of Use Case Review Early Check-In / Check-Out Request

4. Maintenance

Request

Module

The Maintenance Request Module is part of the Hostel Management System and is responsible for managing the reporting and resolution of facility-related issues. The module allows students to submit maintenance tickets for damages such as electrical or plumbing faults and view the real-time status of their requests. It also enables Hostel/Admin Staff to view incoming requests, assign technicians to specific tasks, and update the repair status once the work is completed. This module ensures centralized tracking of defects, improves operational response times, and replaces inefficient manual reporting methods.

Figure 4.8 illustrates the use case diagram for the Maintenance Request Module. The diagram identifies the main actors involved, namely the Student and Hostel/Admin Staff, and outlines the interactions between the actors and the system. The use cases define the functional scope of the module from both the student's reporting perspective and the administrative management perspective. Tables 4.23 to 4.27 summarize the detailed use case descriptions for the Maintenance Request Module.

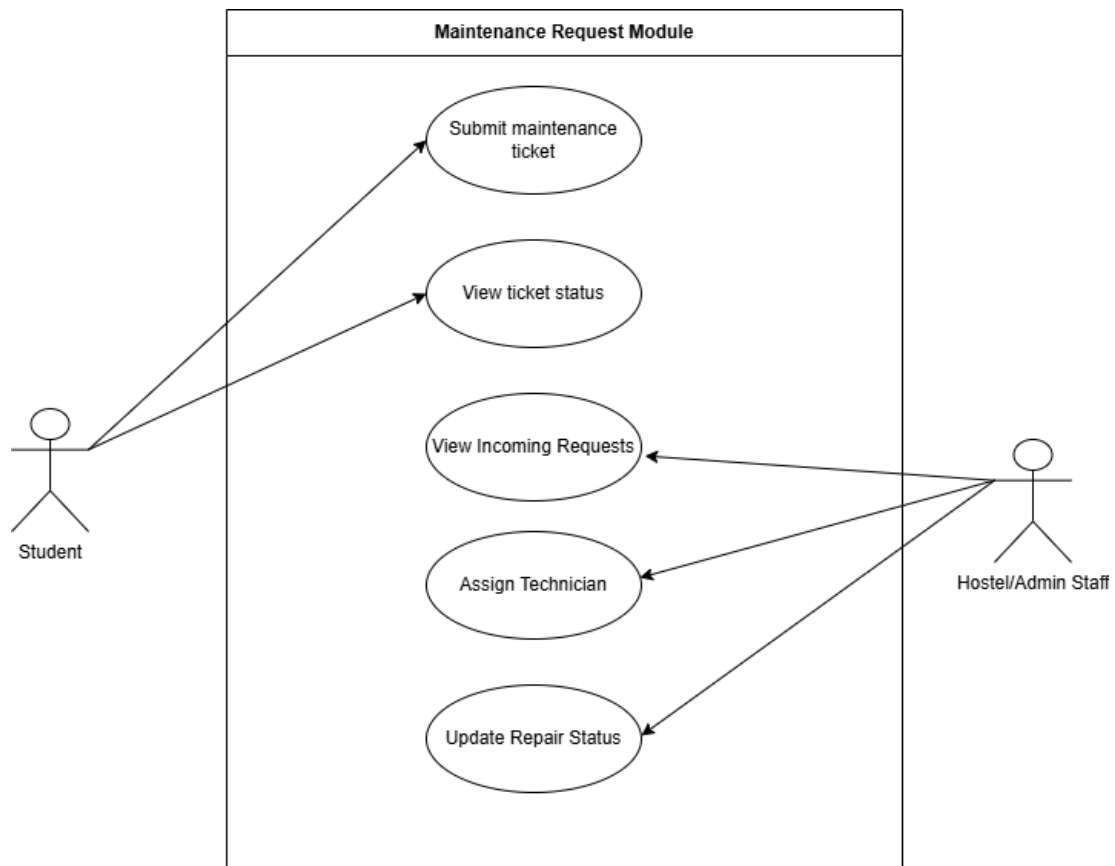


Figure 4.8 Use Case Diagram for Maintenance Request Module

Use Case	Submit Maintenance Ticket
Actors	Student
Description	Allows a student to report a facility issue (e.g., broken fan, leaking pipe) by filling out a digital form and attaching photo evidence.
Pre-condition	1. Student must be logged into the myUTHM profile 2. Student must have a valid room allocation.
Post-condition	A new maintenance ticket is created in the database with the status "Pending".
Basic Path	1. Student selects "Maintenance Request" from the menu. 2. Student clicks "Report New Issue". 3. System displays the report form with Student Name and Room No. auto-filled. 4. Student selects the Category (e.g., Electrical, Plumbing). 5. Student enters the Description of the damage.

	6. Student uploads an evidence photo of the damage. 7. Student clicks "Submit". 8. System validates the input (and file format) and saves the ticket. 9. System displays a success message "Request Submitted Successfully".
Alternative Path	1. If the student leaves the description blank, the system displays an error: "Please describe the issue." 2. If the uploaded photo is too large (>5MB) or in the wrong format (e.g., .exe), the system displays an error: "Invalid file. Please upload a JPG or PNG under 5MB."

Table 4.23 Description of Use Case Submit Maintenance Ticket

Use Case	View Ticket Status
Actors	Student
Description	Allows the student to track the progress of their submitted maintenance requests.
Pre-condition	Student has submitted at least one maintenance ticket.
Post-condition	Student views the current status (Pending, In Progress, or Completed).
Basic Path	1. Student navigates to the "My Requests" tab. 2. System retrieves and lists all tickets submitted by the student. 3. Student views the status column (e.g., "In Progress - Technician Assigned").
Alternative Path	-

Table 4.24 Description of Use Case View Ticket Status

Use Case	View Incoming Requests
Actors	Hostel/Admin Staff
Description	Allows the admin to view a dashboard list of all maintenance complaints submitted by students.
Pre-condition	Admin is logged in with staff privileges.
Post-condition	Admin sees a filtered list of tickets (e.g., all "Pending" items).

Basic Path	1. Admin logs in and selects "Maintenance Dashboard". 2. System displays a list of all maintenance tickets. 3. Admin filters the list by status (e.g., "Pending", "Urgent").
Alternative Path	-

Table 4.25 Description of Use Case View Incoming Request

Use Case	Assign Technician
Actors	Hostel/Admin Staff
Description	Allows the admin to assign a specific technician or contractor to handle a reported issue.
Pre-condition	A ticket exists with the status "Pending".
Post-condition	Ticket status updates to "In Progress" and a technician is linked to the record.
Basic Path	1. Admin selects a "Pending" ticket from the list. 2. Admin clicks "Assign Technician". 3. Admin inputs or selects the Technician Name (e.g., "Mr. Ali - Electrician"). 4. Admin clicks "Confirm Assignment". 5. System updates the ticket status to "In Progress" and notifies the student.
Alternative Path	-

Table 4.26 Description of Use Case Assign Technician

Use Case	Update Repair Status
Actors	Hostel/Admin Staff
Description	Allows the admin to mark a maintenance job as completed or rejected.
Pre-condition	Ticket is currently "In Progress".
Post-condition	Ticket status updates to "Completed" .
Basic Path	1. Admin views "In Progress" tickets. 2. Admin selects a specific ticket where work is finished. 3. Admin updates status to "Completed" . 4. Admin adds optional remarks (e.g., "Fan capacitor replaced"). 5. System saves the changes and closes the ticket.

Alternative Path	-
------------------	---

Table 4.27 Description of Use Case Update Repair Status

5. Hostel Fee Monitoring Module

The Hostel Fee Monitoring Module is designed to streamline and automate the financial oversight of student residential accounts. This module allows students to view their current hostel outstanding balances, executing digital payments, and accessing a history of their past transactions. Simultaneously, it empowers administrative staff with real-time visibility into the hostel's fee collection status through the generation of summarized payment reports and the ability to trigger automated reminders to students with unpaid status. By integrating directly with the university's centralized Finance system, this module ensures data consistency and reduces the manual workload.

Figure 4.9 below illustrates the Use Case Diagram for the Hostel Fee Monitoring Module. The diagram identifies the primary actors, the Student and the Admin Staff. It outlines the specific functional interactions between these users and the system. The use case define the operational scope of the module, ensuring that both student financial needs and administrative monitoring requirements are met within a unified digital framework. Tables 4.28 to Table 4.32 summarize the detailed use case descriptions for the Hostel Fee Monitoring Module.

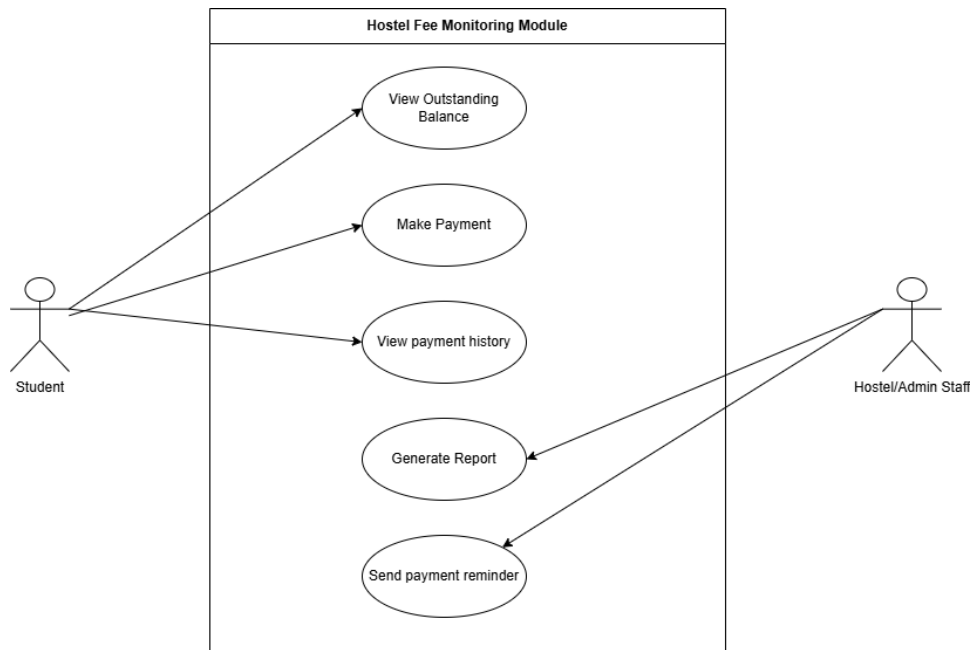


Figure 4.9 Use Case Diagram for Hostel Fee Monitoring Module

Use Case	View Outstanding Balance
Actors	Student
Description	Allow student to query and view their outstanding hostel fees.
Pre-condition	Student is logged into the Hostel Portal.
Post-condition	The system displays the current outstanding balance.
Basic Path	1. PaymentController queries the database for the balance amount. 2. Database returns the balance amount to the controller. 3. Portal will show the outstanding balance of student.
Alternative Path	If outstanding balance is cleared, no need to proceed to payment.

Table 4.28 Description of Use Case View Outstanding Balance

Use Case	Make Payment
Actors	Student
Description	Facilitates the digital transaction process for students to settle their outstanding balances through the hostel portal.
Pre-condition	Student has viewed their balance and decided to proceed with payment.
Post-condition	Payment is processed, and the database status is updated.

Basic Path	1. Student triggers the "makePayment" action. 2. Portal sends the transaction request to the PaymentController. 3. Controller attempts to update the status in the database to "Paid". 4. Database confirms the payment status. 5. System notifies the student of the successful payment status.
Alternative Path	If payment failed, system display error and need to retry.

Table 4.29 Description of Use Case Make Payment

Use Case	View Payment History
Actors	Student
Description	Enables students to access all their previously transactions for personal record-keeping.
Pre-condition	Student is logged into the Hostel Portal.
Post-condition	A historical list of all past transactions (dates, amounts, receipt IDs) is displayed to the user.
Basic Path	1. Student navigates to the "Payment History" section on the dashboard. 2. Portal requests the transaction log from the PaymentController using the student's ID. 3. Controller queries the database for all records with status "Paid" linked to that student. 4. Database returns the list of transaction details (Date, Amount, Receipt Ref). 5. Portal formats and displays the history table to the student. 6. Student clicks on a specific transaction to view/download the receipt.
Alternative Path	System displays a "No transactions found" when no payment history available.

Table 4. Description of Use Case View Payment History

Use Case	Generate Report
Actors	Admin Staff

Description	Provide admin staff with a summarized list of student payment statuses to monitor real-time fee collection.
Pre-condition	Admin is logged into the system
Post-condition	A comprehensive list of paid and unpaid students is displayed.
Basic Path	1. Admin selects "viewPaymentReport". 2. Portal requests student payment data from the PaymentController. 3. Controller queries the database for all records. 4. Database returns the full list of records. 5. Portal displays the list categorized by "Paid" and "Unpaid" status.
Alternative Path	If records are empty, system displays "No records found".

Table 4.31 Description of Use Case Generate Report

Use Case	Send Payment Reminder
Actors	Admin Staff
Description	Allow admin staff to trigger automated notifications or alerts to students who is having an "unpaid" status.
Pre-condition	Admin has generated the report and identified unpaid students.
Post-condition	Unpaid students receive a notification reminder.
Basic Path	1. Admin identifies students with "unpaid" status. 2. Admin triggers the "checkUnpaidStudents" and reminder function. 3. System triggers a notification (triggerNoti) for the selected records. 4. System notifies the student(s) of the required payment. 5. Admin sees a display confirming the reminders were sent.
Alternative Path	If no unpaid records exist, the system informs the Admin Staff that no reminders are necessary.

Table 4.32 Description of Use Case Send Payment Reminder

4.6.3.3 Activity Diagram

The Activity Diagram illustrates the workflow of selected use cases across the system by showing the sequence of activities, decision points, and the flow of control between actors and the system. It is used to represent the dynamic behaviours of the system, clarifying how different tasks are performed, how decisions are made, and how processes are coordinated. In this section, activity diagrams are used to provide a visual overview of the operations and interactions within each of the five submodules, helping to demonstrate the logical flow of actions, dependencies, and conditional paths between students, administrative staff, and the system.

1. Hostel Application Module

Figure 4.10 below shows the activity diagram for the Hostel Application Module. The diagram starts with the Student accesses the hostel application interface. The system will validate the student's record through the Student Management System (SMP). If the student is validated, the system will display the list of available rooms according to respective hostel blocks, otherwise an error message will be displayed. After the student locked in their decision and wants to apply, the system will display student hostel application form. The student is required to fill in their personal information and upload supporting documents before submitting. The system will update its database and saves the application details.

The diagram shows after the application is successfully submitted, the admin staff will view the application displayed by the system. The admin staff can choose either accept or deny the application, which will update the system for the status of the application. The diagram illustrates both the student-driven and admin-driven workflow and the conditional flow based on student eligibility.

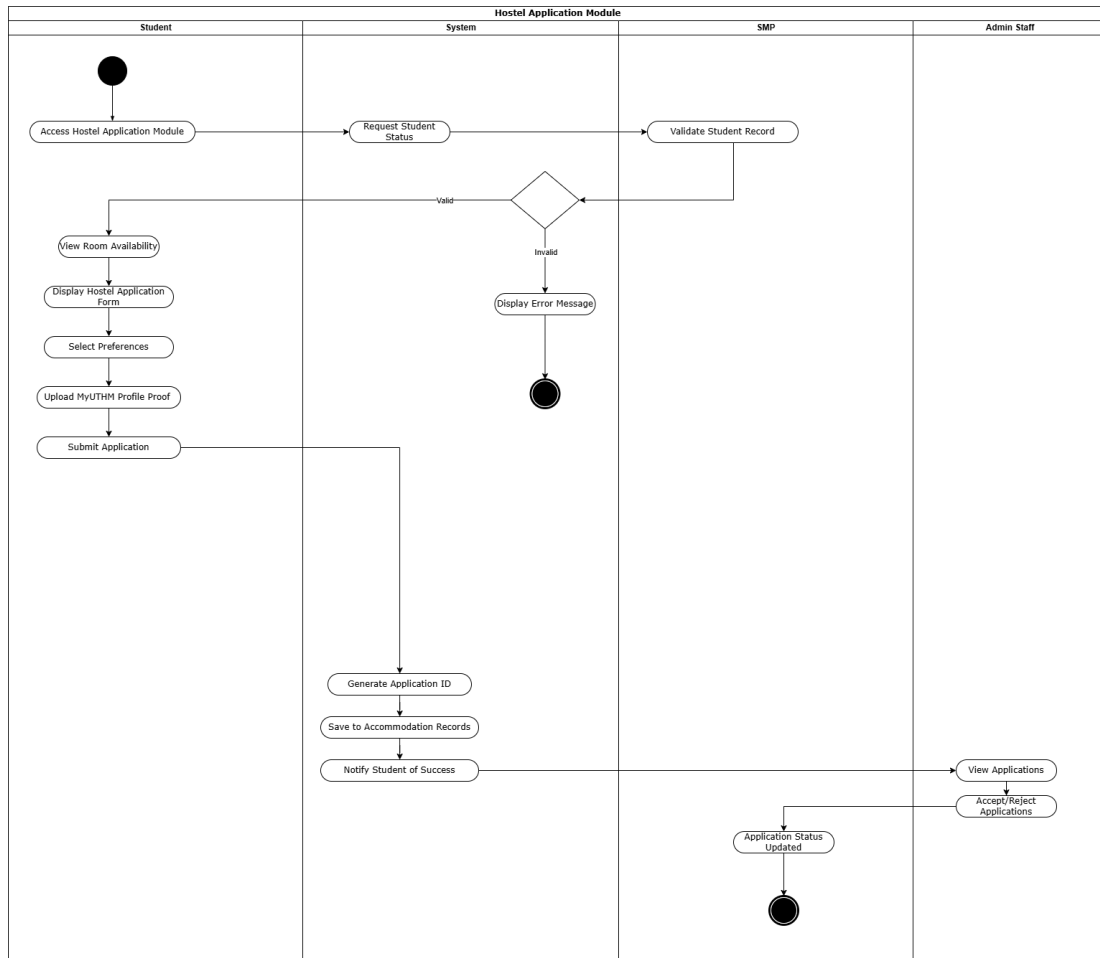


Figure 4.10 Activity Diagram for Hostel Application Module

2. Room Allocation Module

Figure 4.11 below shows the activity diagram for the Room Allocation Module. The diagram begins with the Admin Staff accessing the room allocation interface, where the system first validates the selected student's status by communicating with the Student Management System (SMP). If the student is eligible, the system displays the list of available rooms for allocation. The admin then confirms the room assignment, and the system updates the room occupancy records and saves the allocation details. If the student is found to be ineligible, an error message is displayed. The diagram illustrates the admin-driven workflow and the conditional flow based on student eligibility.

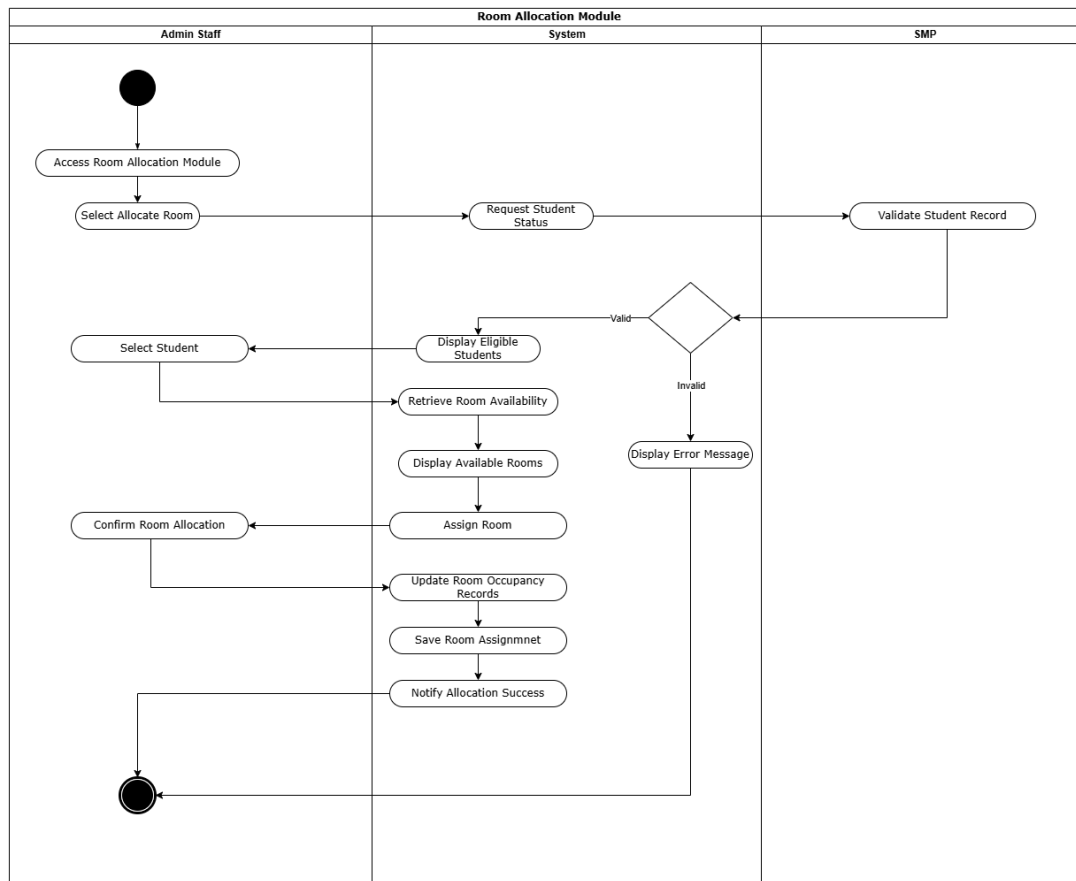


Figure 4.11 Activity Diagram for Room Allocation Module

3. Hostel Check-In/Check-Out Module

Figure 4.12 below shows the activity diagram for the Check-In / Check-Out module. The diagram begins with the student accessing the check-in or check-out interface, where the system first validates whether the check-in/out period is currently open or whether an early request needs review. Students may submit early check-in/out requests along with supporting evidence, which are processed by the system. Once access is granted, students perform QR verification, submit the room condition checklist, and optionally report any observed damages. Administrative actions, including reviewing early requests, reviewing damage reports, approving fines, sending maintenance requests, and generating operational reports, are also represented. The diagram captures both student-driven and admin-driven workflows, highlighting conditional paths based on system availability, request approvals, and damage reporting.

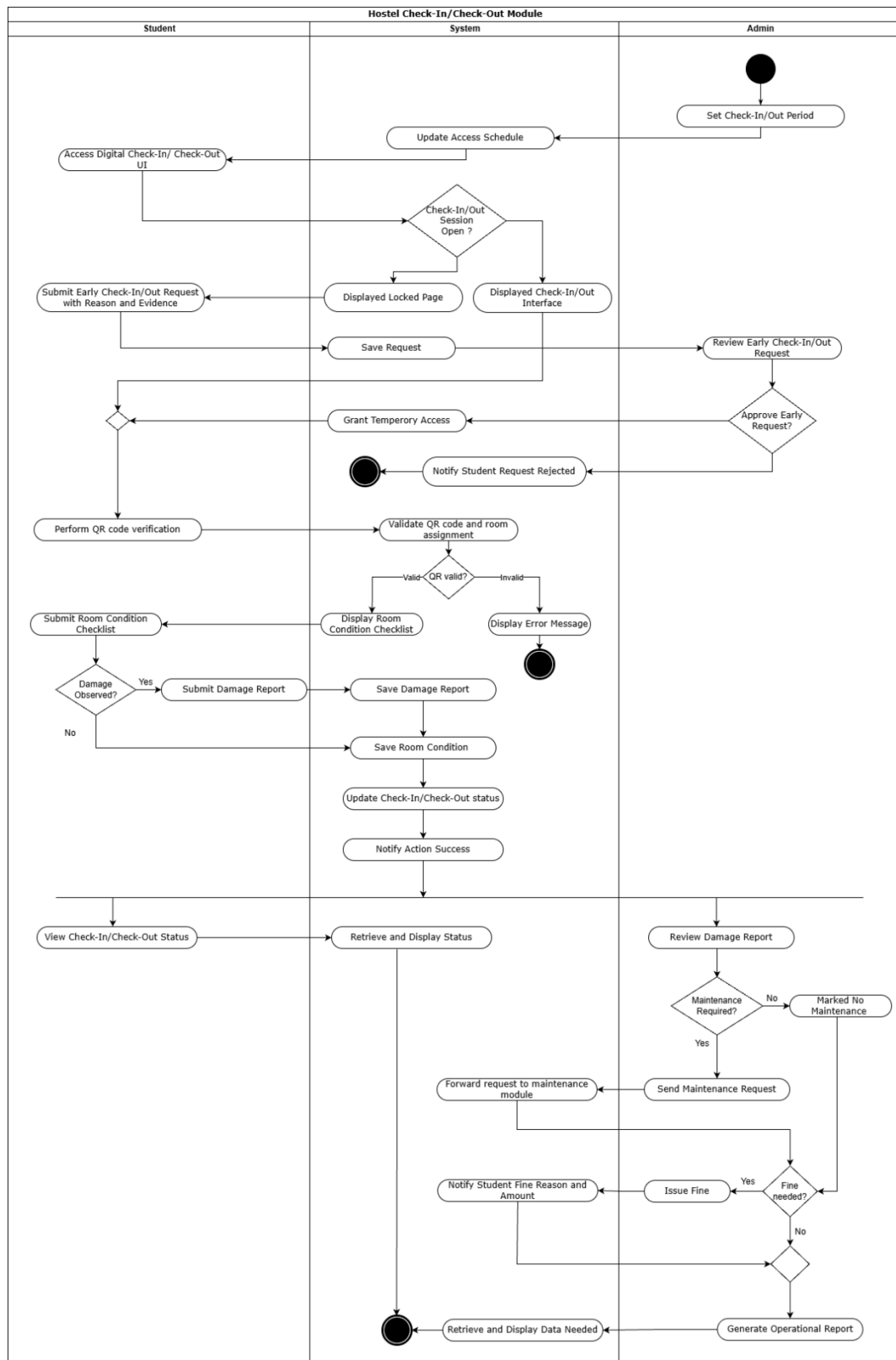


Figure 4.12 Activity Diagram for Hostel Check-In/Check-Out Module

4. Maintenance Request Module

Figure 4.13 below illustrates the operational workflow of the Maintenance Request Module, utilizing swimlanes to clearly distinguish the responsibilities of the Student, the System, and the Admin Staff. The process initiates in the Student lane, where the user logs into their profile, navigates to the hostel page, and submits a maintenance request by filling in the category, description, and uploading an evidence photo. The System then acts as a gatekeeper by validating the input and file size; if the input is invalid, an error message is displayed, terminating the flow. However, if the input is valid, the System creates a new ticket with a "Pending" status and displays a success confirmation.

Subsequently, the workflow shifts to the Admin Staff, who views the dashboard, selects the pending ticket, and assigns a technician, triggering the System to update the status to "In Progress," save the details, and notify the student. The process concludes when the Admin verifies the work completion and updates the status to "Completed," prompting the System to formally close the ticket.

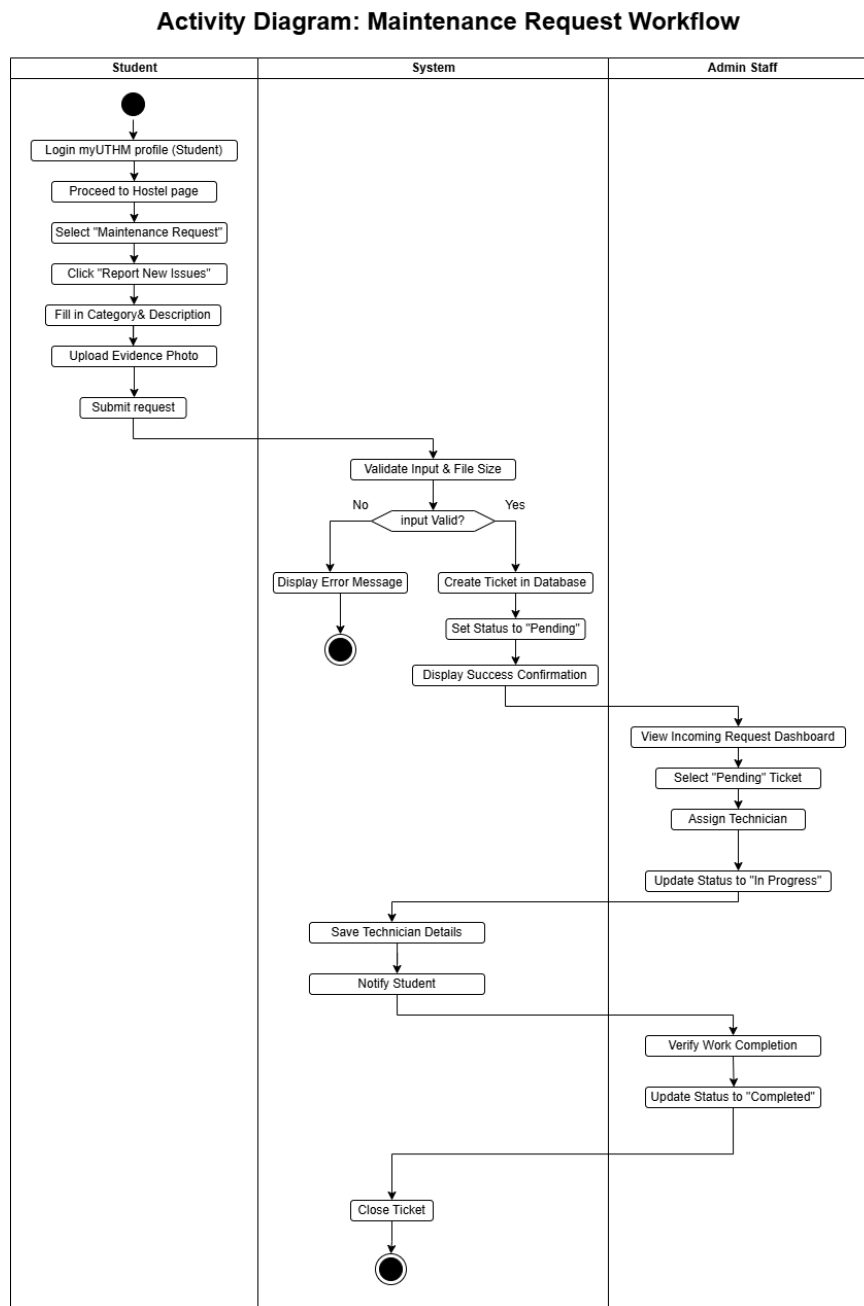


Figure 4.13 Activity Diagram for Maintenance Request Module

5. Hostel Fee Monitoring Module

Figure 4.14 below shows the Activity Diagram for the Hostel Fee Monitoring Module. This is a procedural workflow for both students and administrative staff. For students, the process begins with logging into their profile to query and view their current outstanding balance. If they choose to proceed payment, the system facilitates

the payment transaction, updates the status to "paid" in the database and provides a confirmation message. Simultaneously, the diagram outlines administrative actions where staff can access payment reports to review the list of paid and unpaid students. The module enables staff to trigger the notification system to send automated payment reminders to those with outstanding balances. Lastly, it will send a confirmation message that the reminders were successfully dispatched.

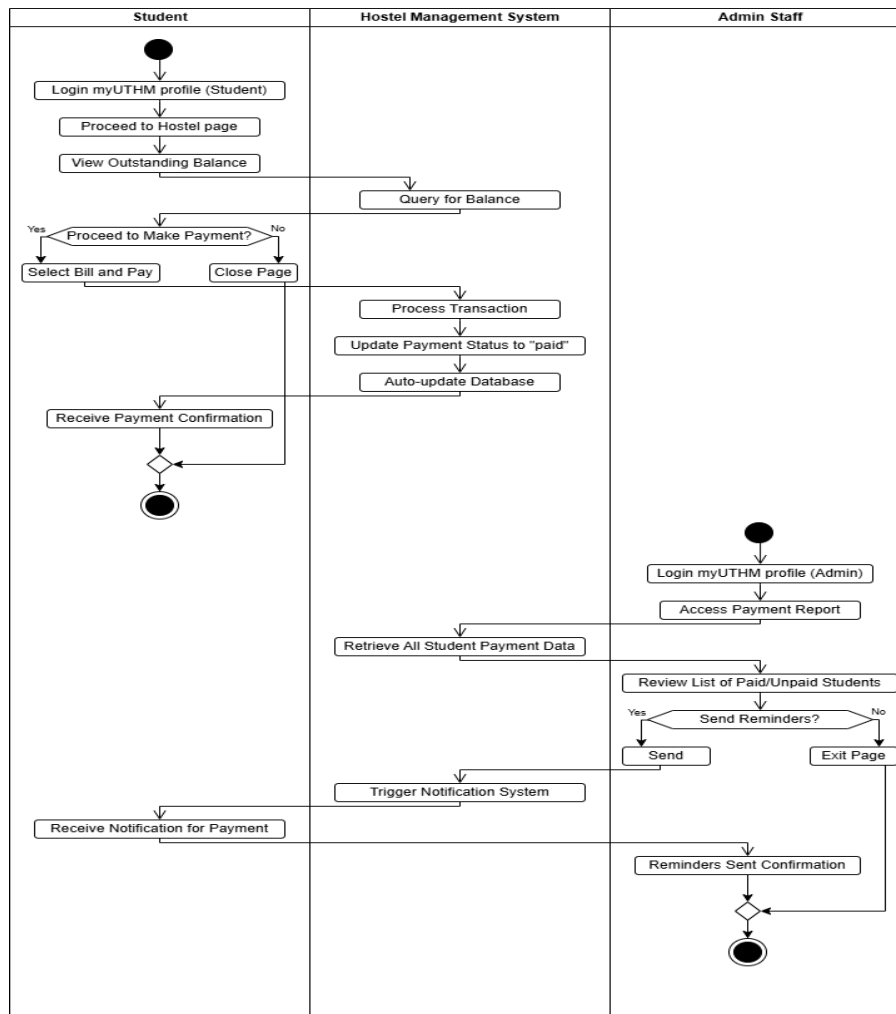


Figure 4.14 Activity Diagram for Hostel Fee Monitoring Module

4.6.3.4 Sequence Diagram

The Sequence Diagram illustrates the interaction between system components and actors over time for a selected use case. It shows how messages are exchanged in a specific order to complete a hostel management operation, helping to clarify system behaviour and logical integration between modules. In this section, separate sequence diagrams are used to represent the interactions for student and administrative activities across the five submodules.

i. Hostel Application Module

Figure 4.15 below illustrates the sequence diagram for the Hostel Application Module, detailing the interaction between the Student, Admin Staff, Application UI, ApplicationController, SMP Database, and the HMS Database. The process begins when the Student clicks to start a new application, triggering the UI to request an eligibility check from the controller using the student's ID. The controller then queries the SMP Database to verify the student's current status; once an "Active" status is returned, the controller instructs the UI to display the application form to the student. The student then submits their preferences, such as building block and room type, which the UI passes to the controller for validation and saving in the HMS Database. Upon receiving a confirmation of a successful save, the controller triggers a success message in the UI, which displays the application status as "pending" to the student.

The second phase of the process involves the Admin Staff, who initiates a request to view all submitted applications through the UI. The ApplicationController retrieves the application list from the SMP Database and displays them via the UI for administrative review. Once the admin staff decides to accept or reject a specific application, the UI sends the decision to the controller, which updates the application record in the HMS Database with the new status. Finally, the database confirms the save, and the controller prompts the UI to display the updated application status, successfully completing the administrative lifecycle of the hostel application.

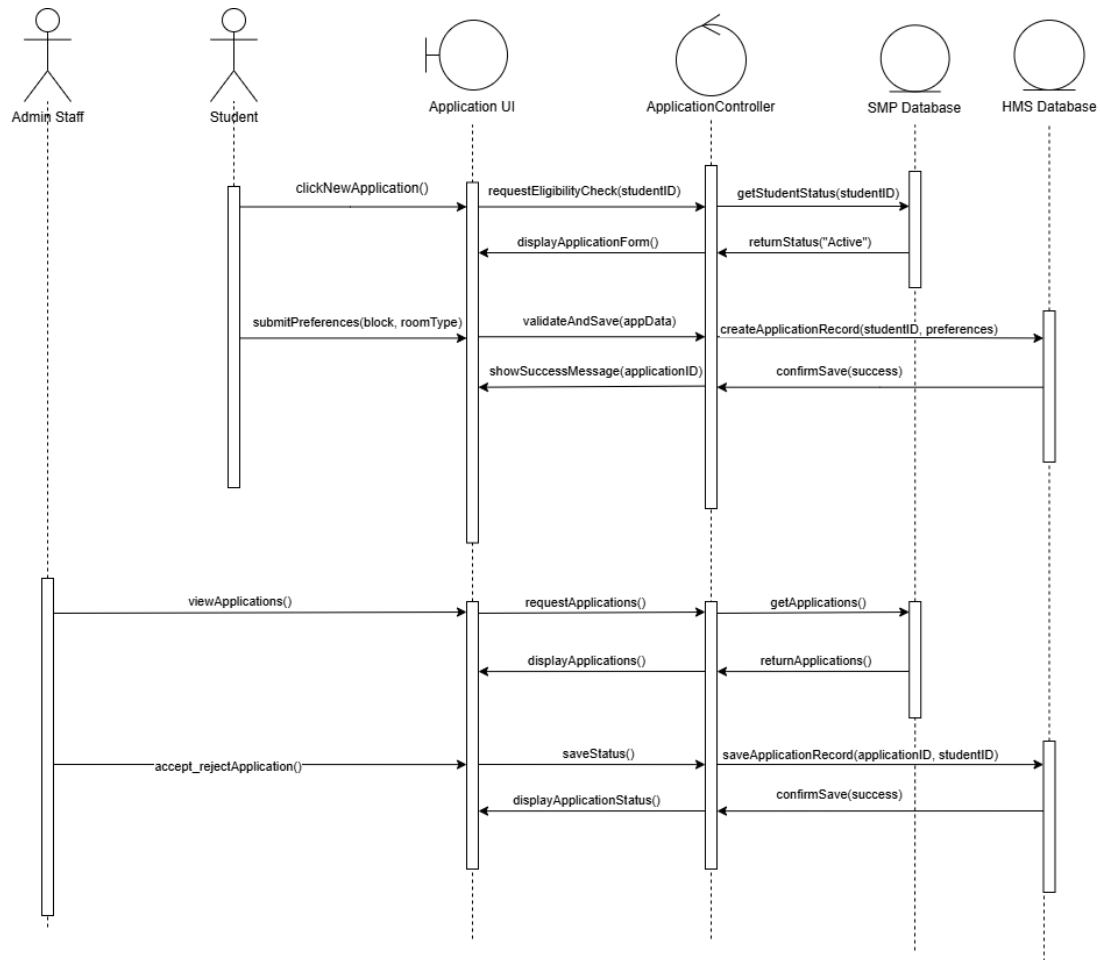


Figure 4.15 Sequence Diagram for Hostel Application Module

ii. Room Allocation Module

Figure 4.16 below illustrates the sequence diagram for the Room Allocation Module, showing the interaction between the Admin Staff, Room Allocation Controller, Room Allocation System, and the SMP database. The process starts when the Admin Staff views room availability, and the controller retrieves the list of available rooms from the system and database. When the admin proceeds to allocate a room, the controller checks the selected room's availability before assigning it to the student. Once confirmed, the system updates the room assignment and room status as occupied in the database. Finally, the system returns the allocation result and displays a successful confirmation to the admin, completing the process.

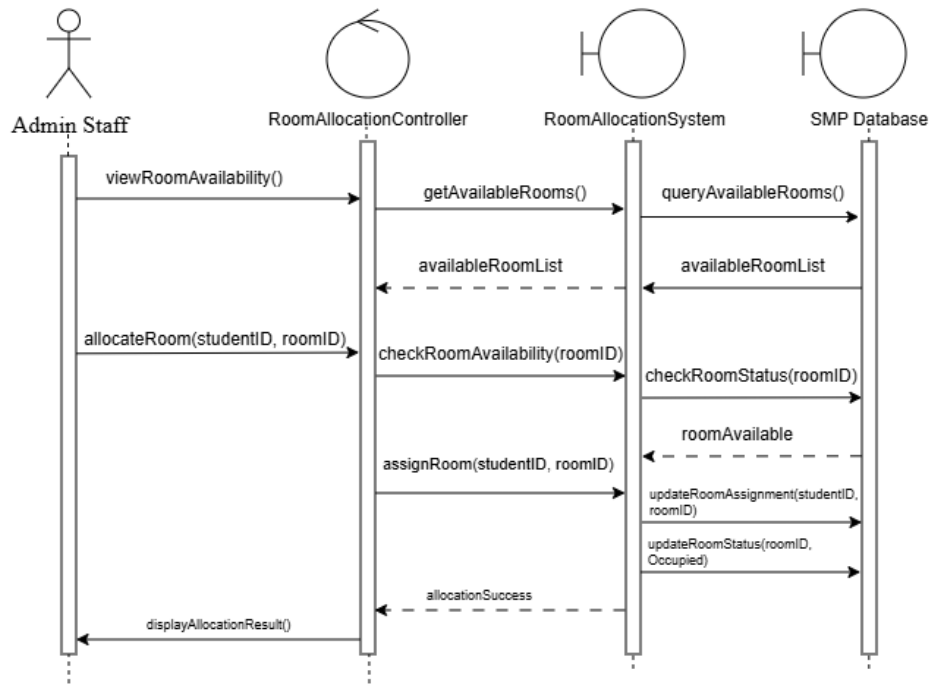


Figure 4.16 Sequence Diagram for Room Allocation Module

iii. Hostel Check In/ Check Out Module

Figure 4.17 below presents the student sequence diagram for the Check-In / Check-Out module. It illustrates the chronological interactions between the Student, Student UI, Check-In/Out Controller, Hostel Management System, and SMP Database. The diagram shows how the system first checks whether the check-in/out period is open and retrieves the student's current status. Students may submit early check-in/out requests if the period is not yet open, and the system records these requests for admin review. Once allowed, students perform QR verification, submit room condition checklists, and optionally report damages. Each step demonstrates the sequential flow of data and system responses, emphasizing both normal and conditional actions.

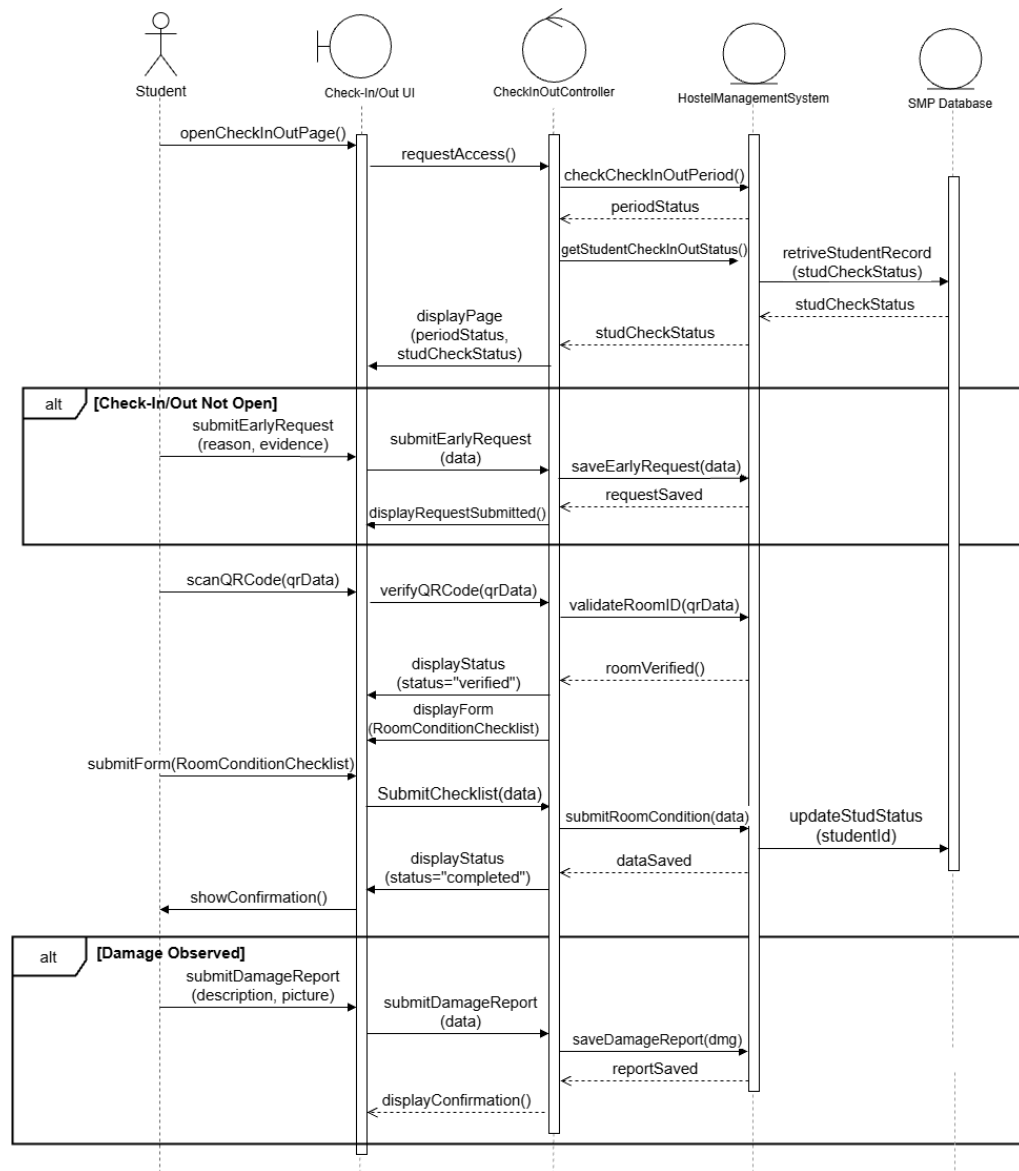


Figure 4.17 Sequence Diagram for Student Hostel Check In/ Check Out Module

Figure 4.18 below presents the admin sequence diagram for the Check-In / Check-Out module. It illustrates the interactions between Admin Staff, Admin UI, Check-In/Out Controller and Hostel Management System. The diagram captures administrative responsibilities such as setting check-in/check-out periods, reviewing early requests, approving or rejecting them, reviewing damage reports, issuing fines, sending maintenance requests, and generating operational reports. Each sequence shows how admin actions trigger system operations, update statuses in the database, and notify students, reflecting the complete lifecycle of administrative decision-making within the module.

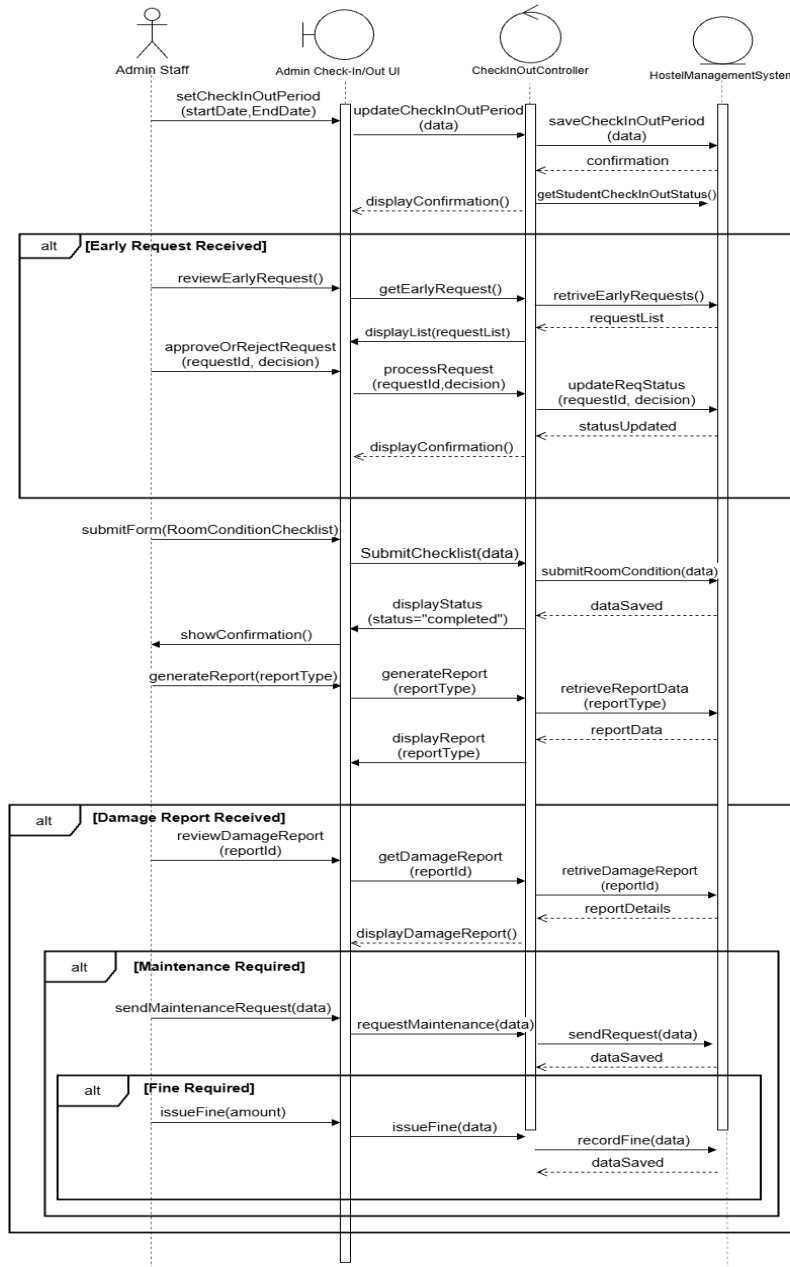


Figure 4.18 Sequence Diagram for Admin Hostel Check In/ Check Out Module

iv. Maintenance Request Module

Figure 4.19 below illustrates the sequence diagram for the Maintenance Request Module, detailing the object interactions and message flows required to process a student's report. The process initiates when the Student submits the maintenance details (category and description) via the MaintenanceUI. The MaintenanceController receives this request and first performs a critical integration

check by querying the SMP_Interface to verify the student's active status. Following this, the controller validates the input data logic.

The diagram utilizes an alternative frame (alt) to depict two potential outcomes: if the input is incomplete, the system returns an error message "Description is required" to the interface. Conversely, if the validation is successful, the controller sends a command to the MaintenanceTicket_DB to create a new record. The database generates a unique Ticket ID and returns it to the controller, which subsequently displays a success confirmation to the student, completing the transaction.

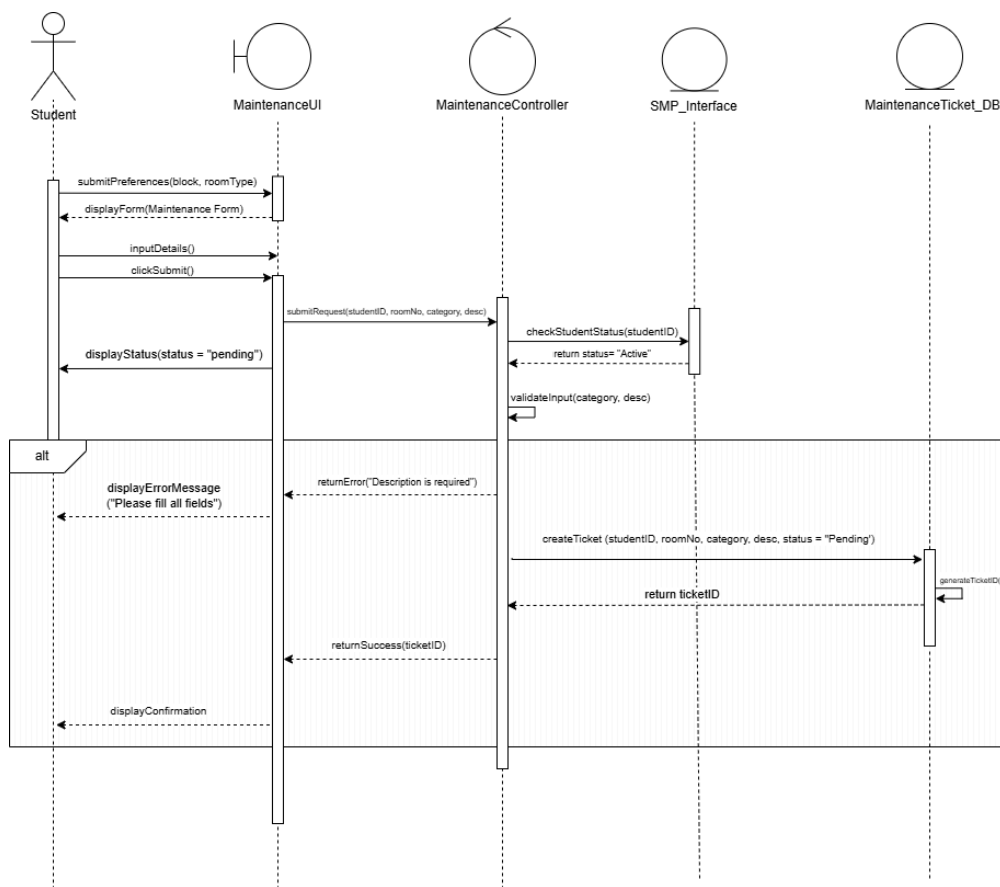


Figure 4.19 Sequence Diagram for Maintenance Request Module

v. Hostel Fee Monitoring Module

Figure 4.20 below presents the Sequence Diagram for the Hostel Fee Monitoring Module, detailing the chronological exchange of messages between the primary actors and the system components. The diagram illustrates two primary workflows: students' financial actions and administrative monitoring.

The student workflow begins with the student logging into the Hostel Portal to trigger a request for their outstanding balance, which the PaymentController retrieves from the Database. Upon deciding to proceed for payment, the system processes this transaction, updates the record status to "Paid" in the database and returns a success notification.

Simultaneously, the administrative workflow shows the Hostel Staff accessing the portal to generate a payment report. The system retrieves all student payment data, categorizing it by "Paid" and "Unpaid" status. For students flagged as "Unpaid," the staff can trigger an automated notification process through the PaymentController, which sends a reminder for payment directly to the student. This structured message flow ensures real-time data consistency between the hostel portal and the underlying financial records.

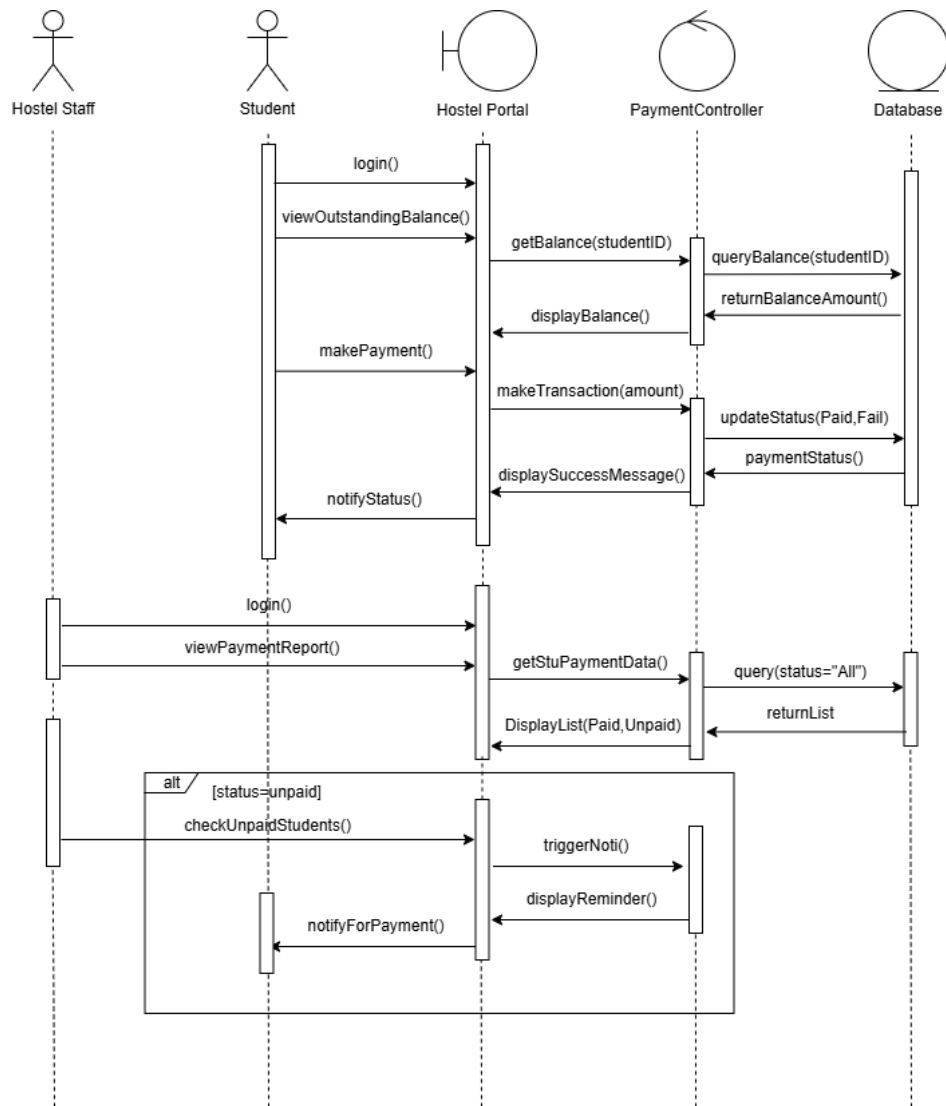


Figure 4.20 Sequence Diagram for Hostel Fee Monitoring Module

4.6.3.5 Database Design

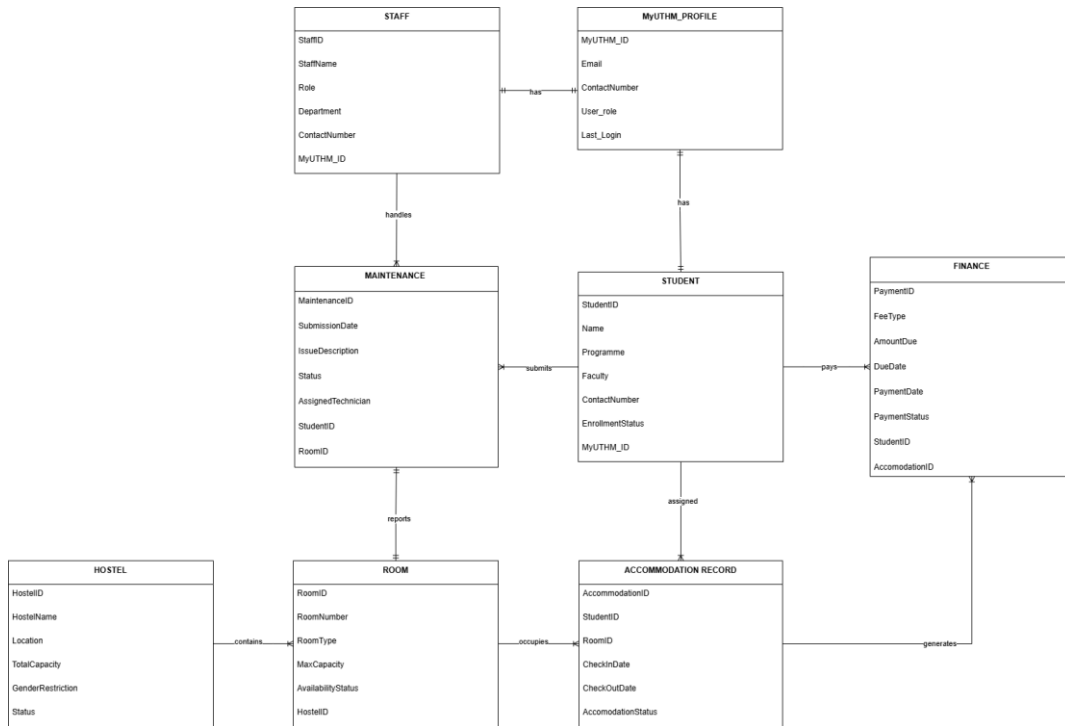


Figure 4.21 Database Design for Hostel Management System

Figure 4.21 above shows The Database Design for the Hostel Management System is structured as a relational model that ensures data integrity by connecting user identities, physical assets, and financial transactions. At the core of the system is the MyUTHM_PROFILE entity, which serves as a centralized identity hub for both STUDENT and STAFF entities, storing essential credentials such as email and user roles. The physical infrastructure is managed through the HOSTEL and ROOM entities; each hostel contains multiple rooms, which are tracked by their specific type, capacity, and current availability status.

Operational workflows are captured by the ACCOMMODATION RECORD, which acts as a bridge between students and rooms, recording critical dates for check-in and check-out alongside the current status of the stay. This record also serves as the trigger for the FINANCE entity, which automatically generates fee records, including amounts due and payment statuses, to ensure accurate billing. Furthermore, the design supports facility upkeep through the MAINTENANCE entity, allowing students to

submit issue descriptions for specific rooms that are then assigned to and handled by staff members. This interconnected structure prevents data silos by ensuring that every student movement or maintenance request is linked directly to the university's master data and financial systems.

4.6.3.6 Interface Design

This section presents the interface design of the Hostel Management System, covering both student and admin perspectives. The design focuses on providing clear, user-friendly, and efficient workflows for all system functionalities. The main modules included are Hostel Application and Approval, Room Allocation, Check-In and Check-Out, Maintenance Request, and Hostel Fee Monitoring. This section is divided into Student and Admin views. Within each view, the interfaces are further organized by module, allowing a detailed presentation of the key features and functions that support hostel management operations from both perspectives.

Student View

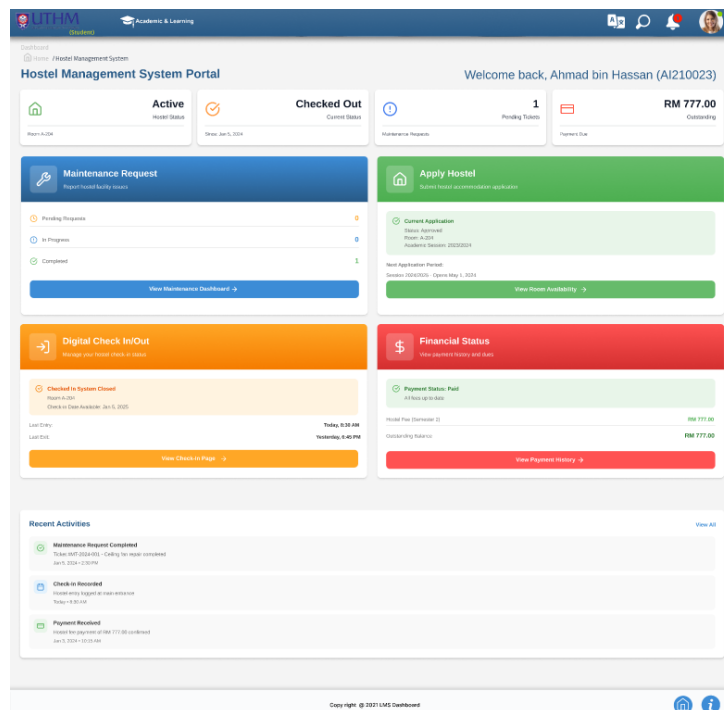


Figure 4.22 Hostel Management System Student Portal Dashboard Page

The Student Portal Dashboard for the Hostel Management System as shown in Figure 4.22 utilizes a clean and modular layout interface. At the top of the page, a series of quick statistic cards provides immediate visibility into the student's essential data, including their current room assignment, check-out status, the number of pending maintenance tickets, and any outstanding financial balances.

At the center section, the system includes a primary dashboard composed of four distinct functional modules: Maintenance Request, Hostel Application, Digital Check In/Out, and Financial Status. Each module acts as a dedicated hub, displaying a brief status update. For deeper interaction, each section has a "View" button (e.g., "View Maintenance Dashboard" or "View Payment History"), which serves as the primary gateway for the student to access more detailed records or perform specific tasks.

At the bottom section is Recent Activities feed which acts as a chronological log of the student's history within the system. This section records specific events, such as the exact time a maintenance task was completed or when a payment was successfully processed. By combining real-time statistics with clear, actionable buttons for each sub-function, the interface provides a highly organized and user-friendly environment for managing university accommodation.

1. Hostel Application Module

i. View Room Availability

The screenshot shows the 'Hostel Room Availability' page on the UTHM Academic & Learning portal. At the top, there's a navigation bar with the UTHM logo and user profile. Below the header, a breadcrumb trail shows 'Home / Hostel Management System / View Room Availability'. The main content area features a search bar and summary statistics: 4 Total Blocks, 415 Total Rooms, 190 Available Rooms, and a 54% Occupancy Rate. Two columns represent different hostel blocks, Block A and Block B. Each block lists room types with their availability status and an 'Apply' button. Block A shows Standard (RM 350/month, 29/30 available), Double (RM 450/month, 12/20 available), and Single with Bathroom (RM 600/month, 5/20 available). Block B shows Standard (RM 350/month, 29/30 available), Double (RM 450/month, 12/20 available), and Single with Bathroom (RM 600/month, 1/15 available, fully booked). A pagination bar at the bottom indicates 1 of 2 pages.

Figure 4.23 Room Availability Page

In Figure 4.23, this interface provides a display of different blocks and rooms with each with own respective availability. The status of room availability is displayed to the student.

ii. Hostel Application Form

The screenshot shows the 'Student Hostel Application' form on the UTHM Academic & Learning portal. The form is divided into two main sections: 'Personal Information' and 'Hostel Preferences'. The 'Personal Information' section includes fields for Full Name, Student ID, Email Address, Phone Number, Program of Study, Year of Study, and Gender. The 'Hostel Preferences' section includes dropdown menus for Block Preferences and Room Type. Below these, there's a file upload section for supporting documents, with a 'Choose Files' button and a list of required documents: Student ID Card (front and back), Academic transcript or offer letter, and a 'Submit Application' button. A footer note states: 'Please ensure all information is accurate before submitting. You will receive a confirmation email once your application has been processed.'

Figure 4.24 Room Application Form

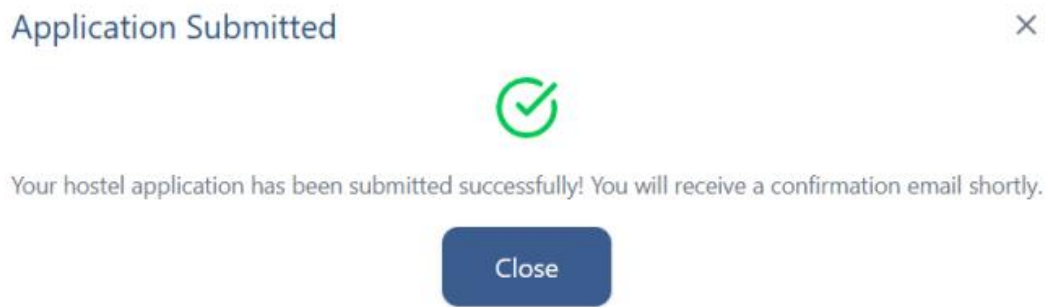


Figure 4.25 Submission Success Pop-Up Message

In Figure 4.24, Student can proceed to apply the desired hostel by clicking any apply buttons, and a hostel application form will be displayed and requires the student to fill in their personal information. A success message will be displayed completing the submission as in Figure 4.25.

ii. Upload Supporting Document

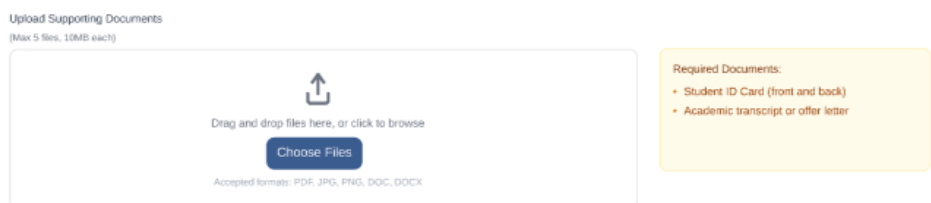


Figure 4.26 Upload Supporting Documents

When filling the hostel application form, supporting documents are required to be uploaded by the student. The student does this by clicking the 'Choose Files' button as shown in Figure 4.26.

2. Hostel Check-In/ Check-Out Module

All student-side actions for check-in and check-out follow the same basic flow. The system automatically detects the relevant date and displays the appropriate action for the student, ensuring that only the functions available for that day can be accessed.

i. Request Early Check-In or Check-Out

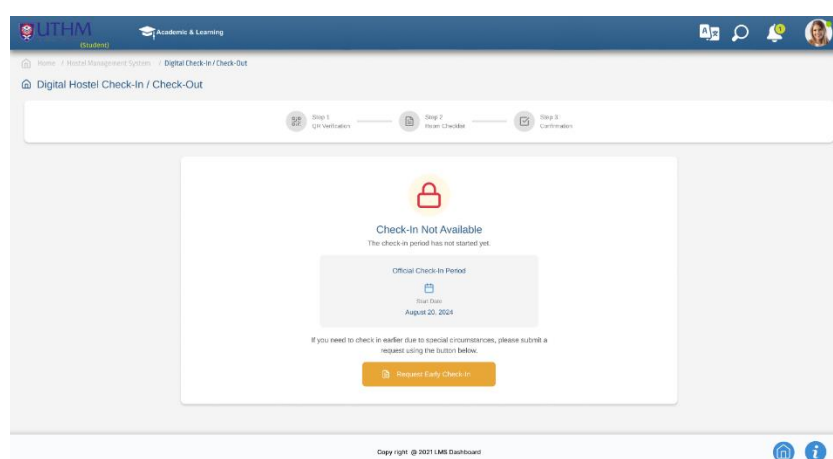


Figure 4.27 Locked Main Page of Digital Hostel Check-In or Check-Out

Figure 4.27 shows the main page of the Digital Hostel Check-In or Check-Out interface, and the check-in function is currently unavailable, as the system remains locked until the admin-set dates arrive. The page shows the official check-in date set by the admin, so students can easily see when the normal check-in period begins. Students can click the “Request Early Check-In” button if their flight arrives early or if they have curriculum requirements that require them.

The screenshot shows the LUTHM Academic & Learning portal. A pop-up window titled "Request Early Check-in" is displayed over the "Digital Hostel Check-In / Check-Out" page. The pop-up contains the following fields and instructions:

- Instructions:** Complete this form to request permission for early or late check-in. Upload supporting documents such as official letters, flight tickets, or other evidence to support your request.
- Request Type:** A dropdown menu with "Early Check-In" selected.
- Requested Start Date:** A date input field.
- Requested End Date:** A date input field.
- Reason for Request:** A text area with the placeholder "Please explain why you need early/late check-in or check-out...".
- Supporting Documents:** A section with the instruction "Upload official letters, flight tickets, or other supporting evidence (PDF or image, max 5MB)". It includes a "Click to upload or drag and drop" button and a file upload icon.
- Buttons:** "Submit Request" (orange) and "Cancel" (white).

At the bottom of the pop-up, it says "Copy right © 2021 LMS Dashboard".

Figure 4.28 Request Early Check-in or Check-out Pop-up Form

After clicking the Request button, a pop-up window appears as shown in Figure 4.28, allowing the student to enter the requested start and end dates for check-in, provide a reason, and upload supporting documents. The system automatically detects whether the request is for early check-in or early check-out. Once the student submits the request, a notification is displayed to inform the student that their request has been submitted for admin review as shown in Figure 4.29 below.

The screenshot shows a notification message on the LUTHM Digital Hostel Check-In / Check-Out page. The notification is titled "Check-In Not Available" and includes the following information:

- Icon:** A red padlock icon inside a yellow circle.
- Title:** "Check-In Not Available"
- Message:** "The check-in period has not started yet."
- Official Check-In Period:** A section with a calendar icon and the text "Start Date August 20, 2025".
- Note:** A yellow box with the text "Your early check-in request has been submitted. You will be notified once it is reviewed."

Figure 4.29 Note Update at Locked Check-in Page

ii. Perform Digital Check-In or Check-Out

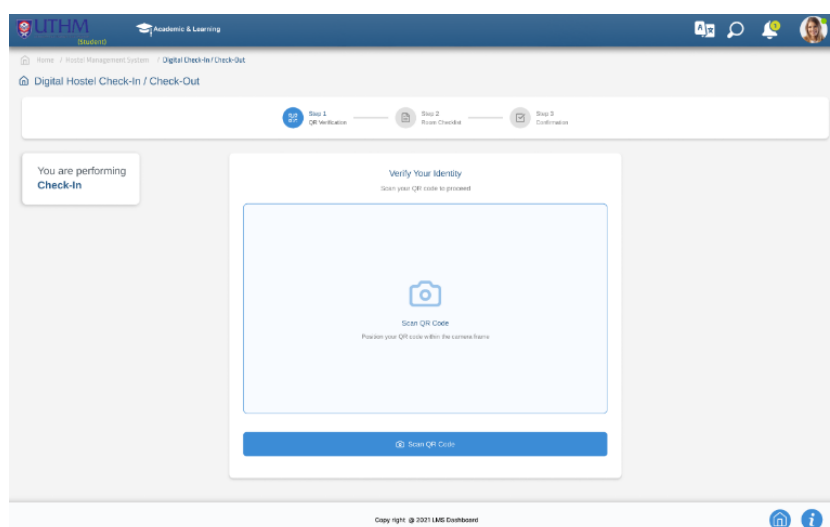


Figure 4.30 Digital Hostel Check-In or Check-Out When Access Opened

When the check-in date arrives or an early request is approved, the main page of the Digital Check-In and Check-Out system changes to a different layout, as shown in Figure 4.30. The page displays the total steps for the student at the top, with the first step being the current page, where the student must verify their identity by scanning the QR code at their hostel. A box on the left indicates that the student is currently performing check-in or check-out.

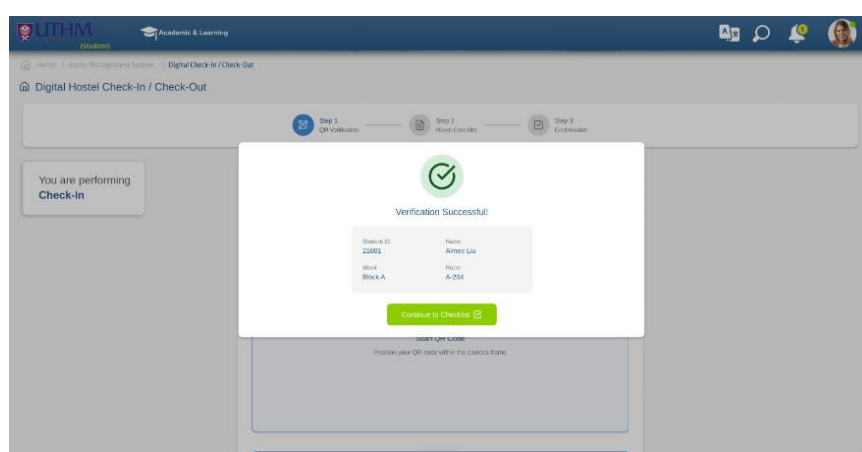


Figure 4.31 Identity Verification Successful Pop-up

Once the verification is successful, a pop-up appears confirming the success and showing the student and room details as shown in Figure 4.31. The student can then click the “Continue to Checklist” button to proceed to the next step.

iii. Submit Room Condition Checklist

The screenshot shows the 'Room Condition Checklist' page. At the top, there's a navigation bar with the UTHM logo and 'Academic & Learning' link. Below it, a breadcrumb trail reads 'Home / Hostel Management System / Digital Check-In / Check-Out'. A progress bar indicates three steps: Step 1 (QR Verification), Step 2 (Room Checklist - currently active), and Step 3 (Confirmation). On the left, a box says 'You are performing Check-In'. The main content area is titled 'Student Information' and shows details for Student ID 21003, Name Ahsan Ali, Room Block A, and Room A-204. Below this is the 'Room Condition Checklist' section, which instructs the user to 'Please inspect each item and select its current condition'. It lists ten items: Bed, Table, Chair, Fan, Light, Wall, Floor, Window, and Socket. Each item has three radio button options: 'Good', 'Minor Damage', and 'Major Damage'. At the bottom of the checklist is a 'Submit Checklist' button and a red warning message: 'Please check the condition of all items before submitting'.

Figure 4.32 Room Condition Checklist Page

Next, the student will navigate to the room condition checklist page, where all the furniture and facilities are listed as shown in Figure 4.32. The student must indicate the condition of each item as “Good”, “Minor Damage”, or “Major Damage”.

The screenshot shows the 'Check-In Complete' page. The navigation bar and breadcrumb trail are the same as in Figure 4.32. The progress bar now shows Step 1 (QR Verification) and Step 2 (Room Checklist) as completed, with Step 3 (Confirmation) as the current step. The main content area features a large green checkmark icon and the text 'Check-In Complete!'. Below this, it says 'Your room condition checklist has been submitted successfully. Welcome to your new room!'. A box displays the status 'Checked In', the date and time '11/12/2020, 11:19:50 PM', and the room 'A-204'. A 'Return to Hostel' button is located at the bottom of this box. The footer includes a copyright notice 'Copyright © 2021 LMS Dashboard' and social media icons for LinkedIn and Facebook.

Figure 4.33 Check-in or Check-Out Completed Page

Once all fields are completed, the student can submit the checklist. The system will then display the check-in complete page as shown in Figure 4.33, update the student's status, and record the submission time.

iv. Report Room Damage

The screenshot displays the UTHM Digital Hostel Check-in / Check-Out interface. At the top, the UTHM logo and navigation links are visible. A progress bar indicates three steps: Step 1 (Get Verification), Step 2 (Room Checklist), and Step 3 (Confirmation). A notification box on the left states "You are performing Check-In". The main form is titled "Room Condition Checklist" and includes a "Student Information" section with fields for Student ID (23003), Name (Ahmad Lila), Room (Block A), and Room (A-204). Below this, the "Room Condition Checklist" section prompts the user to "Please inspect each item and select its current condition". It lists various room items: Bed, Table, Chair, Fan, Light, Wall, Floor, Window, and Socket. Each item has three radio button options: "Good", "Minor Damage", and "Major Damage". The "Chair" item is currently selected for "Major Damage". For each item with a damage report, there is a "Description" field (with a placeholder "Describe the damage in detail...") and an "Upload Photo" button. At the bottom of the form, there is a "Submit Checklist" button and a red warning message: "Please check the condition of all items before submitting".

Figure 4.34 Minor and Major Damage Report in Room Condition Checklist

If any damage is observed, selecting minor or major damage requires the student to provide a description and upload a photo of the damage as shown in Figure 4.34, allowing the admin to review and take appropriate action later.

3. Maintenance Request Module

i. Submit Maintenance Ticket

The screenshot shows the 'Submit Maintenance Request' form in the UTHM Academic & Learning system. The form is titled 'Submit Maintenance Request' and has a 'Back to Dashboard' link. It includes fields for Student Name, Student ID, and Room Number. Below these are fields for Issue Category (a dropdown menu), Issue Description (a text area), and Photo Evidence (a photo upload section). The form is titled 'Submit Maintenance Request' and has a 'Back to Dashboard' link. At the bottom, there are 'Cancel' and 'Submit Request' buttons.

Figure 4.35 Maintenance Request Form

Figure 4.35 illustrates the specific interface where a student can report facility damages. The design features a digital form that requires the student to input a description of the issue. A key component of this interface is the "Upload Evidence" function, which allows students to attach photo proof of the damage before clicking the "Submit" button, ensuring the administration has a clear visual context of the problem.

The screenshot shows the 'Issue Category' dropdown list. The dropdown is open, showing a list of categories: Electrical, Plumbing, Furniture, Internet, and Other. The 'Electrical' category is currently selected.

Figure 4.36 Dropdown List of Issue Category Selection

Figure 4.36 displays the categorization feature within the form. Students are required to select a specific damage category from a dropdown list (e.g., Electrical, Plumbing, Furniture, Internet). This classification helps ensure the request is accurately described for the maintenance team.

ii. View Ticket Status

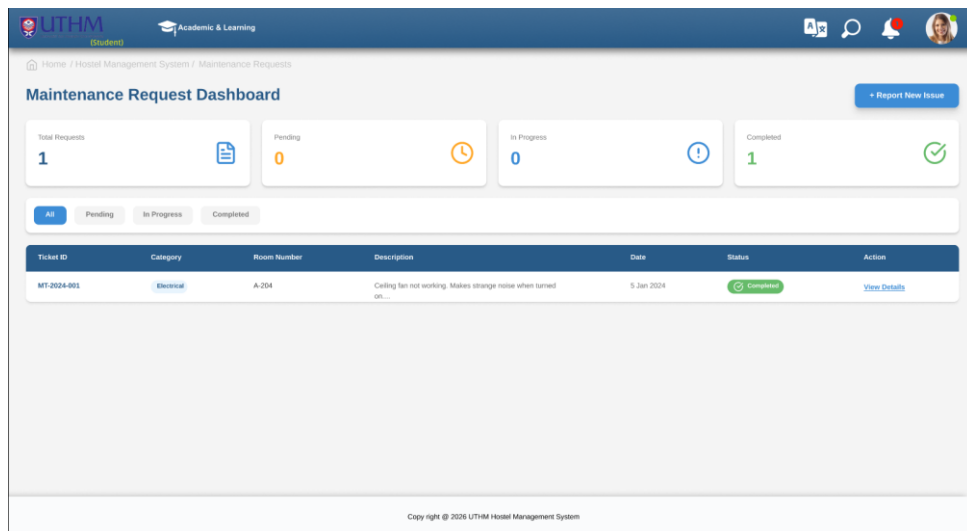


Figure 4.37 Maintenance Request Dashboard

Figure 4.37 displays the Maintenance Request Dashboard page. This interface is designed to provide students with transparency regarding their reported issues. It lists the student's entire history of maintenance tickets and displays the current real-time status of each request (e.g., Pending, In Progress, or Completed).

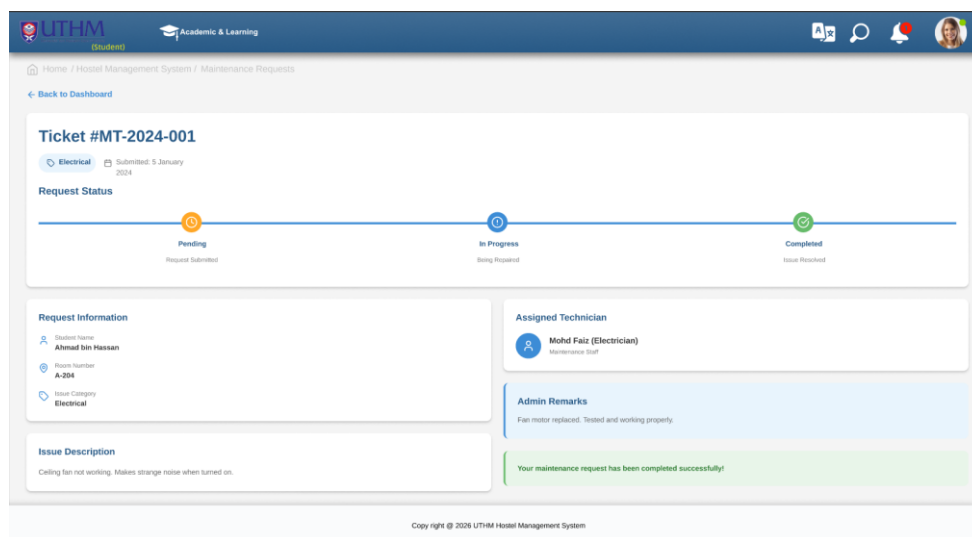


Figure 4.38 Ticket Details

Figure 4.38 shows the detailed view of a specific maintenance record. By selecting a ticket from the dashboard, the student can view the full details of the reported issue. This allows the student to monitor the progress of the repair remotely without needing to physically visit the administrative office.

4. Hostel Fee Monitoring Module

i. View Outstanding Balance in Detail

This interface provides students with a clear view of their current financial standing. It provides a detailed breakdown of outstanding balances such as hostel fees and electric charges.

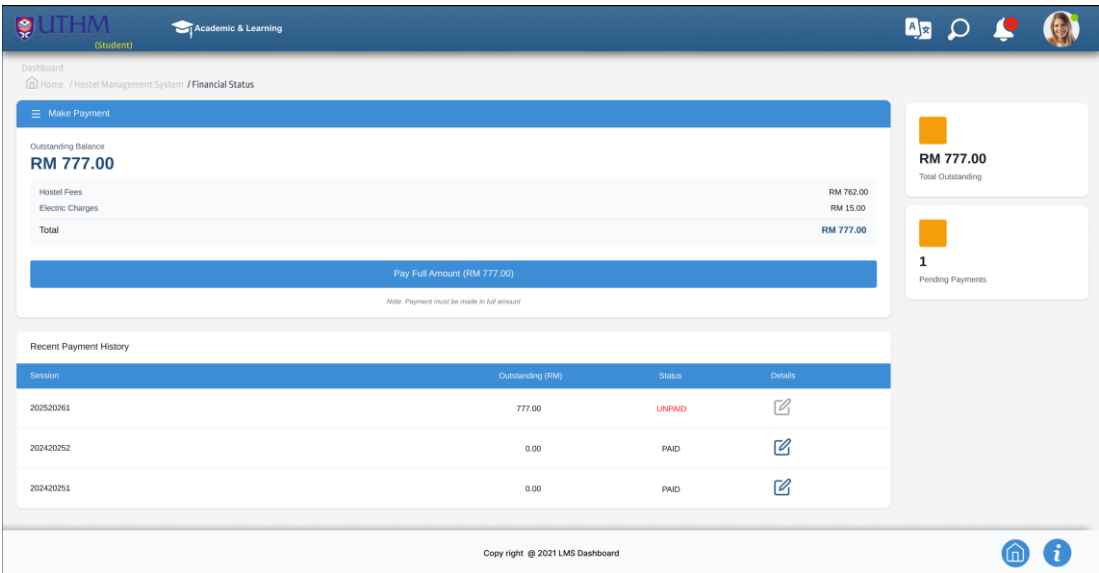


Figure 4.39 Financial Status Page

ii. Make Payment

Students can proceed to payment here with a single click on a "Pay Full Amount" button.

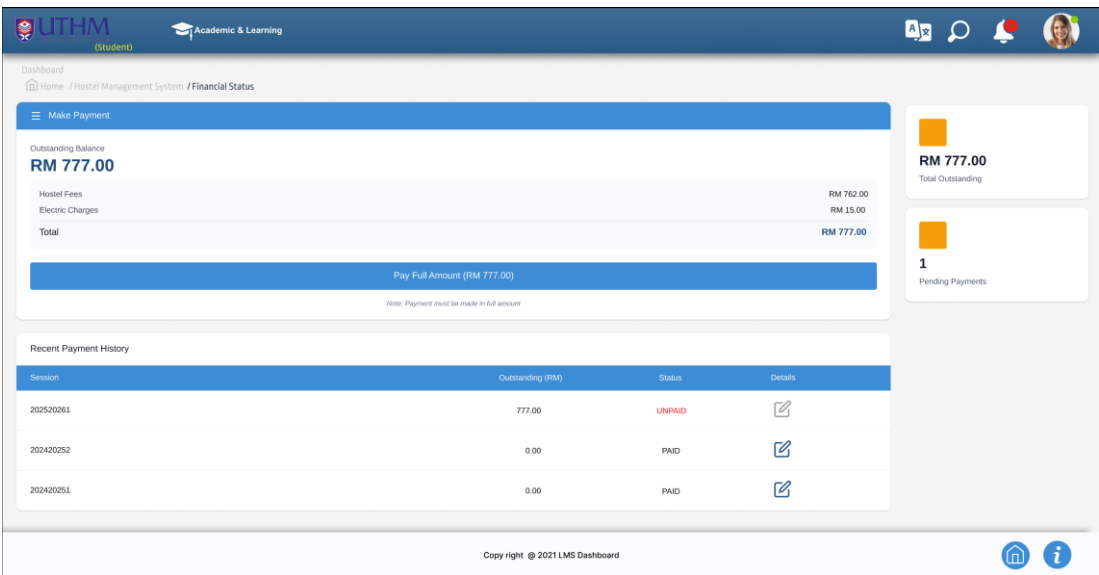


Figure 4.40 Financial Status Page

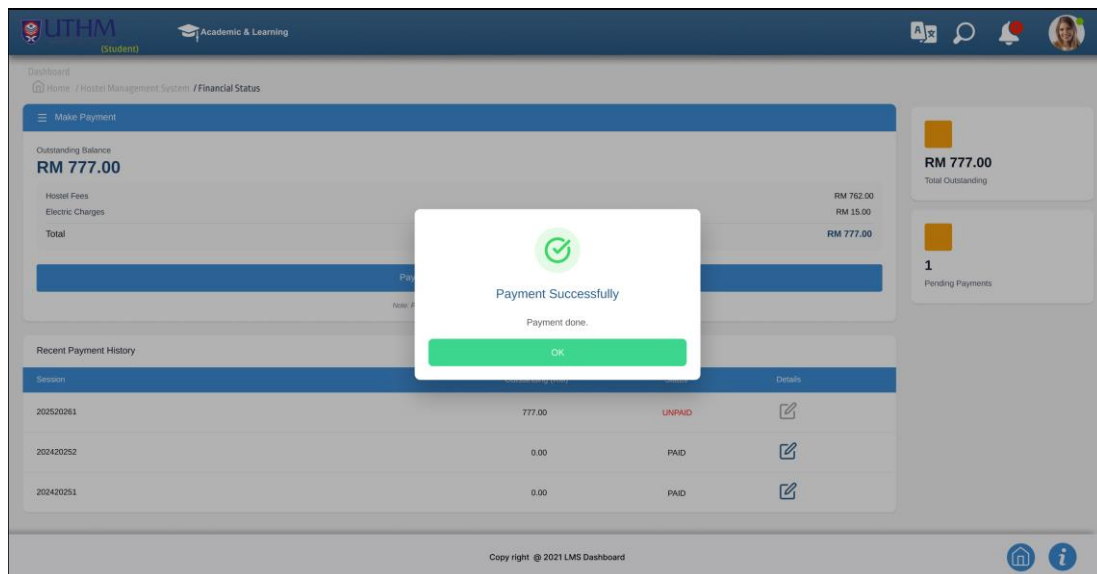


Figure 4.41 Successful Payment Pop-up Message

iii. View Past Transaction Receipt

Students could also view the status and receipt of past billing sessions in the recent payment history log. Students can get their previous receipts by clicking on the icon in “Details” column. It provides a formal record of the transaction. Students have the option to download or print the document for their personal records.

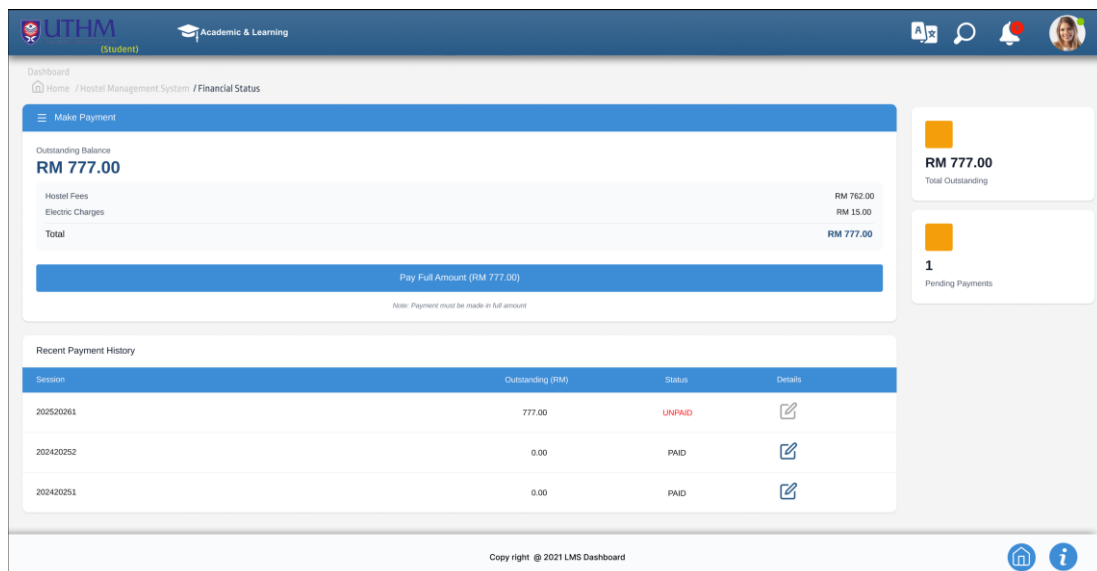


Figure 4.42 Financial Status Page

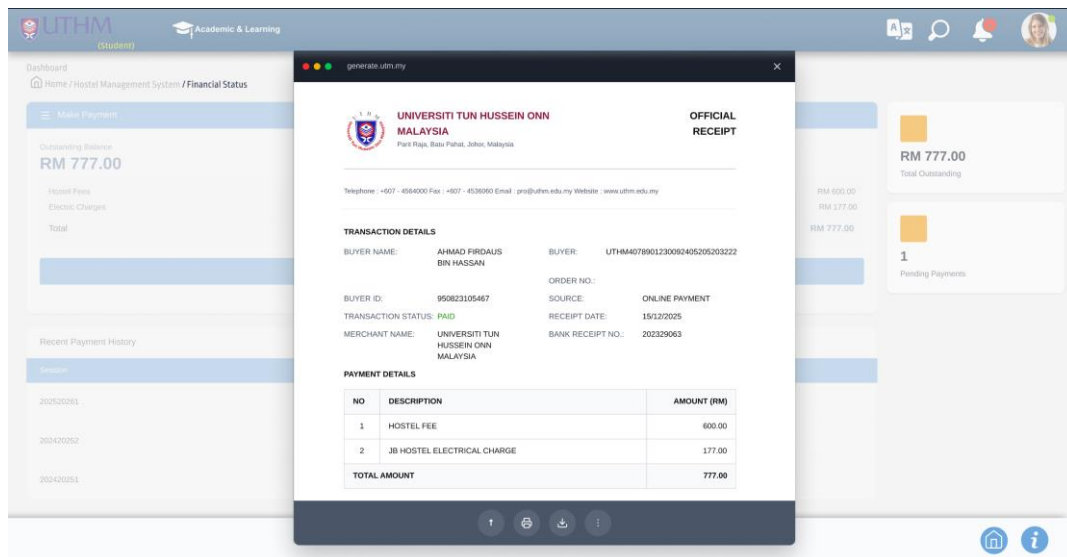


Figure 4.43 The Official Receipt Generated After a Successful Payment

iv. View Notification Bar

This notification bar displays administrative alerts and reminders regarding outstanding fees and approaching deadlines.

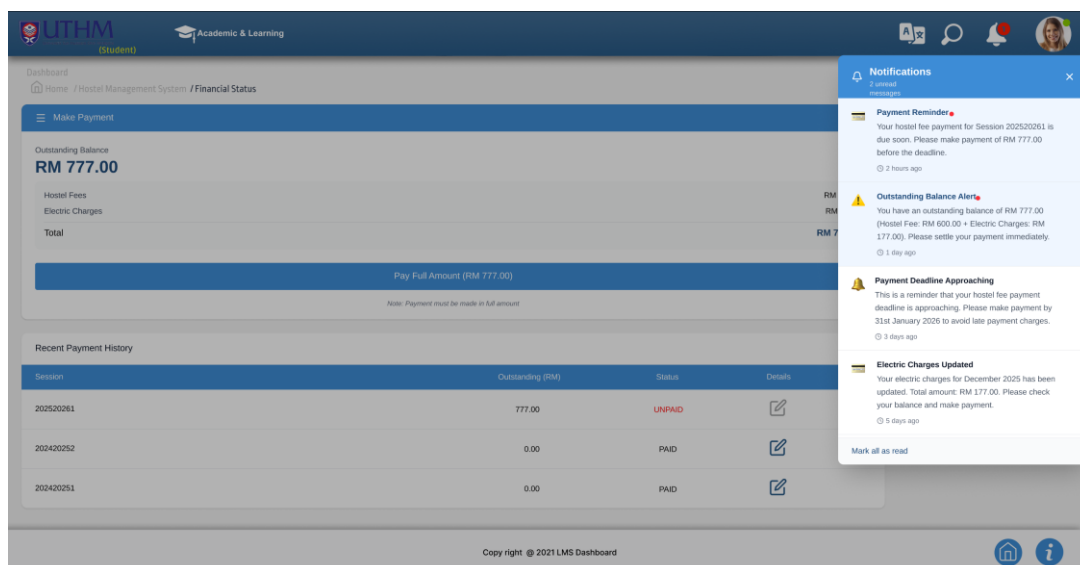


Figure 4.44 Notification Bar

Admin View

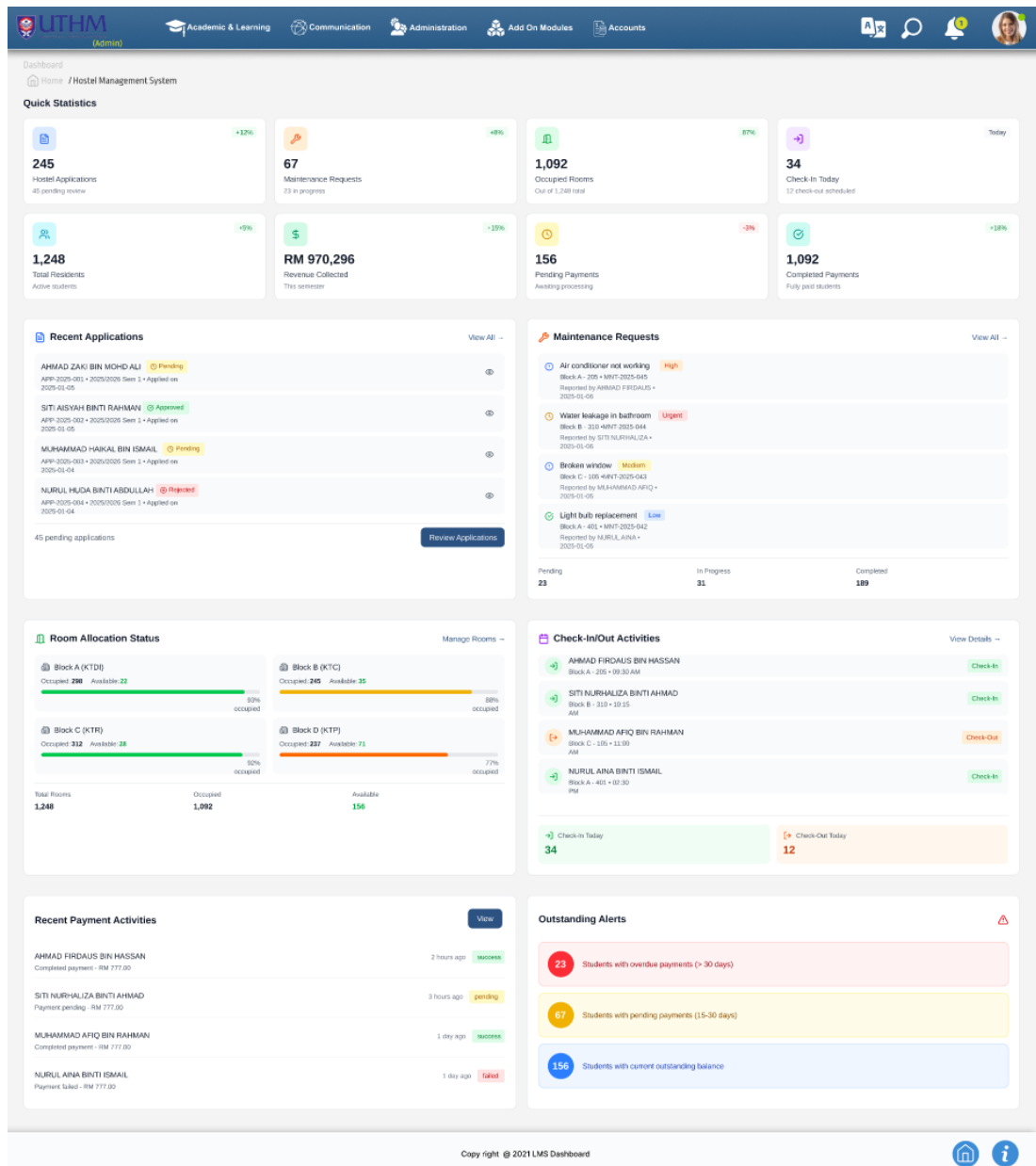


Figure 4.45 Hostel Management System Admin Portal Dashboard Page

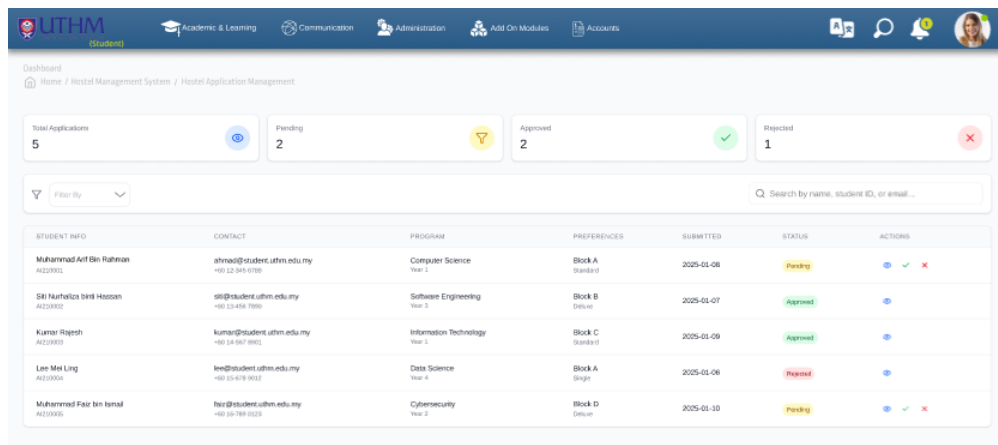
The admin interface of the Hostel Management System is structured to provide administrators with overview of facility operations and student management through a data-driven layout as shown in Figure 4.45. Similar to the student view, the page begins with a quick statistics section at the top, consisting of eight data cards that track essential metrics such as total hostel applications, active maintenance requests, room occupancy rates, and total revenue collected for the semester. These cards use color-coded percentage indicators to highlight growth or declines in specific areas, allowing for immediate performance monitoring.

The middle and lower sections of the dashboard are organized into several functional modules that provide summaries of ongoing administrative tasks. These include a "Recent Applications" pane for tracking student admission statuses, a "Maintenance Requests" module categorized by urgency (High, Urgent, Medium, Low), and a "Room Allocation Status" section that provides a visual occupancy breakdown by residential block. Additionally, a "Check-In/Out Activities" feed and a "Recent Payment Activities" log offer real-time tracking of student movements and financial transactions, each equipped with "View All" or "Manage" links to access management tools.

The interface includes Outstanding Alerts section at the bottom right. This module serves as a notification hub to categorizing urgent administrative concerns. By centralizing these diverse operational data points into a single dashboard, the interface enables administrators to efficiently pivot between facility maintenance, financial oversight, and student residency management.

1. Hostel Application Module

i. View Applications



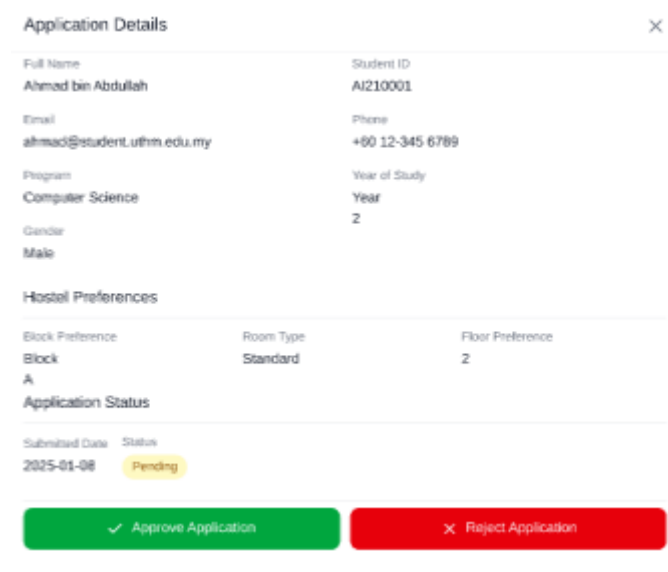
The screenshot shows the UTHM Hostel Application Management dashboard. At the top, there's a navigation bar with links to Academic & Learning, Communication, Administration, Add On Modules, and Accounts. Below the navigation bar, there's a summary section with four cards: Total Applications (5), Pending (2), Approved (2), and Rejected (1). Each card has a corresponding icon. Below the summary section, there's a table with columns: STUDENT INFO, CONTACT, PROGRAM, PREFERENCES, SUBMITTED, STATUS, and ACTIONS. The table contains five rows of application data.

STUDENT INFO	CONTACT	PROGRAM	PREFERENCES	SUBMITTED	STATUS	ACTIONS
Muhammad Afi Bin Rahman AI210002	ahmad@student.uthm.edu.my +60 12 345 6789	Computer Science Year 1	Block A Standard	2025-01-08	Pending	View Check Delete
Siti Nurhaliza binti Hassan AI210003	siti@student.uthm.edu.my +60 13 456 7890	Software Engineering Year 2	Block B Deluxe	2025-01-07	Approved	View
Kumar Rajesh AI210003	kumar@student.uthm.edu.my +60 14 567 8901	Information Technology Year 1	Block C Standard	2025-01-09	Approved	View
Lee Mei Ling AI210004	lee@student.uthm.edu.my +60 15 678 9012	Data Science Year 4	Block A Single	2025-01-06	Rejected	View
Muhammad Fair bin Ismail AI210005	fair@student.uthm.edu.my +60 16 789 0123	Cybersecurity Year 2	Block D Deluxe	2025-01-10	Pending	View Check Delete

Figure 4.46 View Applications

In Figure 4.46, the admin can see the list of applications submitted by the students, and see each application details by clicking the eye icon.

ii. Accept or Reject Applications



The screenshot shows the 'Application Details' modal form. It contains fields for Full Name, Student ID, Email, Phone, Program, Year of Study, Gender, and Hostel Preferences (Block Preference, Room Type, Floor Preference). Below these fields is the 'Application Status' section, which shows the Submitted Date (2025-01-08) and the Status (Pending). At the bottom of the modal, there are two buttons: 'Approve Application' (green) and 'Reject Application' (red).

Application Details		
Full Name	Student ID	
Ahmad bin Abdullah	AI210001	
Email	Phone	
ahmad@student.uthm.edu.my	+60 12-345 6789	
Program	Year of Study	
Computer Science	Year	
Gender		
Male		
Hostel Preferences		
Block Preference	Room Type	Floor Preference
Block	Standard	2
Application Status		
Submitted Date	Status	
2025-01-08	Pending	
Approve Application Reject Application		

Figure 4.47 Accept or Reject Applications

The admin is given two buttons 'Accept Application' and 'Reject Application' as shown in Figure 4.47. The status of the application will be updated and displayed in the list as shown in previous Figure 4.46.

2. Room Allocation Module

i. View Room Availability

Admin can view all the room status that shows whether rooms are available or full, along with resident details and capacity.

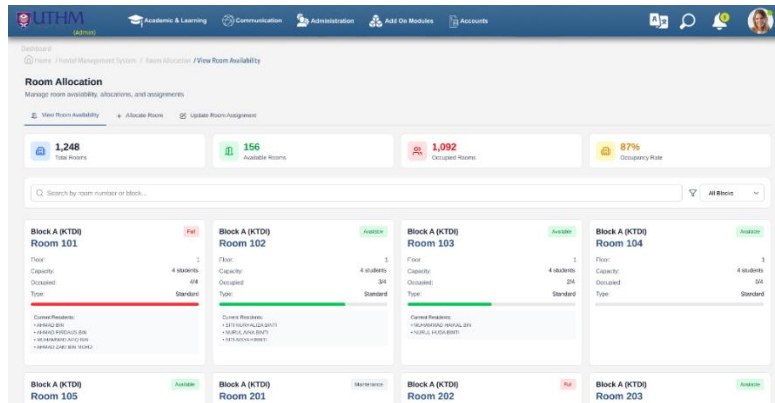


Figure 4.48 View Room Availability

ii. Allocate Room

In the Allocate Room tab, available rooms display an "+ Allocate" button that the admin can click to assign a student to that room.

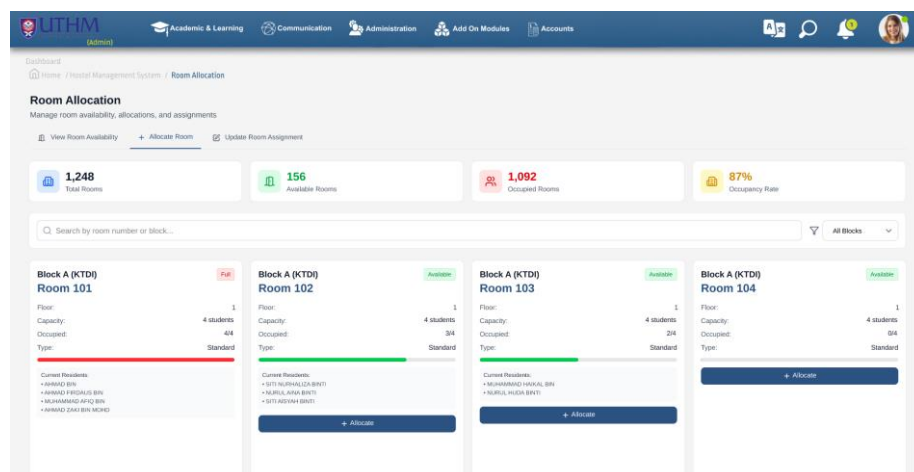


Figure 4.49 Allocate Room Page

After clicking the button, the system prompts a form that requires the admin to enter the Student ID/Name, select the Session, and choose the Allocation Date, followed by pressing the "Allocate Room" button.

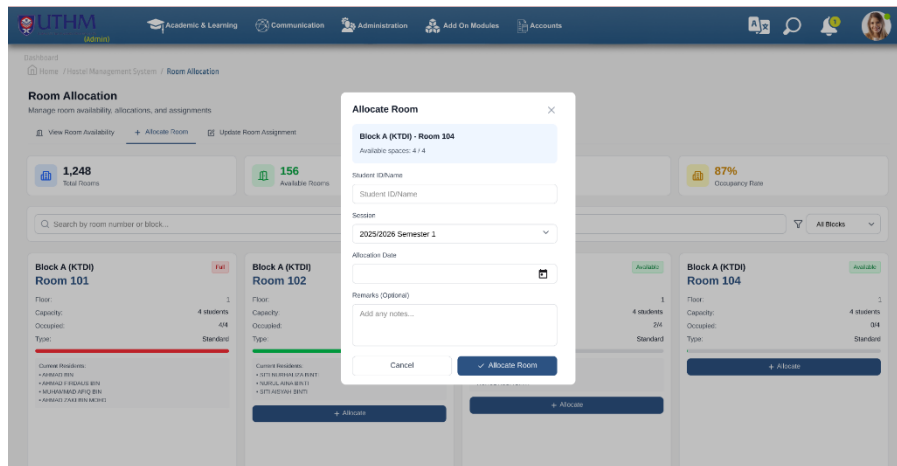


Figure 4.50 Allocate Room Form Pop-up Window

The system then validates the action and displays a "Room Allocated Successfully" confirmation pop-up to verify that the student has been assigned to the room.

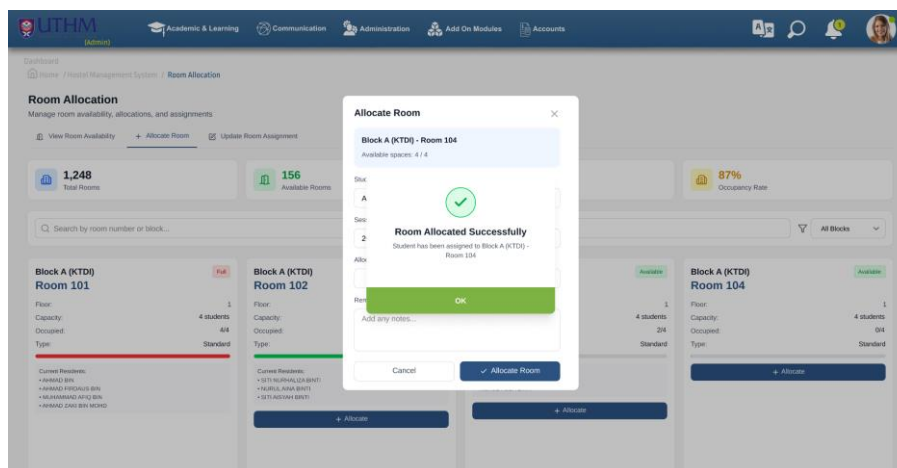


Figure 4.51 Room Allocate Successfully Pop-Up Window

Finally, the dashboard automatically updates to reflect the new occupancy status (1/4) and lists the newly assigned student's name (Ali Bin Abu) under the room details.

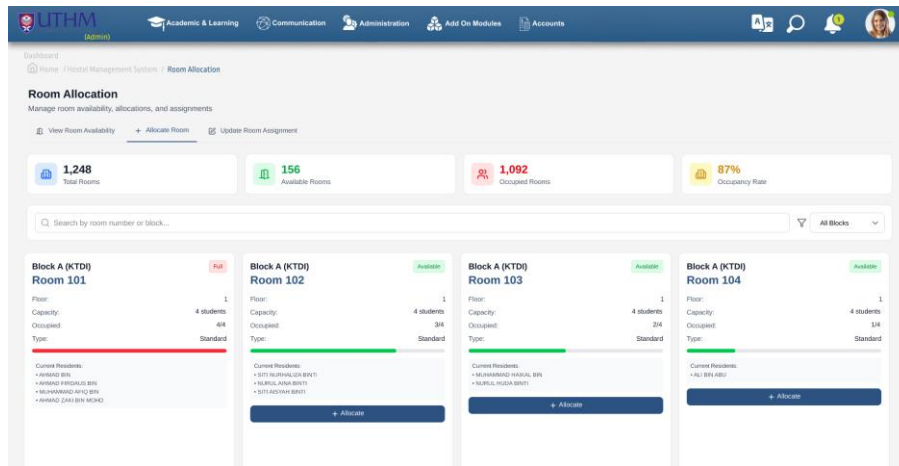


Figure 4.52 Updated Room Allocation Page

iii. Update Room Assignment

In the Update Room Assignment tab, each room card displays an orange "Update" button that the admin can click to modify the existing student assignments for that room.

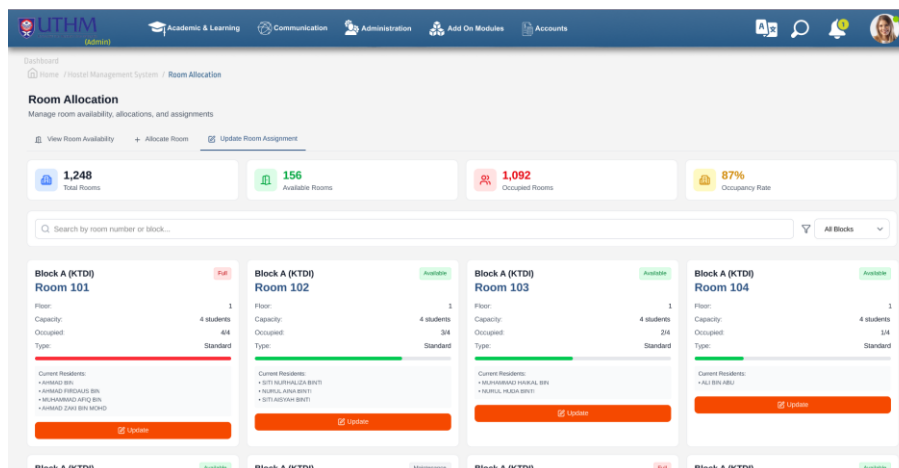


Figure 4.53 Update Room Assignment Page

After clicking the button, an "Update Room Assignment" window appears, requiring the admin to select a specific "Current Resident," choose the "Action" to take, set the "Effective Date," and enter a "Reason" for the change.

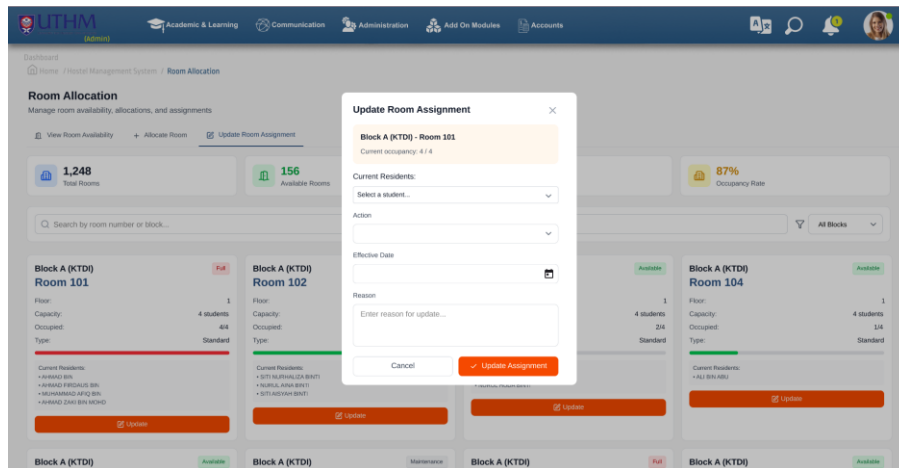


Figure 4.54 Update Room Assignment Form Pop-up Window

Once the "Update Assignment" button is pressed, the system confirms the action with a pop-up stating "Assignment Updated Successfully" to verify that the changes have been saved.

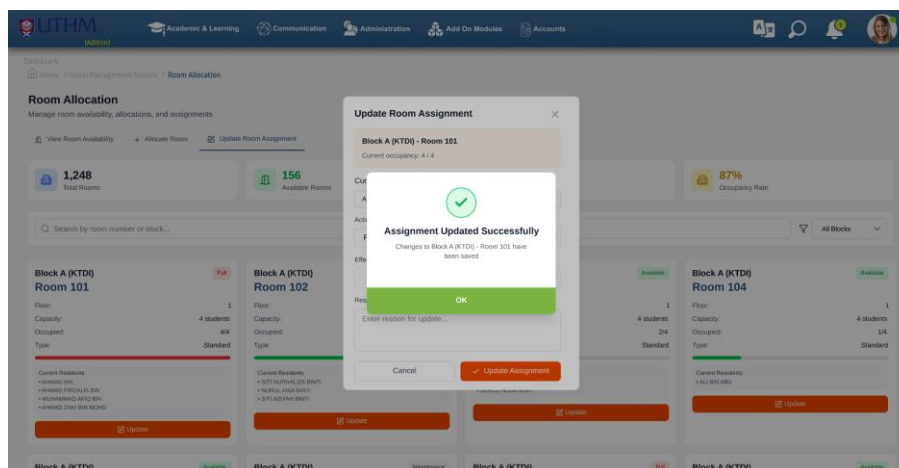


Figure 4.55 Assignment Update Successfully Pop-Up Window

Finally, the dashboard automatically updates to reflect the changes; for example, Room 101 is now marked as "Available" with 3/4 occupancy, and the removed student's name is no longer listed in the resident details.

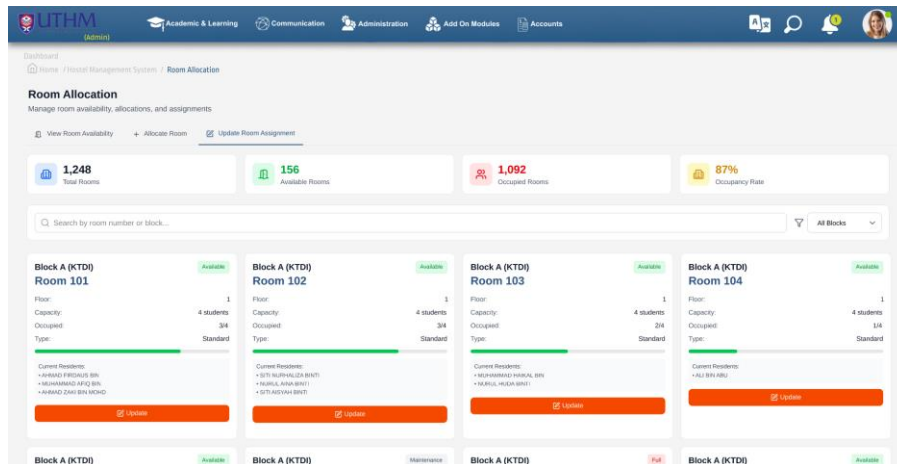


Figure 4.56 Updated Room Assignment Page

3. Check In/ Check Out Module

i. Set Check-In and Check-Out Period

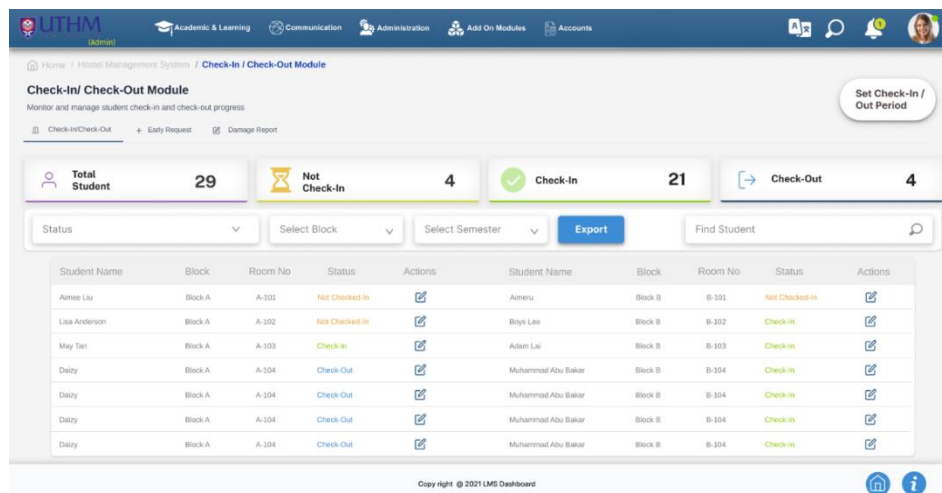


Figure 4.57 Check-in and Check-out Page

Figure 4.57 shows the main page of the Check-In and Check-Out module, which displays statistics and a student list at the bottom. The centre section includes sorting options by student status, block, and semester, allowing the admin to view the list based on selected criteria. A “Set Check-In and Check-Out Period” button is also provided to access the period management function.

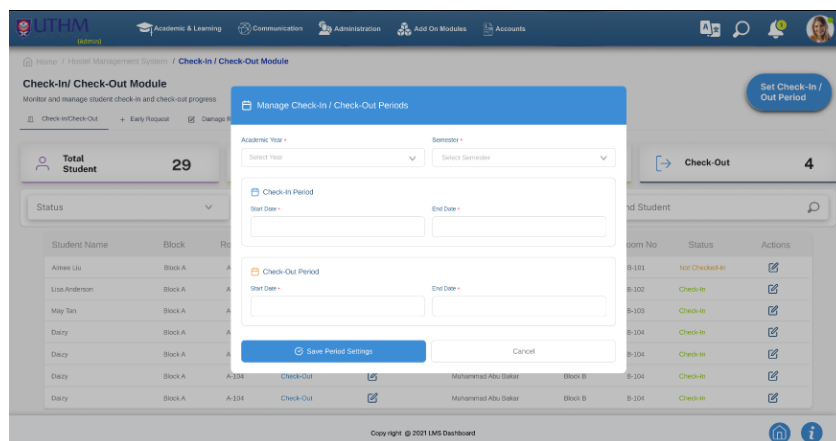


Figure 4.58 Set Check-in and Check-out Period Pop-up Window

When the button is clicked, a pop-up window will appear as shown in Figure 4.58. The admin can manage the check-in and check-out period by selecting the academic year and semester and entering the start and end dates for both periods. After saving the settings, the system will automatically update and open or close the check-in and check-out functions for students according to the configured dates.

ii. Review Early Check-In or Check-Out Request

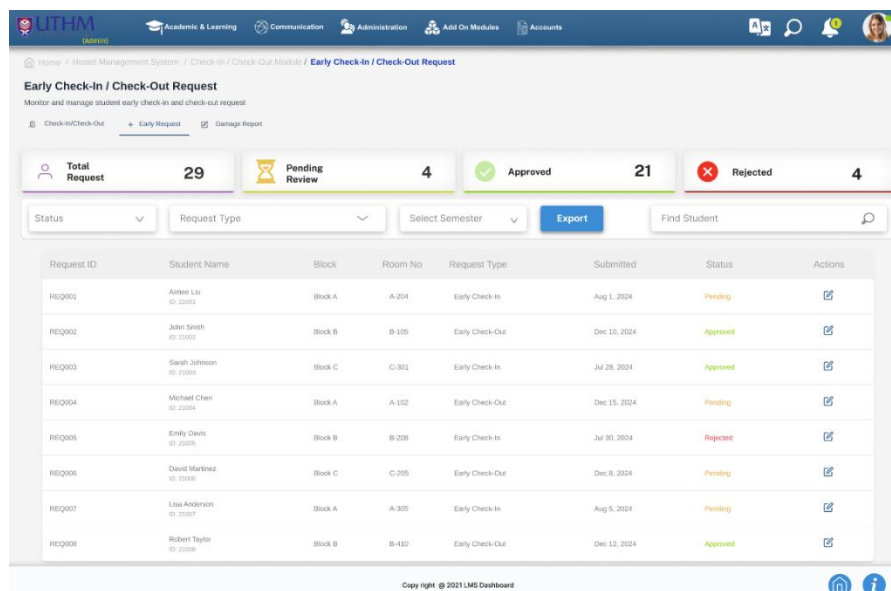


Figure 4.59 Early Check-in and Check-out Page

This interface as shown in Figure 4.59 is the Review Early Check-In and Check-Out Request page. The page layout is similar to the main Check-In and Check-Out module, including statistics and sorting options such as status, request type, and

semester. The early request list is displayed at the bottom, and the admin can press the action button for each request to review the details.

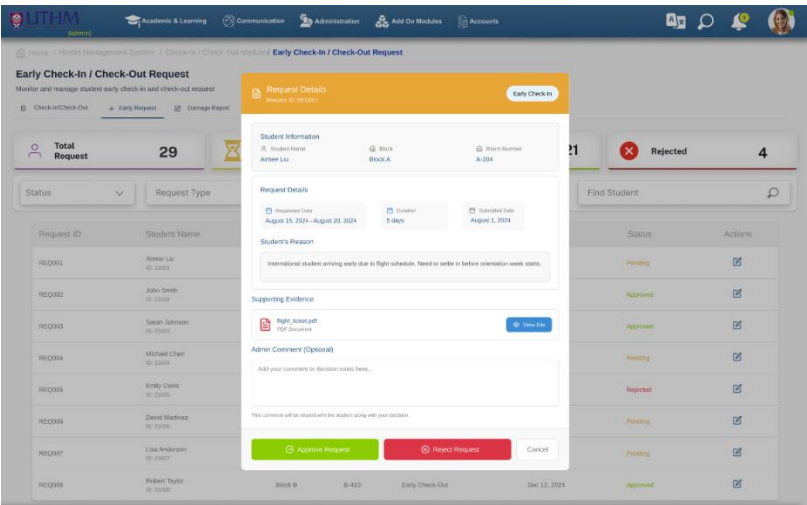


Figure 4.60 Student's Early Request Details Pop-up Window

After the admin clicks the action button, a pop-up window will appear, as shown in Figure 4.60. The window displays the student information and request details submitted by the student, together with the supporting evidence file. The “View File” button allows the admin to open and review the uploaded evidence. The admin can then choose to approve or reject the request. Once a decision is made, the system will notify the student. If the request is approved, the system will allow the student to access the check-in or check-out page earlier than the normal period.

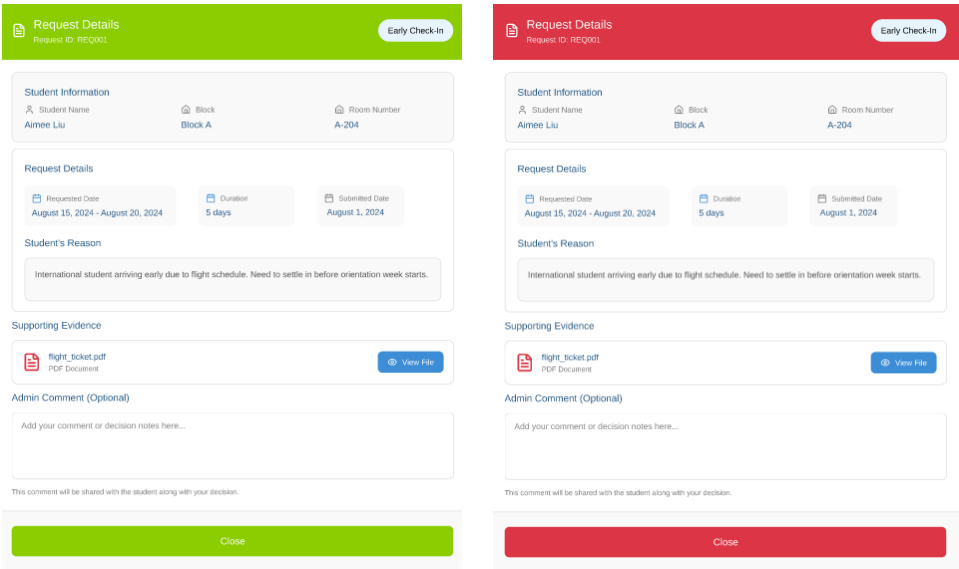
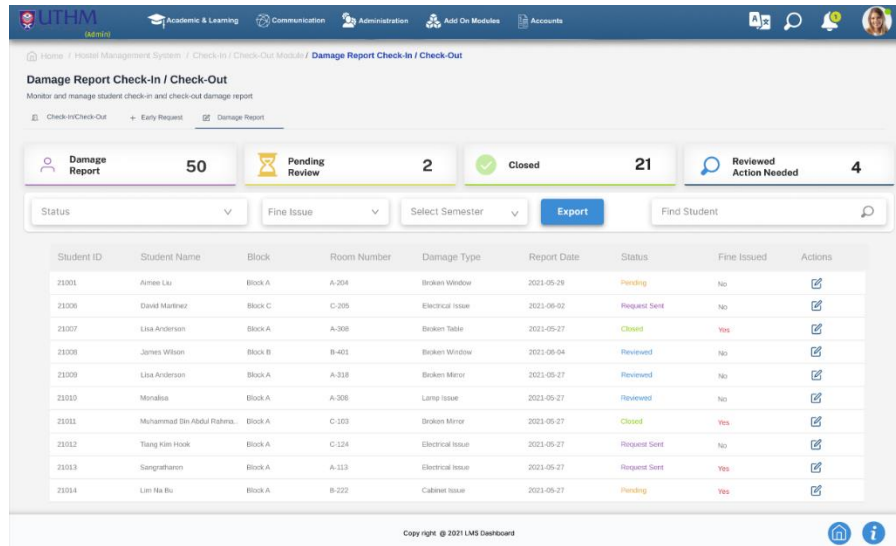


Figure 4.61 Confirmed Request and Rejected Request Pop-up Window

The confirmed and rejected request records are shown in Figure 4.61. Confirmed requests are highlighted in green, while rejected requests are highlighted in red, allowing the admin to easily differentiate the request status.

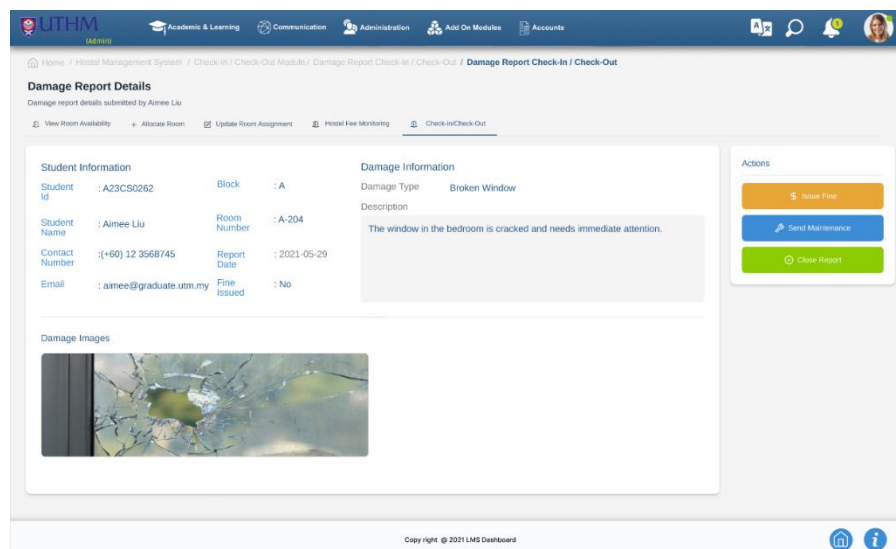
iii. Review Damage Report



Student ID	Student Name	Block	Room Number	Damage Type	Report Date	Status	Fine Issued	Actions
21001	Aimee Liu	Block A	A-204	Broken Window	2021-05-28	Pending	No	View
21009	David Martinez	Block C	C-205	Electrical Issue	2021-06-02	Request Sent	No	View
21007	Lisa Anderson	Block A	A-308	Broken Table	2021-05-27	Closed	Yes	View
21009	James Wilson	Block B	B-401	Broken Window	2021-05-04	Reviewed	No	View
21009	Lisa Anderson	Block A	A-318	Broken Mirror	2021-05-27	Reviewed	No	View
21010	Minahsa	Block A	A-308	Lamp Issue	2021-05-27	Reviewed	No	View
21011	Muhammad Bin Abdul Rahman	Block A	C-103	Broken Mirror	2021-05-27	Closed	Yes	View
21012	Tsang Kim Hock	Block A	C-124	Electrical Issue	2021-05-27	Request Sent	No	View
21013	Sangnaphann	Block A	A-113	Electrical Issue	2021-05-27	Request Sent	Yes	View
21014	Lin Ha Bu	Block A	B-222	Cabinet Issue	2021-05-27	Pending	Yes	View

Figure 4.62 Damage Report Page

Next is the Review Damage Report interface as shown in Figure 4.62. Similar to the previous pages, it includes statistics and sorting options such as status, fine issued, and semester. The list of reported damages is displayed at the bottom, and the admin can press the action button for each report to review the details.



Student Information

Student ID : A23CS0262

Student Name : Aimee Liu

Contact Number : (+60) 12 3568745

Email : aimee@graduate.utm.my

Block : A

Room Number : A-204

Report Date : 2021-05-29

Fine Issued : No

Damage Information

Damage Type : Broken Window

Description : The window in the bedroom is cracked and needs immediate attention.

Damage Images



Actions

[Issue Fine](#)

[Send Maintenance](#)

[Close Report](#)

Figure 4.63 Damage Report Details Page

After clicking the action button, the system will navigate to the damage report details page as shown in Figure 4.63. The student information and damage details will be shown, together with the damage images submitted by the student. On the right side of the page, there are three action options for the admin to choose from: issue a fine to the student, send a maintenance request to the maintenance team, or close the report. The admin can close the report if the damage is not serious, can be handled directly, or if no further action is required.

iv. Issue Fine

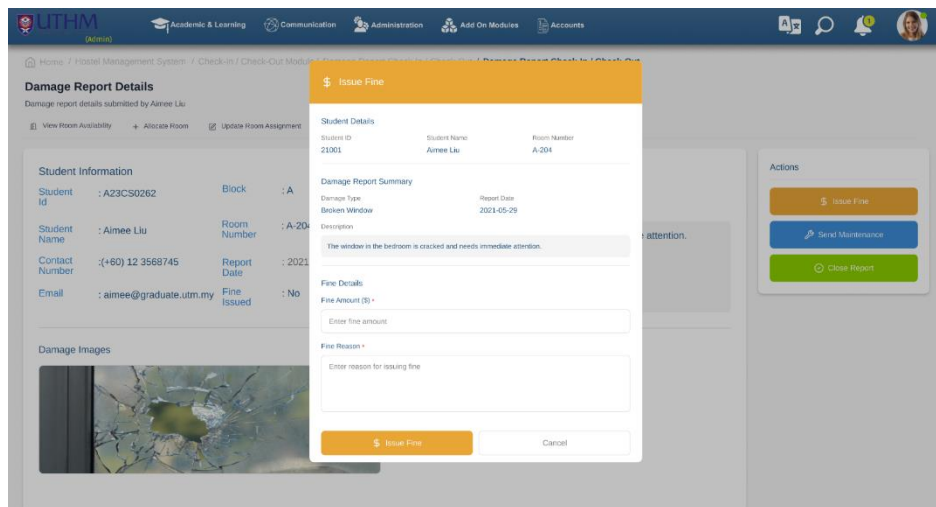


Figure 4.64 Issue Fine Pop-up Window

If the admin chooses to issue a fine, a pop-up window will be shown as shown in Figure 4.64. It will display the student and damage details again, and the admin will be able to enter the fine amount and the reason for the fine. After clicking the “Issue Fine” button, the system will update the record and notify the student.

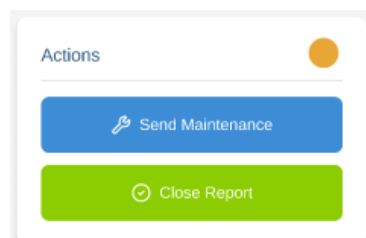


Figure 4.65 Action Box after Issue Fine Complete

Once the fine has been issued, the action options will no longer include the Issue Fine option as shown in Figure 4.65 above. The admin will be able to click the orange indicator to view the fine issue history.

v. Send Maintenance Request

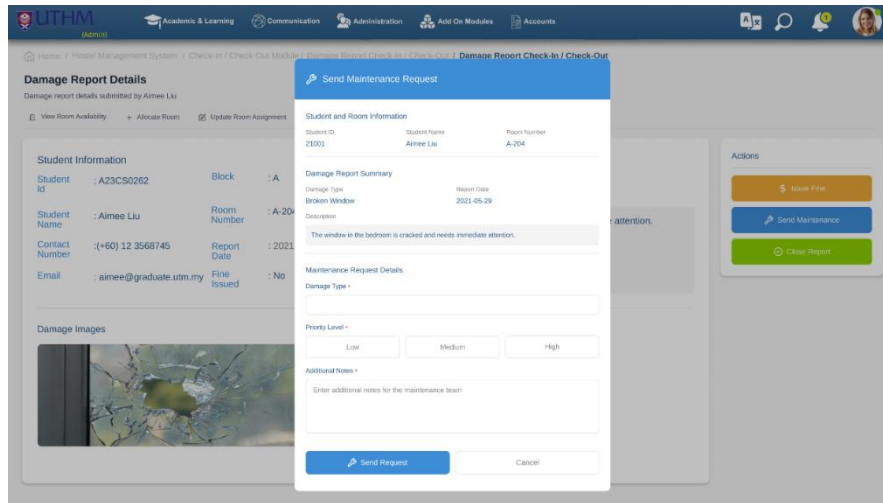


Figure 4.66 Send Maintenance Request Pop-up Window

If the admin chooses to send a maintenance request, a pop-up window will be shown as shown in Figure 4.66. It will display the student and room information together with the damage report summary. The admin can enter the damage type, select the priority level, and provide additional notes. After clicking the “Send Request” button, the request will be sent to the maintenance team.

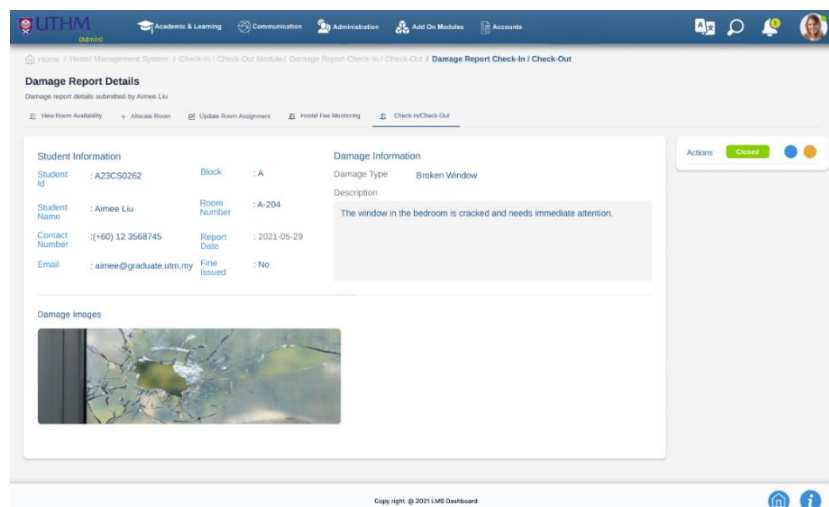


Figure 4.67 Damage Report Details Page with All Action Completed

Similar to the fine function, once the maintenance request has been sent, the action options will no longer include this option as shown in Figure 4.67. The admin can also click the blue indicator to view the maintenance request history. The “Close Report” action is required for every damage report, as it serves to confirm that the admin has reviewed the report and considered all necessary actions.

vi. Generate Operational Report

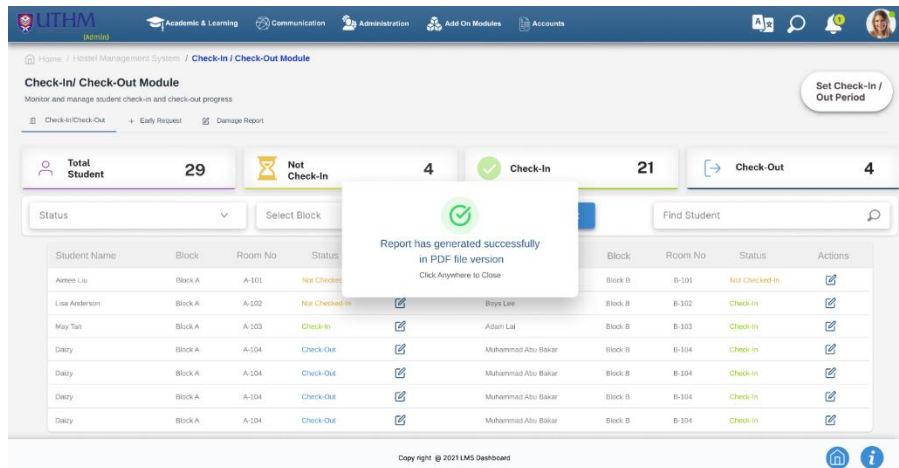


Figure 4.68 File Generate Successfully Pop-up

For the generate operational report function, each of the pages with sorting options mentioned earlier will include an “Export” button on the side. This allows the admin to generate a report based on the current sorting, or they can export the full report without applying any sorting. After clicking the “Export” button, a pop-up will be shown confirming that the file has been successfully generated.

4. Maintenance Request Module

i. View Incoming Request

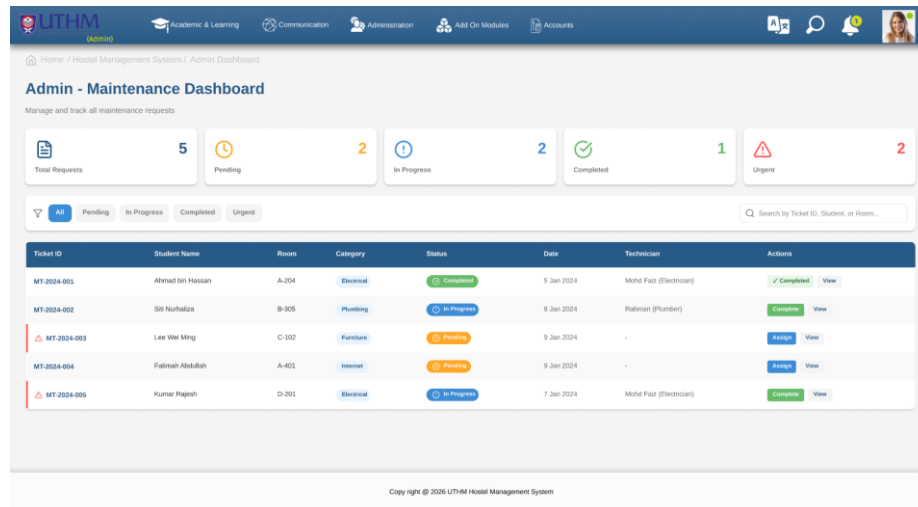


Figure 4.69 Maintenance Dashboard for Admin

Figure 4.69 illustrates the central dashboard used by administrators to view incoming maintenance requests. The interface compiles a list of all tickets submitted by students. It includes filtering capabilities that allow the administrator to sort requests by status (such as filtering for only "Pending" items) or urgency. This visual organization assists the admin staff in prioritizing tasks effectively.

ii. Assign Technician

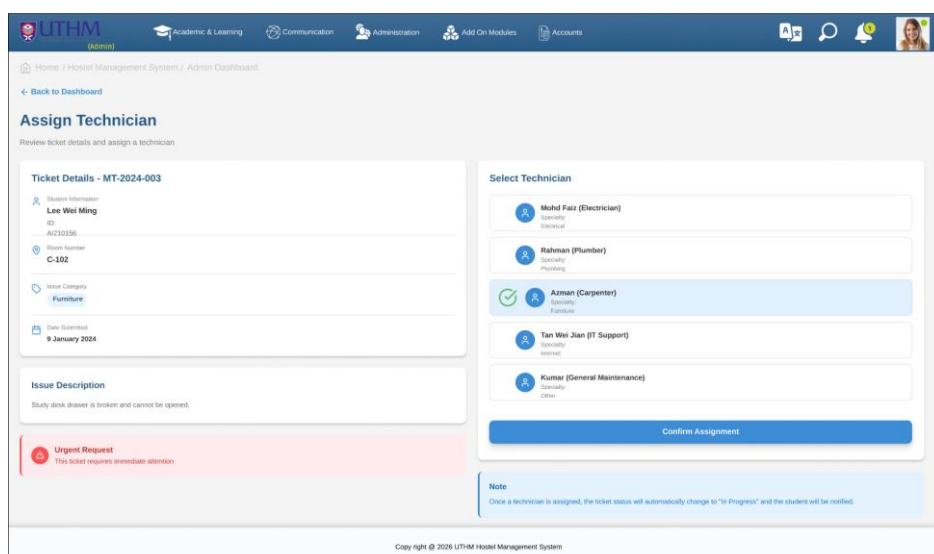


Figure 4.70 Assign Technician Page

Figure 4.70 demonstrates the workflow for assigning personnel to a specific task. The interface provides fields for the administrator to select a specific technician. In this example, the administrator is assigning the technician "Azman" to the maintenance request submitted by student "Lee Wei Ming."

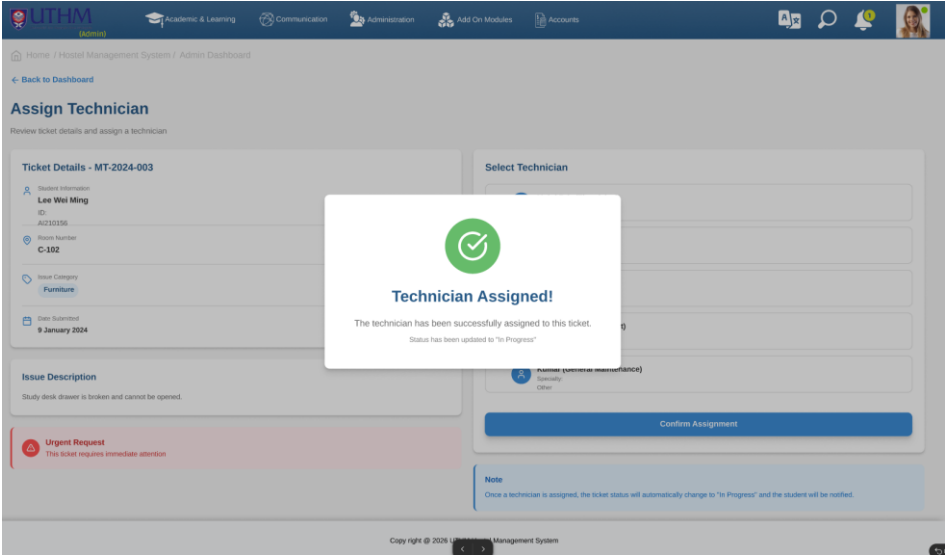


Figure 4.71 Pop-up Message: Successfully Assign the Technician

Figure 4.71 displays the system confirmation message. This pop-up ensures the administrator that the technician assignment has been successfully recorded in the database.

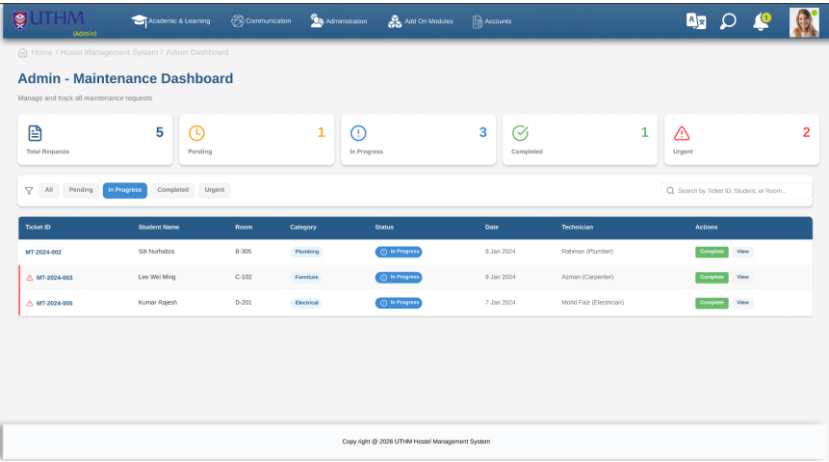


Figure 4.72 Status Update from Pending to In Progress

Figure 4.72 shows the immediate update to the ticket status. Once the technician is assigned, the system automatically changes the status of the request from "Pending" to "In Progress," indicating that action is being taken.

iii. Update Repair Status

The screenshot displays the 'Update Repair Status' interface. At the top, there's a navigation bar with UTHM logo and various menu items. Below it, a breadcrumb trail shows 'Home / Hostel Management System / Admin Dashboard'. The main heading is 'Update Repair Status' with a subtext 'Complete the maintenance request and add remarks'. The page is divided into several sections: 'Ticket Details - MT-2024-003' showing student information (Lee Wei Ming, ID: A1210156, Room Number: C-102, Issue Category: Furniture), 'Issue Description' (Study desk drawer is broken and cannot be opened), 'Assigned Technician' (Azman (Carpenter), Maintenance Staff), and 'Current Status' (In Progress). A 'Mark as Completed' button is visible, along with a 'Completion Remarks' field and an 'Important' note at the bottom.

Figure 4.73 Update Repair Status Page

Figure 4.73 illustrates the interface used to finalize a maintenance job. Once the physical repair is done, the administrator uses this screen to update the ticket status.

This screenshot shows the same 'Update Repair Status' page as Figure 4.73, but with a modal pop-up message in the center. The message has a green checkmark icon and reads: 'Ticket Completed! The maintenance request has been marked as completed. The student will be notified about the completion.' The background content of the page is dimmed.

Figure 4.74 Pop-Up Message: Ticket Completed

Figure 4.74 shows the success notification that appears after the administrator submits the update. This confirms that the system has processed the completion of the repair work.

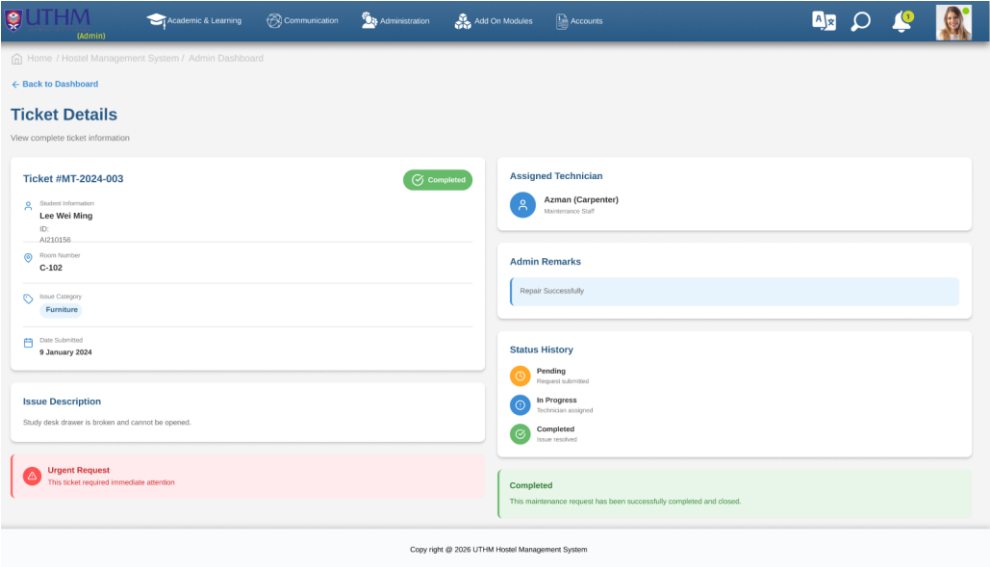


Figure 4.75 Status Changed to Completed

Figure 4.75 highlights the specific record for student "Lee Wei Ming," showing that the status column has now been updated to "Completed."

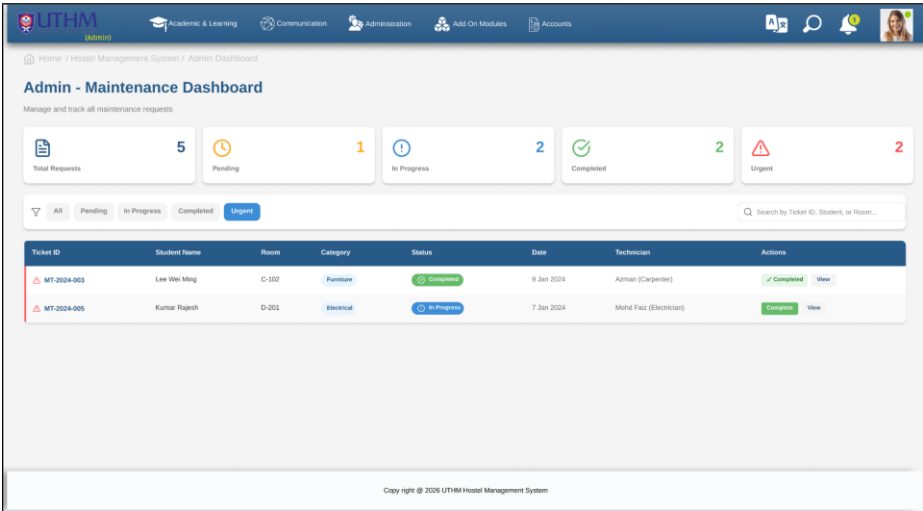


Figure 4.76 Latest Dashboard Status

Figure 4.76 depicts the updated main dashboard view. The completed ticket is now reflected in the overall list, ensuring the administrator has a real-time view of all resolved and ongoing maintenance activities.

5. Hostel Fees Monitoring Module

i. Hostel Fee Payment Report and Reminders Dispatch

This administrative interface provides an overview for monitoring and managing hostel fee payments. It features summary cards displaying the total student count, the number of paid and unpaid students and the total outstanding balance. Administrators can filter student lists by payment status (All/ Paid/ Unpaid) and utilize the "Send Reminder to All Unpaid" button to send automated notifications to students who have not yet settled their fees.

Hostel Fee Payment Report
Monitor and manage student hostel fee payments

Hostel Fee Monitoring

Total Students: 100
Paid Students: 60
Unpaid Students: 40
Total Outstanding: RM 31080.00

Filters: All Students | Paid Only | Unpaid Only

Search by name, matrix no, or IC...

Matrix No.	Student Name	IC Number	Hostel	Total Fees	Outstanding	Status	Action
A1210009	ABDUL BINTI KARIM	950517732893	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 0.00	PAID	
A1210065	HAKIMI BINTI RAHMAN	950315803556	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 777.00	UNPAID	
A1210083	AISYAH BINTI AZIZ	981209560389	KOLEJ TUN AMINAH	RM 777.00	RM 777.00	UNPAID	
A1210090	NADIA BIN KARIM	950506649678	KOLEJ TEMENGGUNG IBRAHIM	RM 777.00	RM 777.00	UNPAID	

Payment Reminder
Send payment reminder to 40 students who haven't paid their hostel fees yet.

[Send Reminder to All Unpaid](#)

Copy right © 2021 LMS Dashboard

Figure 4.77 The List of All Students



Admin

Academic & Learning

Communication

Administration

Add On Modules

Accounts

Dashboard

Home / Hostel Management System / Hostel Fee Payment Report

Hostel Fee Payment Report

Monitor and manage student hostel fee payments

View Room Availability

+ Allocate Room

Update Room Assignment

Hostel Fee Monitoring

Total Students

100

Paid Students

60

Unpaid Students

40

Total Outstanding

RM 31080.00

All Students

Paid Only

Unpaid Only

Search by name, matrix no, or IC...

Matrix No.	Student Name	IC Number	Hostel	Total Fees	Outstanding	Status	Action
AI210009	ABDUL BINTI KARIM	950517732893	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 0.00	PAID	
AI210001	AINA BINTI TAIB	991020387304	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 0.00	PAID	
AI210019	AZRI BINTI TAIB	990701847259	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 0.00	PAID	
AI210054	MUHAMMAD BIN YUSOF	990619791546	KOLEJ TUN FATIMAH	RM 777.00	RM 0.00	PAID	
AI210037	AMIRA BINTI ZAINAL	971103676993	KOLEJ TEMENGGUNG IBRAHIM	RM 777.00	RM 0.00	PAID	
AI210011	NURHAI IZA BINTI ALI	990813371412	KOLEJ TUN FATIMAH	RM 777.00	RM 0.00	PAID	

Copy right © 2021 LMS Dashboard



Figure 4.78 The Summarized List of Paid Students



Admin

Academic & Learning

Communication

Administration

Add On Modules

Accounts

Search

Refresh

Notification



Homepage / Hostel Management System / Hostel Fee Payment Report

Hostel Fee Payment Report

Monitor and manage student hostel fee payments

View Room Availability

+ Allocate Room

Update Room Assignment

Hostel Fee Monitoring

Total Students

100

Paid Students

60

Unpaid Students

40

Total Outstanding

RM 31080.00

All Students

Paid Only

Unpaid Only

Search by name, matrix no, or IC...

Matrix No.	Student Name	IC Number	Hostel	Total Fees	Outstanding	Status	Action
AI210062	ARIFF BIN KARIM	970206649982	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 777.00	UNPAID	
AI210061	MOHAMAD BINTI HAMID	991201655450	KOLEJ TUN HUSSEIN ONN	RM 777.00	RM 777.00	UNPAID	
AI210064	SHAFIQ BIN ISMAIL	951019123538	KOLEJ TEMENGGUNG IBRAHIM	RM 777.00	RM 777.00	UNPAID	
AI210068	AFIQ BIN AHMAD	970414519452	KOLEJ TUN DR ISMAIL	RM 777.00	RM 777.00	UNPAID	
AI210071	AZRI RINTI ABDULLAH	990318886125	KOLEJ TEMENGGUNG IBRAHIM	RM 777.00	RM 777.00	UNPAID	

Payment Reminder

Send payment reminder to 40 students who haven't paid their hostel fees yet.

Send Reminder to All Unpaid

Copy right © 2021 LMS Dashboard

Figure 4.79 The Summarized List of Unpaid Students

4.7 Summary

The suggested Hostel Management System for Universiti Tun Hussein Onn Malaysia (UTHM) was thoroughly analyzed and designed in this chapter. The analysis started with a summary of the current system environment and UTHM's organizational structure, emphasizing important problems like data silos, manual processes, system fragmentation, and poor subsystem integration. These difficulties demonstrated the necessity of an organized Enterprise Architecture (EA) strategy to enhance operational effectiveness, data consistency, and coordination.

To illustrate the enhancements provided by the integrated solution, a comparison between the existing and suggested systems was carried out based on the identified issues. In order to facilitate real-time data access, automated workflows, and logical integration with key systems like the Student Information System (SMP) and the Finance system, the proposed system centralizes hostel-related operations.

The system requirements, which form the basis for system development and include both functional and non-functional requirements, were also defined in this chapter. The Enterprise Architecture model and system architecture were then used to present the system design, showing how business processes, applications, data, and technology layers interact within a single ecosystem. System functionality and user interactions were clearly documented using detailed UML diagrams that included use case, activity, sequence, database, and interface designs.

All things considered, this chapter provides a strong technical and architectural basis for the Hostel Management System, guaranteeing that the suggested solution complies with enterprise architecture principles and facilitates effective, integrated residential management at UTHM.

CHAPTER 5

CONCLUSION

5.1 Introduction

This chapter presents the overall conclusion of the project. It summarizes the key contributions and achievements of the study, identifies the main constraints encountered, outlines possible directions for future improvements, and provides a final summary of the work. The aim of this chapter is to reflect on the outcomes of the project and highlight its significance in supporting a structured and integrated university information system.

5.2 System Contribution / Achievements

The development of the Hostel Management System has successfully addressed critical operational gaps within UTHM's residential management by transitioning from fragmented, manual methods to a centralized digital solution. By consolidating disjointed processes such as room application, fee monitoring, and facility maintenance into a single unified platform, the system effectively eliminates the previous reliance on tools like Google Forms and Excel. This centralization ensures that all hostel-related data is stored in a structured, accessible format, significantly reducing data redundancy and the risk of administrative errors that plagued the legacy workflow.

A major achievement of the system is the automation of core operational tasks, particularly room allocation and facility management. By digitally verifying student eligibility through simulated integration with the Student Information System (SMP) and checking payment status via the Finance System, the project demonstrates how

administrative bottlenecks can be removed to ensure fair and policy-compliant room assignments. Furthermore, the implementation of the Maintenance Request Module has revolutionized facility management by providing students with a direct channel to report issues, while allowing admins to track repair status in real-time. This shift from manual logging to digital tracking directly contributes to a better quality of life for students by ensuring faster resolution of facility defects.

Finally, the system delivers significant value by enhancing real-time visibility for administrators and improving accessibility for students. The solution empowers residential college admins with immediate access to critical operational data, such as real-time occupancy rates and pending maintenance tickets. This enables proactive decision-making that was previously difficult due to a lack of analytics. Simultaneously, the development of a high-fidelity prototype using Figma visualizes a modern, mobile-responsive interface that aligns with UTHM's "Smart Campus" initiative. This interactive design demonstrates how students can conveniently perform essential tasks. For example, check-in/out via QR code and fee checking directly from their devices, proving the feasibility of a fully digital student experience.

5.3 System Constraints

Despite the successful development of the Hostel Management System prototype, there were several constraints encountered throughout the project that limited the scope and depth of implementation. These constraints are related to integration and project boundaries rather than system design quality.

Firstly, one key constraint is the lack of direct access to UTHM's live enterprise systems such as the Student Information System (SMP), Finance/Bursary System, Staff Master, and MyUTHM authentication services. As a result, system integration was simulated conceptually using assumed data structures and workflows instead of real time data exchange. While this approach supports architectural reasoning, it restricts the ability to demonstrate full end-to-end automation.

Another significant constraint is the limited development scope and timeframe

of the project. Due to academic scheduling and resource constraints, the system was developed as a high-fidelity prototype rather than a fully deployed operational system. Certain advanced functionalities, such as real-time payment processing, automated fine deduction, and system-wide analytics dashboards, were therefore not fully implemented.

Furthermore, security and compliance limitations were also present during development. Features such as role-based access control, audit logging, and data encryption were designed at a conceptual level but not implemented using enterprise-grade security mechanisms. This limitation is largely due to the absence of institutional security infrastructure and restricted access to production-level environments.

Finally, the system is constrained due to its dependency on user adoption and data accuracy. The effectiveness of digital check-in, maintenance reporting, and fee monitoring relies heavily on students and staff consistently using the system as intended. Without proper training, policy enforcement and organizational change management may not be immediately realized.

5.4 Future Works

Based on the current development of the Hostel Management System prototype and the defined project scope, several areas have been identified for future enhancement to further support UTHM's digital transformation goals. Future development should focus on the following aspects:

- (a) Build a Unified Mobile Integration to improve student accessibility. The Hostel Management Subsystem should be fully integrated into the existing MyUTHM mobile application ecosystem too that could provide a seamless "one-stop" experience for all residential services.
- (b) Apply and include advanced analytics and predictive maintenance into future development. It leverages the centralized data layer to implement AI-driven

analytics, allowing administrators to predict rooms' facilities maintenance needs before failures occur and better forecast room demand for upcoming new semesters.

- (c) Establish a full monetary integration for future system. The current prototype only conceptualizes the actual fee monitoring flow. For future iterations, we could implement real-time integration with university banking gateways to facilitate actual monetary transactions and automated digital receipt issuance.
- (d) Use of AI-Powered Virtual Assistant. A chatbot or virtual assistant could be integrated into UTHM website to provide 24/7 support for students regarding room applications, payment inquiries or if they have any common questions.
- (e) Develop blockchain for financial integrity. Implementing blockchain technology for the Data Layer could ensure that all hostel fee transactions and payment histories are immutable and unbreakable. This could further enhance the security and transparency of student financial records.

5.5 Summary

In conclusion, this project has successfully designed a Hostel Management System prototype to address the limitations of the existing manual hostel management processes at UTHM. By centralizing key functions such as room application, fee monitoring and maintenance reporting into a single platform, the project demonstrates how digital solutions can improve operational efficiency, data organization and communication between students and admin staff.

Although the system was developed as a phototype with simulated integrations and a limited implementation scope, it fulfils the main objectives of the project. The identified system constraints and proposed future enhancement provide a clear direction for future improvement. Overall, this project serves as a valuable learning experience in system analysis, design and prototyping while also offering a practical foundation for the future development of a full-scale hostel management system.

Figma Prototype Link:

<https://www.figma.com/design/qh6fpmZEPd7t6pcoYJ7l8x/Hostel-Management-System?node-id=111-686&t=txy5gGB4ttaz7eQc-0>

Youtube Video Link:

<https://youtu.be/LlGWjuVWeYE>

GITHUB Link:

<https://github.com/pohlokyee/University-Enterprise-Architecture-and-System-Integration-Design>

REFERENCES

- Dumitriu, D., & Popescu, M. A. (2020). Enterprise Architecture Framework design in IT Management. *Procedia Manufacturing*, 46, 932–940.
<https://doi.org/10.1016/j.promfg.2020.05.011>
- Brée, T., & Karger, E. (2022). Challenges in enterprise architecture management: Overview and future research. *Journal of Governance and Regulation*, 11(2, special issue), 355–367. <https://doi.org/10.22495/jgrv11i2siart15>
- Supriadi, H., Kom, M., & Amalia, E. (n.d.). University's Enterprise Architecture Design using Enterprise Architecture Planning (EAP) based on the Zachman's Framework approach. <https://eric.ed.gov/?id=EJ1215571>
- Meutia, N. N. S., Sulistiyani, E., Budiarti, R. P. N., & Sari, R. (2022). Enterprise Architecture Framework in Higher Education: Systematic Literature Review. *Applied Technology and Computing Science Journal*, 5(2), 33–39.
<https://doi.org/10.33086/atcsj.v5i2.3751>
- Batmetan, J. R. (2022). Model enterprise architecture for information technology services in universities. *International Journal of Information Technology and Education*, 1(4), 18–34. <https://doi.org/10.62711/ijite.v1i4.73>
- Abdul Kahar, M. S., Md Shahrum, N., Mustafa, N. S., & Mohd Basari, N. (2017). Investigating the influence of facility management on students' quality of life in student residential hall in Universiti Utara Malaysia. Symposium on Technology Management & Logistics (STML–Go Green) 2016, Universiti Utara Malaysia.
- Hashim, I. H. (2025). *Facilities Management Digitalisation Model: A Systematic Literature Review*. School of Technology Management & Logistics, Universiti Utara Malaysia.
- Redzuan, R. B. (2016). Universiti Teknologi MARA e-Kolej: The Relationship of User Satisfaction and Usage to Individual Performance (Doctoral dissertation, Universiti Teknologi MARA).
- Ulmi, U., Putra, A. P. G., Ginting, Y. D. P., Laily, I. L., Humani, F., & Ruldeviyani, Y. (2020). Enterprise Architecture Planning for Enterprise University

Information System using the TOGAF Architecture Development Method.
IOP Conference Series Materials Science and Engineering, 879(1), 012073.
<https://doi.org/10.1088/1757-899x/879/1/012073>

LIST OF PUBLICATIONS

Journal Articles

- Qasem, S. N., Shamsuddin, S. M., Hashim, S. Z. M., Darus, M., & AlShammari, E. (2013). Memetic multiobjective particle swarm optimization based radial basis function network for classification problems. *Information Sciences*, 239, 165–190. <https://doi.org/10.1016/j.ins.2013.03.021>. (Q1, IF: 4.305)
- Qasem, S. N., & Shamsuddin, S. M. (2011). Radial basis function network based on time variant multi-objective particle swarm optimization for medical diseases diagnosis. *Applied Soft Computing*, 11(1), 1427–1438. <https://doi.org/10.1016/j.asoc.2010.04.014>. (Q1, IF:3.907)
- Shen, L. W., Asmuni, H., & Weng, F. C. (2015). A modified migrating bird optimization for university course timetabling problem. *Jurnal Teknologi*, 72(1), 89–96. <https://doi.org/10.11113/jt.v72.2949>. (Indexed by SCOPUS)

Conference Proceedings

- Muhamad, W. Z. A. W., Jamaludin, K. R., Ramlie, F., Harudin, N., & Jaafar, N. N. (2017). Criteria selection for MBA programme based on the mahalanobis Taguchi system and the Kanri Distance Calculator. In 2017 IEEE 15th Student Conference on Research and Development (SCOREd) (pp. 220–223). IEEE. <https://doi.org/10.1109/SCORED.2017.8305390>. (Indexed by SCOPUS).